АКАДЕМИЈА ТЕХНИЧКО-ВАСПИТАЧКИХ СТРУКОВНИХ СТУДИЈА
ОДСЕК НИШ

Završni master rad

Razvoj veb aplikacije za rezervaciju učionica

Predmet: Serverske tehnologije

Mentor:

dr Miloš Stojanović

Student:

Miloš Petrović ITS 0723

Septembar 2026.

АКАДЕМИЈА ТЕХНИЧКО-ВАСПИТАЧКИХ СТРУКОВНИХ СТУДИЈА
ОДСЕК НИШ

Završni master rad

Razvoj veb aplikacije za rezervaciju učionica

Predmet: Serverske tehnologije

Mentor:

dr Miloš Stojanović

Student:

Miloš Petrović ITS 0723

Članovi komisije:

1. _____

2. _____

3. _____

IZJAVA O AUTORSTVU

Potpisan/ a _____

(Ime studenta)

Broj indeksa _____

IZJAVLJUJEM

Da je završni master rad pod naslovom

_____

_____

_____

- Rezultat sopstvenog stručno istraživačkog rada,
- Da su rezultati korektno navedeni,
- Da nisam kršio/la autorska prava i koristio intelektualnu svojinu drugih lica.

U Nišu, _____

Ptopis Studenta

_____

ZAHVALNICA

Pre svega, želeo bih da izrazim svoju najdublju zahvalnost Bogu što me je vodio ovim putem i doveo do ovog dostignuća.

Duboko sam zahvalan i svojoj porodici na bezuslovnoj ljubavi, strpljenju, nepokolebljivoj podršci i stalnoj veri u mene tokom celog mog života i školovanja.

Želeo bih da uputim iskrenu zahvalnost svom mentoru na kontinuiranom vođenju, dragocenim savetima i punoj podršci tokom istraživanja i izrade ovog projekta. Najdublju zahvalnost dugujem i svim profesorima koji su podržali kako moje celokupno obrazovanje, tako i realizaciju ovog završnog rada.

Posebnu zahvalnost dugujem kolegama iz Shindiri Studija, čije mi je mentorstvo tokom prakse pomoglo da rastem i lično i profesionalno, kao i da akademsko znanje uspešno primenim u rešavanju realnih inženjerskih izazova.

Na kraju, ali ni manje važno, želim da zahvalim svim svojim prijateljima na emotivnoj podršci, ohrabrenju i veri u mene tokom ovog izuzetnog putovanja. Vaše prisustvo i pomoć, kako u životu tako i na ovom putu, značili su mi sve.

ABSTRACT

EduScheduler - Web Application for Educational Scheduling and Management

Miloš Petrović/ dr Miloš Stojanović

Keywords: Educational Scheduling, React 19, TypeScript, PHP 8.2, MariaDB, Excel Parsing, Web Application Architecture

The management and dissemination of academic timetables represent a critical operational challenge for modern educational institutions. Legacy workflows frequently rely on fragmented spreadsheet files, manual entry processes, and disconnected communication channels, leading to resource scheduling conflicts, administrative overhead, and poor information accessibility for students and faculty.

This master's thesis presents **EduScheduler**, a unified, full-stack web application engineered to centralize, streamline, and automate educational schedule management. The platform is designed around a decoupled multi-tier architecture, combining a responsive client application built with React 19, TypeScript, Vite, and TanStack Query with a high-performance PHP 8.2 and MariaDB backend database layer.

Key technical contributions of the EduScheduler platform include:

1. **Automated Spreadsheet Ingestion:** A server-side parsing pipeline capable of extracting, validating, and persisting legacy Excel schedule workbooks (.xlsx) within sub-second execution windows (~497 ms for datasets exceeding 340 entries).
2. **Interactive Schedule Visualization:** A highly optimized frontend grid component (ScheduleGrid) that leverages dynamic rowSpan DOM consolidation-achieving a ~35% reduction in active browser nodes—and memoized layout computations for fluid rendering.
3. **Robust Security and Access Control:** A dual-step authentication architecture backed by OWASP-aligned security practices, including parameterized database queries, context-

aware XSS sanitization, and role-based route protection for administrators, professors, and students.

Performance benchmarks and system evaluations confirm that EduScheduler delivers sub-millisecond query performance for complex schedule lookups while maintaining relational data integrity across multi-department academic structures. Finally, the thesis identifies existing architectural boundaries and outlines a roadmap for future enhancements, including asynchronous job queues, Docker containerization, and real-time WebSocket conflict notifications.

SAŽETAK

EduScheduler - Razvoj veb aplikacije za rezervaciju učionica

Miloš Petrović/ dr Miloš Stojanović

Ključne reči: Raspored nastave, React 19, TypeScript, PHP 8.2, MariaDB, Parsiranje Excel fajlova, Arhitektura veb aplikacija

Upravljanje i distribucija rasporeda nastave predstavlja ključni operativni izazov za savremene obrazovne ustanove. Tradicionalni procesi se često oslanjaju na fragmentisane tabelarne dokumente, ručni unos podataka i neefikasne kanale komunikacije, što dovodi do vremenskih preklapanja resursa, povećanog administrativnog opterećenja i otežanog pristupa informacijama za studente i nastavno osoblje.

U ovom master radu predstavljen je **EduScheduler**, integrisana veb aplikacija punog steka (*full-stack*) projektovana sa ciljem centralizacije, automatizacije i optimizacije procesa upravljanja rasporedom nastave. Sistem je zasnovan na višeslojnoj arhitekturi koja povezuje klijentsku aplikaciju izrađenu pomoću tehnologija React 19, TypeScript, Vite i TanStack Query sa pozadinskim servisima zasnovanim na jeziku PHP 8.2 i relacijskoj bazi podataka MariaDB.

Glavni tehnički doprinosi sistema EduScheduler obuhvataju:

1. **Automatizovani uvoz tabelarnih podataka:** Pozadinski servis za parsiranje i validaciju postojećih Excel fajlova rasporeda (.xlsx), koji omogućava obradu i upis kompleksnih skupova podataka u bazu u vremenu kraćem od jedne sekunde (~497 ms za preko 340 stavki).
2. **Interaktivni prikaz rasporeda:** Optimizovana komponenta interfejsa (ScheduleGrid) koja primenom dinamičkog spajanja ćelija (rowSpan) smanjuje broj aktivnih DOM čvorova u pregledaču za oko 35%, uz memorisane proračune rasporeda za brzo i fluidno iscrtavanje.
3. **Bezbednost i kontrola pristupa:** Dvoetapni mehanizam autentifikacije i autorizacije usklađen sa OWASP preporukama, koji uključuje pripremljene SQL upite, zaštitu od XSS napada i restriktivnu kontrolu pristupa zasnovanu na ulogama korisnika (administratori, profesori, studenti).

Empirijska evaluacija sistema potvrdila je visoke performanse izvođenja upita i stabilnost baze podataka pri radu sa složenim rasporedima. U radu su takođe identifikovana trenutna tehnička ograničenja i predloženi pravci budućeg razvoja, koji obuhvataju asinhronu obradu zadataka, Docker kontejnerizaciju i primenu WebSockets protokola za obaveštavanje o konfliktima u realnom vremenu.

Table of Contents

1. UVOD 1

1.1 KONTEKST I MOTIVACIJA 2
1.2 DEFINICIJA PROBLEMA 5
1.3 CILJEVI SISTEMA 6
1.4 OBIM I OGRANIČENJA SISTEMA 9
1.5 CILJNE GRUPE KORISNIKA I NJIHOVE POTREBE 10
1.6 PREGLED STRUKTURE RADA 11

2. IZBOR TEHNOLOGIJA 12

2.1 FRONTEND TEHNOLOGIJE 14
2.1.1 React 19 sa TypeScript-om 15
2.1.2 Vite kao alat za prevođenje i razvojni server 17
2.1.3 React Router DOM v7 za rutiranje na strani klijenta 18
2.1.4 TanStack React Query za upravljanje stanjem servera 19
2.1.5 React Hook Form i Zod za validaciju formulara 21
2.1.6 Styled-Components za stilizovanje (CSS-in-JS) 24
2.1.7 React Toastify za obaveštenja korisnika 24
2.1.8 PHPMailer za slanje e-pošte (Integracija sa backend-om) 25
2.2 BACKEND TEHNOLOGIJE 27
2.2.1 PHP 8.2 sa PDO i MySQLi ekstenzijama 27
2.2.2 PHP-DI kontejner (Ubrizgavanje zavisnosti) 28
2.2.3 MySQL / MariaDB 10.4 kao server baze podataka 28
2.2.4 Autentifikacija zasnovana na PHP sesijama 29
2.2.5 RESTful API arhitektura (JSON preko HTTP-a) 29
2.2.6 CORS konfiguracija za zahteve sa različitih porekla 30
2.3 TEHNOLOGIJE ZA TESTIRANJE I BEZBEDNOST 30
2.3.1 PHPUnit testni skup 30
2.3.2 Bezbednosni mehanizmi usklađeni sa OWASP standardima 31
2.4 RAZVOJNA I DEPLOYMENT INFRASTRUKTURA 32
2.4.1 XAMPP (Apache + MariaDB + PHP) u Windows okruženju 32
2.4.2 Visual Studio Code kao razvojno okruženje (IDE) 32
2.4.3 Git i arhitektura repozitorijuma 33
2.4.4 phpMyAdmin za administraciju baze podataka 33

2.4.5 npm i Node.js za klijentske pakete 33
2.5 SAŽETAK TEHNOLOŠKOG STEAK-A 34
3. ARHITEKTURA SISTEMA I DIZAJN BAZE PODATAKA 36
3.1 VISOKONIVOJSKA ARHITEKTURA SISTEMA (HIGH-LEVEL SYSTEM ARCHITECTURE) 36
3.1.1 Klijent-server model (Client-Server Model) 36
3.1.2 Petostepena hijerarhija uloga (Guest, Student, Professor, Admin, Superadmin) 37
3.1.3 Troslojna arhitektura (Presentation, Business Logic, Data) 38
3.1.4 Komunikacija frontenda i backend-a putem REST API-ja 38
3.1.5 Tok autentifikacije i upravljanje sesijama (Session-Based Authentication Flow) 40
3.1.6 Jedinstveni endpoint za prijavu i mehanizam razrešavanja uloga 42
3.2 ARHITEKTURA FRONTEND-A 44
3.2.1 Hijerarhija komponenata i organizacija datoteka 44
3.2.2 Upravljanje stanjem: React Context API + TanStack Query (State Management) 45
3.2.3 Strategija rutiranja i zaštita ruta 47
3.2.4 Apstrakcija API sloja 47
3.2.5 Tipski sigurni modeli podataka sa TypeScript interfejsima 48
3.3 ARHITEKTURA BACKEND-A 50
3.3.1 Struktura direktorijuma i razdvajanje nadležnosti 50
3.3.2 Sloj konekcije sa bazom podataka 51
3.3.3 Inicijalizacija i pomoćne funkcije (functions/init.php, functions.php) 51
3.3.4 Organizacija API endpoint-ova (api/ Direktorijum) 52
3.3.5 Skripte administratorskog panela (admin/ Direktorijum) 53
3.3.6 Pipeline za uvoz Excel rasporeda (app/Services/) 53
3.3.7 Mehanizam za razrešavanje imena nastavnika (TeacherNameMatcher) 54
3.3.8 Upravljanje otkazanim časovima (Cancelled Classes) 55
3.3.9 Sistem šablona e-mail obaveštenja (Email Notification System) 55
3.4 DIZAJN BAZE PODATAKA 56
3.4.1 Pregled odnosa entiteta (ER dijagram) 56
3.4.2 Specifikacija strukture 12 tabela baze podataka 58
4. DETALJI IMPLEMENTACIJE 61
4.1 FRONTEND IMPLEMENTACIJA 61

4.1.1 Ulazna tačka aplikacije i korensko stablo (App.tsx) 61
4.1.2 Konfiguracija rutiranja (Routes.tsx) 62
4.1.3 Kontekst autentifikacije korisnika (AuthContext.tsx, useAuth.tsx) 63
4.1.4 Kontekst autentifikacije administratora (AdminAuthContext.tsx, useAdminAuth.ts) 64
4.1.5 API sloj i mrežna komunikacija (api.ts, adminApi.ts, publicApi.ts) 64
4.1.6 Komponente stranica 65
4.1.7 Biblioteka komponenti i korisnički interfejs 68
4.1.8 Upravljanje stanjem servera pomoću TanStack Query-ja 73
4.1.9 Validacija formi pomoću Zod šema 75
4.1.10 Interfejs za otkazane časove 76
4.1.11 Interfejs za upravljanje šablonima e-pošte 76
4.2 BACKEND IMPLEMENTACIJA 77
4.2.1 Ulazne tačke i upravljanje sesijama 77
4.2.2 Autorizacioni endpoint (api/check_auth.php) 78
4.2.3 Kompletan pregled svih 30 REST API endpoint-a 79
4.2.4 Skripte administratorske table (24 skripte) 82
4.2.5 Cevovod za uvoz Excel datoteka (ScheduleImportService) 84
4.2.6 Servis za razrešavanje imenâ nastavnika (TeacherNameMatcher) 89
4.2.7 Implementacija otkazanih časova 90
4.3 MIGRACIJE BAZE PODATAKA (13 SQL DATOTEKA) 90
4.4 INTEGRACIJA ZOD / REACT HOOK FORM BIBLIOTEKA 91
5. BEZBEDNOST SISTEMA I EVALUACIJA PERFORMANSI 93
5.1 BEZBEDNOST I ARHITEKTURA AUTENTIFIKACIJE 93
5.1.1 Integritet autentifikacije i dvostepena verifikacija uloga 93
5.1.2 Politike heširanja i skladištenja lozinki 96
5.1.3 Kriptografski mehanizam za oporavak naloga 98
5.1.4 Mitigacija OWASP ranjivosti 100
5.2 PHPUNIT SKUP TESTOVA I VERIFIKACIJA API ENDPOINT-A 101
5.2.1 PHPUnit testni skup (9 modula) 101
5.2.2 Verifikacija REST API endpoint-a (30 endpoint-a) 102
5.2.3 Trenutna ograničenja u procesu testiranja 103
5.3 EVALUACIJA PERFORMANSI I OPTIMIZACIJA 103
5.3.1 Benchmark obrade masovnog uvoza (Bulk Import) 103

5.3.2 Indeksiranje baze podataka i optimizacija upita 104
5.3.3 Efikasnost klijentskog renderovanja (React 19) 106
6. ZAKLJUČAK, TEHNIČKA OGRANIČENJA I PREPORUKE ZA DALJI RAZVOJ APLIKACIJE 108
6.1 PREGLED TEHNIČKIH DOSTIGNUĆA 108
6.1.1 Savremena arhitektura frontend-a 108
6.1.2 Robustan backend i sloj baze podataka 110
6.1.3 Dvostepena autentifikacija i autorizacija 112
6.1.4 Mitigacija ranjivosti usklađena sa OWASP standardima 112
6.2 OGRANIČENJA SISTEMA 113
6.2.1 Hardkodovani SMTP kredencijali 113
6.2.2 Razdvojena struktura direktorijuma bez jedinstvenog repozitorijuma 113
6.2.3 Ručno upravljanje šemom baze podataka 114
6.2.4 Hardkodovani kredencijali za bazu podataka 115
6.2.5 Nepotpuna pokrivenost testovima 115
6.2.6 Konfiguracija prilagođena isključivo lokalnom razvoju 116
6.3 PREPORULE ZA DALJI RAZVOJ APLIKACIJE 116
6.3.1 Konfiguracija okruženja pomoću vlucas/phpdotenv 116
6.3.2 Kontejnerizacija pomoću Docker-a 117
6.3.3 Automatsko End-to-End testiranje pomoću Playwright i PHPUnit alata 119
6.3.4 Detekcija konflikata u realnom vremenu putem WebSocket-a ili Server-Sent Events 119
6.3.5 Integracija SSO autentifikacije putem Microsoft Teams 121
6.3.6 Unapređenje bezbednosti za produkciono okruženje 121
7. ZAKLJUČAK 122
LITERATURA 123
BIOGRAFIJA 125

1. Uvod

Savremeni obrazovni ekosistem zahteva efikasnu administrativnu koordinaciju, pouzdane kanale komunikacije i robusnu digitalnu infrastrukturu kako bi se podržale svakodnevne operacije akademskih ustanova. Kako univerzitet i škole rastu u obimu - primajući sve veći broj studenata raspoređenih po različitim studijskim programima, godinama studija i specijalizovanim predmetima - tradicionalne metode upravljanja institucionalnim rasporedom i korisničkim nalozima postaju neodržive. Ručno kreiranje rasporeda, fragmentisano skladištenje podataka i netransparentni radni tokovi komunikacije neizbežno dovode do operativne neefikasnosti, konflikata u terminima i ugrožene bezbednosti podataka.

Ovaj master rad prikazuje dizajn, razvoj i implementaciju sistema **EduScheduler**, sveobuhvatne *full-stack* web aplikacije zasnovane na ulogama, kreirane sa ciljem da reši složene izazove planiranja i upravljanja obrazovnim rasporedom. EduScheduler premošćuje jaz između administrativnog nadzora i dostupnosti za krajnje korisnike, pružajući centralizovano digitalno okruženje koje ujedinjuje administratore, nastavno osoblje i studente pod jedinstvenom i bezbednom arhitekturom. U svojoj osnovi, EduScheduler zamenjuje zastarele prakse podložne greškama savremenim web tehnologijama. Sistem nudi dvostruki režim generisanja rasporeda koji podržava masovni uvoz iz Excel tabela, kao i interaktivni urednik za ručno upravljanje mrežom rasporeda u realnom vremenu, uz sprovođenje stroge kontrole pristupa i vidljivosti specifične za svaku ulogu. Pored toga, aplikacija integriše bezbednosne prakse prilagođene produkcionim uslovima, kao što su autorizacija zasnovana na serverskim sesijama i kriptografski bezbedni mehanizmi za oporavak lozinke putem tokena.

Naredna poglavlja ovog rada detaljno opisuju konceptualne osnove, primenjeni tehnološki stek, arhitektonski dizajn frontend i backend komponenti, kao i rigorozne bezbednosne mere primenjene kako bi se osiguralo da EduScheduler služi kao pouzdano, skalabilno i bezbedno rešenje za savremene obrazovne ustanove.

1.1 Kontekst i motivacija

Savremene obrazovne ustanove (univerziteti, fakulteti i škole) suočavaju se sa izuzetno složenim organizacionim izazovom: planiranjem i rasporedom učionica, koordinacijom nastavnog osoblja, upravljanjem grupama studenata na više studijskih programa i godina studija, kao i obradom rezervacija sala i vanrednih izmena u rasporedu. Kada se ovi zadaci obavljaju ručno ili pomoću fragmentisanih, zastarelih alata, oni oduzimaju značajne administrativne resurse, podložni su ljudskim greškama i često dovode do konflikata u rasporedu koji remete akademski proces i za studente i za nastavnike.

Tipičan univerzitetski departman može nadgledati desetine ili čak stotine učionica i amfiteatara različitih kapaciteta i opremljenosti, od kojih je svaka potencijalno dostupna za različite vrste nastavnih aktivnosti - predavanja, računarske vežbe i laboratorijski rad. Profesori imaju individualna nastavna opterećenja raspoređena na više studijskih programa i grupa, od kojih svaka ima sopstvenu strukturu kurikuluma. Studenti su upisani na specifične studijske programe (smer) na određenim godinama studija (godina), te njihov nedeljni raspored mora biti filtriran tako da prikazuje samo predmete relevantne za njihov konkretan program i godinu. Pored toga, proces planiranja mora uzeti u obzir ograničenja kao što su opremljenost sala (projektor, računarska laboratorija, specijalizovana oprema), dostupnost vremenskih termina i izbegavanje preklapanja časova u istoj učionici ili za istog profesora.

Istorijski posmatrano, mnoge obrazovne ustanove su se oslanjale na štampane rasporede, tabele (npr. Microsoft Excel) ili jednostavne aplikacije sa bazama podataka. Iako ove metode mogu biti adekvatne za rad u manjem obimu, one pokazuju značajna ograničenja kada se primene na savremena akademska okruženja:

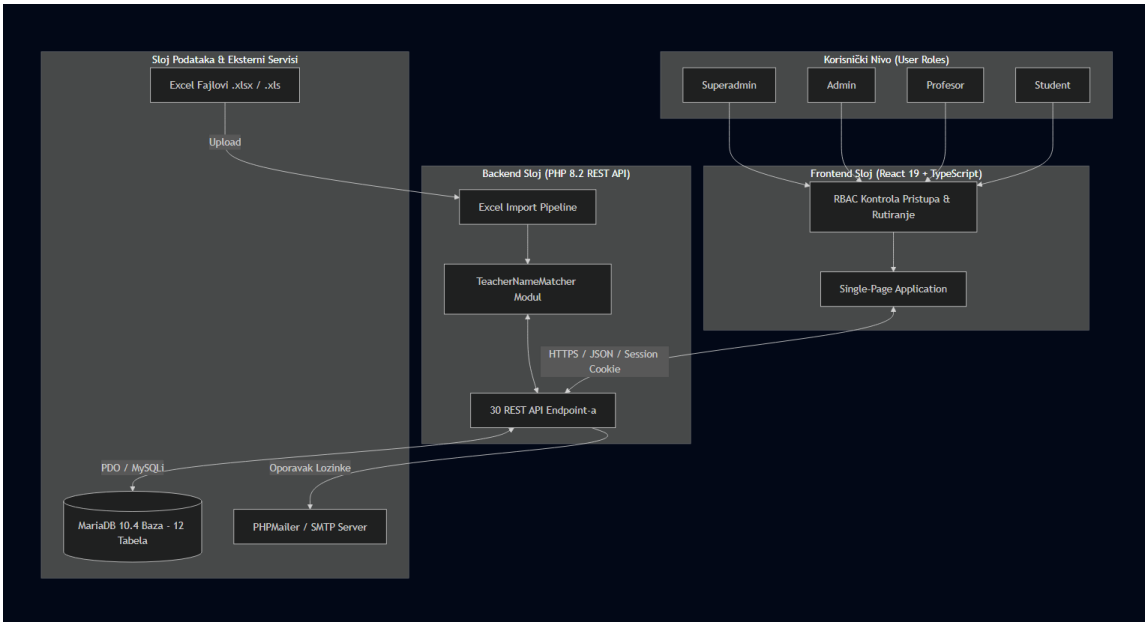
- **Nedostatak saradnje u realnom vremenu:** Kada dođe do promene u rasporedu - poput sprečenosti profesora zbog bolesti pa je potrebna zamena, ili kada sala postane nedostupna - ažurirane informacije moraju se ručno preneti svim pogođenim stranama. Ovo kašnjenje stvara prostor za zabunu tokom kojeg studenti mogu otići u pogrešnu učionicu, a profesori započeti nastavu u već zauzetoj sali.
- **Inherentna podložnost greškama:** Preklapanje časova u istoj sali, duplirane rezervacije profesora u istim vremenskim terminima i unosi u raspored koji ne odgovaraju ograničenjima programa ili godine studija mogu se lako prevideti i ostati neprimećeni sve dok ne izazovu konkretne zastoje u nastavi.

- **Nefleksibilni administrativni radni tokovi:** Upravljanje osnovnim podacima - dodavanje novih profesora ili studenata, ažuriranje raspodele po studijskim programima, kreiranje ili uklanjanje učionica - često je komplikovano i odvojeno od samog prikaza rasporeda.
- **Ad-hoc kanali komunikacije:** Rutinski zadaci upravljanja korisnicima, kao što je oporavak lozinke, često se obavljaju nestandardizovano. U nekim slučajevima, administrator mora ručno resetovati lozinku ili preneti privremene pristupne podatke preko nebezbednih kanala, što unosi nepotrebne bezbednosne rizike.

Ova zapažanja direktno motivišu razvoj sistema **EduScheduler** - savremene, centralizovane web aplikacije za planiranje i upravljanje rasporedom, dizajnirane da reši navedene probleme putem jedinstvene digitalne platforme zasnovane na ulogama. Glavna motivacija je zamena fragmentisanih, ručnih ili papirnih metoda jedinstvenim sistemom koji obezbeđuje vidljivost u realnom vremenu, automatsku detekciju konflikata, strukturirane administrativne radne tokove i bezbedne mehanizme za samostalni oporavak naloga.

Izbor web modela aplikacije je nameran: on elimiše potrebu za instalacijom softvera kod svakog pojedinačnog korisnika, omogućava pristup sa bilo kog uređaja sa web pretraživačem i olakšava centralizovano održavanje i ažuriranje bez potrebe za izmenama na klijentskoj strani. [1]

Aplikacija je izgrađena korišćenjem savremenih, dobro prihvaćenih tehnologija: React 19 sa TypeScript-om na frontend-u, PHP 8.2 na backend-u, MariaDB 10.4 kao server baze podataka i Apache kao web server (putem XAMPP distribucije). Sistem obuhvata 30 backend API ruta (endpoints), 12 tabela u bazi podataka, približno 24 stranice administrativnog interfejsa i jednostraničnu React aplikaciju (Single-Page Application) sa kontrolom pristupa zasnovanom na četiri različite korisničke uloge: Superadmin, Admin, Profesor i Student. Visokonivoalna arhitektura sistema i interakcija između navedenih slojeva prikazana je na Slici 1.1.



Slika 1.1: Arhitektura EduScheduler sistema i tok podataka između slojeva

1.2 Definicija problema

Obrazovne ustanove se suočavaju sa nekoliko međusobno povezanih izazova pri upravljanju akademskim rasporedom.

1. Ručno kreiranje i izmena rasporeda je podložno greškama: jedan previdjeti konflikt između istovremenih nastavnih obaveza profesora i duplirane rezervacije učionice može poremetiti čitavu nedelju nastave.
2. Podaci o rasporedu mogu poticati iz heterogenih formata - Excel tabela koje održavaju različita odeljenja, od kojih svako koristi sopstvene konvencije imenovanja i kvalitet podataka - što otežava automatsku integraciju.
3. Usklađivanje imena nastavnika onako kako se pojavljuju u uvezenim datotekama (koje mogu koristiti ćirilično pismo, titule ili skraćenice) sa njihovim kanonskim zapisima u bazi podataka ustanove zahteva netrivijalan proces uparivanja koji se ne može svesti na jednostavan upit preko stranog ključa.
4. Izmene u odobrenim rasporedima (kao što su otkazivanje ili zamena čas) moraju se pratiti i pravovremeno preneti pogođenim stranama.
5. Oporavak lozinke za korisnike koji zaborave svoje pristupne podatke mora se odvijati bezbedno i bez potrebe za ručnom intervencijom administratora.

Nijedan jedinstven, široko prihvaćen i pristupačan softverski proizvod ne rešava svih pet navedenih problema u kontekstu jedne obrazovne ustanove. EduScheduler je stoga razvijen kao prilagođeno rešenje koje objedinjuje uvoz rasporeda, razrešavanje imena nastavnika, praćenje otkazanih časova, interaktivnu vizuelizaciju rasporeda i bezbedan oporavak naloga u jednu koherentnu web aplikaciju.

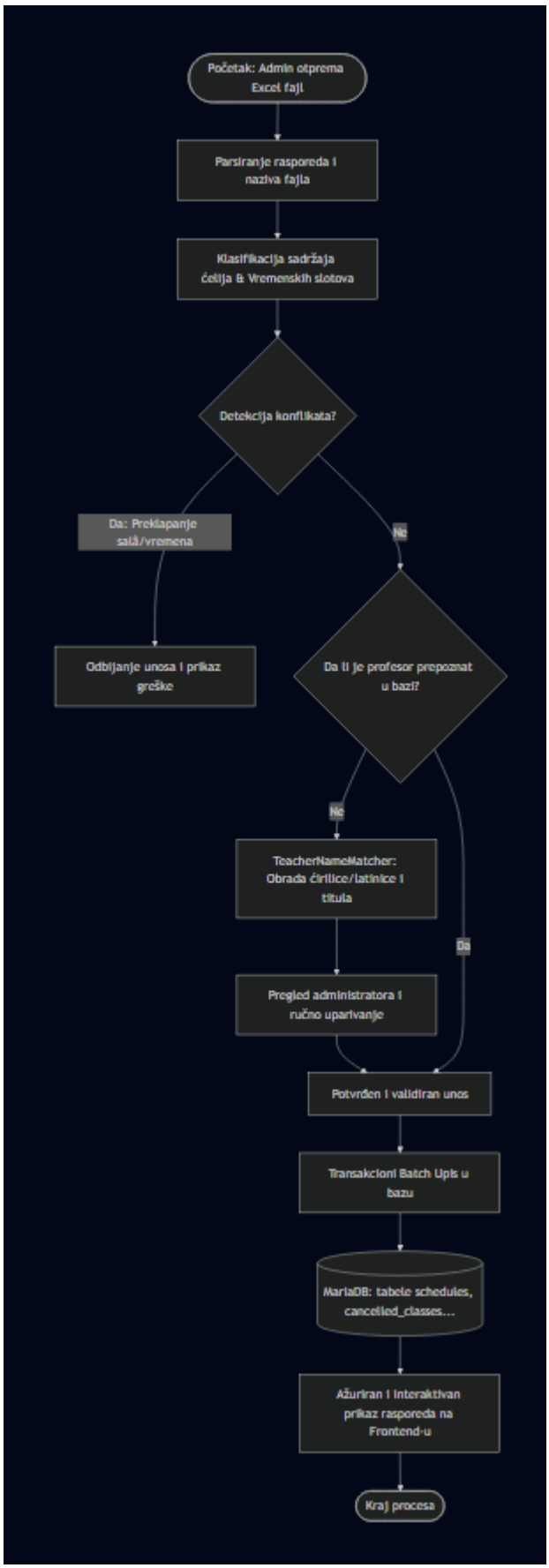
1.3 Ciljevi sistema

Glavni ciljevi sistema EduScheduler su:

1. **Centralizovana baza podataka:** Obezbeđivanje centralizovane baze podataka za sve entitete rasporeda (profesori, studenti, učionice, studijski programi, unosi rasporeda, otkazani časovi i zapisi o uvozu), koja se sastoji od 12 relacionih tabela sa strogo definisanim referencijalnim integritetom.
2. **Masovni uvoz podataka (Bulk Import):** Omogućavanje masovnog uvoza nedeljnih rasporeda iz Excel (.xlsx, .xls) datoteka kroz verifikovani cevovod (pipeline) koji parsira nazive datoteka, čita ćelije radne sveske, klasifikuje sadržaj svake ćelije, proverava unose u odnosu na poslovna pravila (vremenski konflikti, duplirani redovi, nevažeći podaci) i čuva važeće unose u transakcionom batch-u - obrađujući približno 200 redova za manje od 500 milisekundi na razvojnom hardveru.
3. **Izvoz i distribucija podataka (Data Export):** Omogućavanje jednostavnog izvoza filtriranih ili kompletnih podataka o raspordu nastave, zauzetosti učionica i nastavnom osoblju u standardizovanim formatima (Excel), čime se olakšava dalja administrativna obrada i arhiviranje podataka.
4. **Razrešavanje imena nastavnika:** Rešavanje neslaganja u imenima nastavnika između uvezenih podataka i kanonske tabele profesora putem TeacherNameMatcher sistema, koji vrši približno (fuzzy) uparivanje imena profesora sačuvanih na latiničnom i ćiriličnom pismu, obrađuje titule i prikazuje nerešene unose administratoru na pregled.
5. **Praćenje otkazanih i zamenskih časova:** Evidentiranje otkazanih i zamenskih časova kroz tabelu cancelled_classes, čuvajući vezu između originalnog unosa u rasporedu i njegove zamene, uz prikaz ovih izmena u vizuelizaciji rasporeda.
6. **Kontrola pristupa zasnovana na ulogama (RBAC):** Prikaz interfejsa prilagođenih za četiri tipa korisnika - Superadmin, Admin, Profesor i Student - gde svaki ima odgovarajući nivo pristupa funkcionalnostima sistema, što se sprovodi kako na nivou UI ruta, tako i na API nivou putem dvostepenog modela autentifikacije.
7. **Bezbedan oporavak lozinke:** Implementacija bezbednog toka za resetovanje lozinke usklađenog sa OWASP standardima, korišćenjem kriptografski nasumičnih 256-bitnih tokena sa trajanjem od jednog časa i jednokratnom invalidacijom, koji se isporučuju putem PHPMailer-a preko STARTTLS SMTP protokola. [7]

8. **Mitigacija web ranjivosti:** Smanjenje rizika od uobičajenih web ranjivosti (SQL injection, XSS, CSRF) korišćenjem parametrizovanih upita, automatskog JSX izbegavanja (escaping), dinamičke CORS bele liste (allow-list) i HttpOnly kolačića sesije.

Kompletan algoritam obrade Excel datoteka, detekcije konflikata i uparivanja imena prikazan je na Slici 1.3.1.



Slika 1.3.1: Dijagram toka masovnog uvoza Excel rasporeda i razrešavanja imena nastavnika

1.4 Obim i ograničenja Sistema

Obim sistema EduScheduler obuhvata pun životni ciklus upravljanja obrazovnim rasporedom: uvoz, validaciju, detekciju konflikata, razrešavanje imena nastavnika, praćenje otkazanih časova, vizuelizaciju i administraciju. Sistem je dizajniran za lokalno ili institucionalno okruženje i cilja obim jedne visokoškolske ustanove ili departmana.

U radu se navode sledeća uočena ograničenja:

1. **Hardkodovane SMTP akreditacije:** Funkcionalnost slanja e-mail poruka za resetovanje lozinke sadrži SMTP parametre direktno u PHP izvornim datotekama. Ne postoji konfiguraciona datoteka okruženja (npr. .env). Ovo predstavlja poznati bezbednosni rizik koji je dokumentovan u odeljku 6.
2. **Razdvojena struktura direktorijuma:** Frontend i backend se nalaze na odvojenim lokacijama u fajl sistemu (C:\Users\Shindiri\Desktop\Projekti\eduscheduler i C:\xampp\htdocs\EduScheduler), što komplikuje kontrolu verzija, produkciono podizanje i koordinaciju zavisnosti.
3. **Ručno upravljanje SQL šemom:** Evolucija šeme baze podataka vodi se kroz 13 sirovih SQL skripti koje se izvršavaju ručno, bez automatskog alata za migracije, podrške za rollback ili praćenja istorije migracija.
4. **Hardkodovani lokalni parametri za bazu:** Konekcija sa bazom podataka koristi korisničko ime root bez lozinke, što odgovara podrazumevanoj XAMPP konfiguraciji, ali je neadekvatno za produkciono okruženje.
5. **Razvojna konfiguracija:** URL za preusmeravanje pri resetovanju lozinke koristi http://localhost:5173, CORS porekla su ograničena na razvojne adrese, a poruke o greškama prikazuju interne tehničke detalje.
6. **Nekompletna pokrivenost testovima:** Backend sadrži 9 PHPUnit jedinica testova (unit tests); frontend nema automatizovane testove, niti postoje end-to-end (E2E) testovi.

1.5 Ciljne grupe korisnika i njihove potrebe

EduScheduler opslužuje pet kategorije korisnika, od kojih svaka ima specifične zahteve:

- **Superadmin:** Uloga sa najvišim nivoom privilegija. Superadmin može upravljati svim entitetima u sistemu, konfigurisati sistemska podešavanja, pristupati svim revizorskim dnevnicima (audit logs) i vršiti administrativne funkcije koje obični administratori ne mogu. Ova uloga predstavlja proširenje tradicionalnog trostepenog modela i implementirana je u kodu.
- **Admin:** Potpuni CRUD pristup nad profesorima, studentima, učionicama, rasporedima i studijskim programima. Administratori mogu uvoziti Excel rasporede, upravljati mrežom rasporeda, rešavati neuparena imena nastavnika, pregledati zapise o otkazanim časovima i pokretati slanje e-mail poruka za resetovanje lozinki profesorima i studentima.
- **Profesor:** Pristup čitanju sopstvenog rasporeda i rasporeda svog departmana. Profesori mogu upravljati rezervacijama sala i pregledati unose u rasporedu koji se odnose na njihove nastavne obaveze.
- **Student:** Pristup čitanju ličnog rasporeda, filtriranog na osnovu njihovog studijskog programa i godine studija (study_program_id, study_year), što se razrešava iz serverske sesije. Studenti mogu samostalno resetovati svoju lozinku putem tokenizovanog toka.
- **Gost:** Neautentifikovani korisnici vide javnu početnu stranicu sa opcijama za prijavu na system.

1.6 Pregled strukture rada

Ostatak ovog master rada organizovan je u sledeća poglavlja, od kojih se svako bavi posebnom dimenzijom sistema EduScheduler:

- **Poglavlje 1 - Uvod:** Kontekst, definicija problema, ciljevi, obim, ciljni korisnici i struktura rada.
- **Poglavlje 2 - Primenjene tehnologije:** Kompletan inventar frontend steka (React 19, Vite, React Router DOM v7, TanStack Query, Zod, React Hook Form, Styled-Components, React Toastify, PHPMailer), backend i database steka (PHP 8.2, PHP-DI Container, PDO/MySQLi dualni pristup, MariaDB 10.4, PHP sesije, REST API, CORS), i razvojne infrastrukture (XAMPP, VS Code, Git, phpMyAdmin, npm).
- **Poglavlje 3 - Arhitektura sistema i dizajn baze podataka:** Klijent-server arhitektura, trostepena arhitektura, četvorostepena hijerarhija uloga, REST komunikacija, dvostepena autentifikacija, frontend arhitektura (hijerarhija komponenti, upravljanje stanjem, rutiranje, API sloj), backend arhitektura (struktura direktorijuma, konekcija sa bazom, servis za uvoz, razrešavanje imena nastavnika) i kompletna relaciona šema od 12 tabela.
- **Poglavlje 4 - Detalji implementacije:** Frontend implementacija (ulazna tačka, konteksti autentifikacije, komponente stranica, TanStack Query, Zod validacija, interfejs za otkazane časove) i backend implementacija (autorizacione rute, CRUD rute kroz 30 API endpoints, admin skripte sa PHPMailer-om, cevovod za uvoz Excel datoteka, API za otkazane časove).
- **Poglavlje 5 - Bezbednost sistema i evaluacija performansi:** Autentifikaciona arhitektura, heširanje lozinki, tokenizovani mehanizam resetovanja, OWASP mitigacije, indeksiranje baze i optimizacija upita, benchmark masovnog uvoza, efikasnost renderinga na klijentu, PHPUnit rezultati i bezbednosni test scenariji.
- **Poglavlje 6 - Zaključak, tehnička ograničenja i budući rad:** Sumiranje tehničkih dostignuća, dokumentovanje realnih ograničenja i smernice za budući razvoj (upravljanje okruženjem preko vlucas/phpdotenv, Docker kontejnerizacija, E2E testiranje pomoću Playwright-a, real-time WebSocket detekcija konflikata).

Rad se sadrži Bibliografiju u kojoj su navedeni svi izvori korišćeni tokom procesa istraživanja i razvoja, kao i priloge u toku rada koji sadrže dijagram šeme baze podataka, kompletnu referentnu tabelu API ruti (krajnjih tačaka), dijagram hijerarhije komponenti, snimke ekrana ključnih prikaza korisničkog interfejsa, kao i primere test slučajeva sa rezultatima.

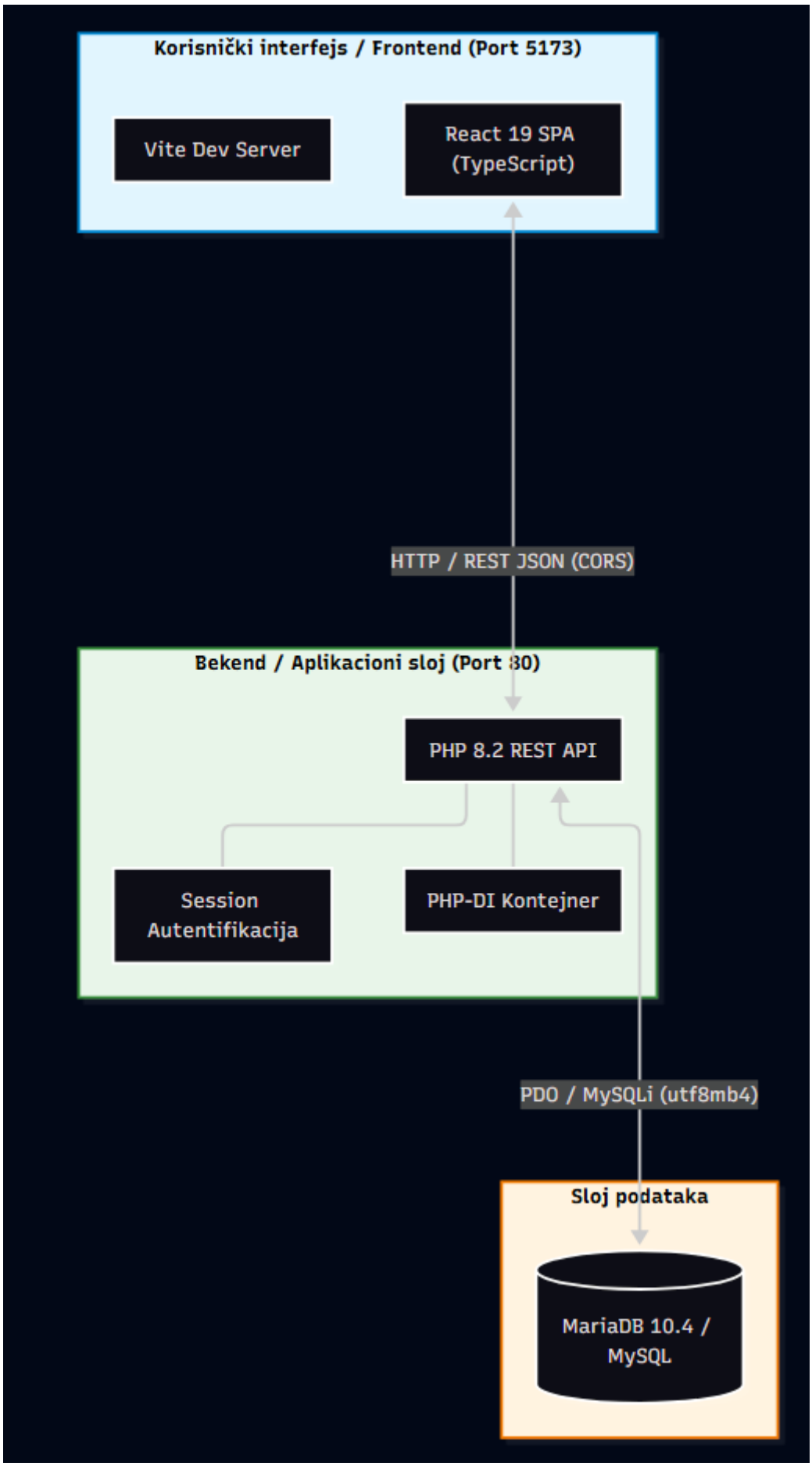
2. IZBOR TEHNOLOGIJA

Ovo poglavlje predstavlja detaljan pregled softverskih tehnologija, okvira (framework-a), biblioteka i arhitektonskih alata korišćenih pri projektovanju, implementaciji i postavljanju EduScheduler platforme. Izgradnja savremene, skalabilne i bezbedne veb-aplikacije za upravljanje rasporedom nastave u visokoškolskim ustanovama zahteva pažljivo odabran tehnološki stek koji je sposoban da iznese interakciju u realnom vremenu, kompleksnu autorizaciju zasnovanu na ulogama korisnika i obradu podataka iz više različitih izvora.

Tehnička osnova EduScheduler platforme strukturirana je oko klasične klijent-server (troslojne) arhitekture, koja jasnim granicama razdvaja sistem na tri osnovna sloja:

1. **Korisnički interfejs / Frontend (Prezentacioni sloj):** Jednostranična veb-aplikacija (Single-Page Application – SPA) izgrađena pomoću savremenih veb-tehnologija, koja pruža intuitivan, odzivan (responsive) i korisničkim ulogama prilagođen interfejs na različitim uređajima.
2. **Bekend / Backend (Aplikacioni sloj):** Server-side RESTful API odgovoran za izvršavanje poslovne logike, upravljanje sesijama, bezbednu autentifikaciju više tipova korisnika, automatizovane cjevovode (pipelines) za parsiranje podataka i eksterne integracije.
3. **Baza podataka (Sloj podataka):** Sistem za upravljanje relacionim bazama podataka (RDBMS) projektovan tako da očuva integritet podataka, primeni složene domenske relacije kroz normalizaciju i restrikcije stranih ključeva, i obezbedi upite sa niskim kašnjenjem (low latency).

Pored tehnologija koje se izvršavaju u vreme rada (runtime stack), ovo poglavlje dokumentuje i razvojne alate, alate za prevođenje i pomoćne alate - kao što su menadžeri paketa, lokalna serverska okruženja, alati za statičku analizu koda i politike kontrole verzija — koji su podržali ceo životni ciklus implementacije. Svako poglavlje detaljno obrazlaže tehničku motivaciju iza izbora konkretnih alata, analizirajući njihov doprinos ostvarivanju ciljeva pouzdanosti, održivosti i efikasnosti razvoja. Tehnička osnova EduScheduler platforme strukturirana je oko klasične klijent-server arhitekture, što je grafički prikazano na Slici 2.1.1.



Slika 2.1.1: Troslojna klijent-server arhitektura EduScheduler platforme

2.1 Frontend tehnologije

Korisnički interfejs (frontend) platforme EduScheduler implementiran je kao jednostranična veb-aplikacija (SPA) koja se u potpunosti izvršava unutar veb-pregledača korisnika. Klijentski deo je odgovoran za renderovanje korisničkog interfejsa, prihvatanje i validaciju unosâ korisnika, upravljanje ruterom na strani klijenta i komunikaciju sa aplikativnim programskim interfejsom (API) na backend-u putem Hypertext Transfer Protocol-a (HTTP).

Ovo poglavlje navodi i obrazlaže svaku tehnologiju koja čini frontend stek. Izbor ovih tehnologija bio je vođen sa tri osnovna principa:

- **Tipska bezbednost (Type Safety):** Obezbeđivanje statičke provere tipova tokom celokupnog ciklusa razvoja.
- **Separacija stanja (State Separation):** Održavanje jasne i predvidive razdvojenosti između stanja na serveru i stanja korisničkog interfejsa.
- **Razvojno iskustvo (Developer Experience):** Omogućavanje razvoja sa niskim trenjem koji ubrzava iteracije bez žrtvovanja održivosti koda.

Klijetski kod se nalazi u projekto-orijentisanom direktorijumu:

C:\Users\Shindiri\Desktop\Projekti\eduscheduler, dok je izvorni kod aplikacije orgaanizovan unutar stable src/.

2.1.1 React 19 sa TypeScript-om

React je deklarativna JavaScript biblioteka zasnovana na komponentama za izgradnju korisničkih interfejsa, koju održava kompanija Meta (Facebook) uz široku zajednicu otvorenog koda. EduScheduler je izgrađen na React 19 verziji (zavisnosti u package.json deklarišu react@^19.2.4 i react-dom@^19.2.4) [3], što je predstavljalo najnovije stabilno izdanje biblioteke u vreme pisanja. React je izabran u odnosu na alternativne okvire poput Angular-a i Vue.js-a iz nekoliko razloga koji se direktno poklapaju sa akademskim i praktičnim ciljevima projekta:

- **Modularna arhitektura zasnovana na komponentama:** React-ov model komponenti prirodno se preslikava na domen edukativnog sistema za raspoređivanje nastave, gde se ekran sastoji od višekratno iskoristivih i nezavisnih jedinica - nedeljne mreže rasporeda, bočne trake sa prikazom sledećeg predavanja, modalnih dijaloga za kreiranje i izmenu entiteta, kao i komandnih tabli specifičnih za svaku ulogu. Svaka od ovih celina implementirana je kao namenska komponenta (npr. ScheduleGrid.tsx, AdminDashboard.tsx, ManualScheduler.tsx, kao i kolekcija modalnih komponenti pod src/components/Modals/).
- **Jednosmerni tok podataka (Unidirectional Data Flow):** React-ov jednosmerni tok podataka i eksplicitno prosleđivanje atributa (props) olakšavaju praćenje zavisnosti podataka u svakom prikazu, što je izuzetno vredno u akademskom kontekstu gde se arhitektura mora jasno objasniti i odbraniti.
- **Bogat ekosistem:** React uživa podršku najzrelijeg ekosistema pomoćnih biblioteka, što objašnjava prisustvo biblioteka kao što su React Router, TanStack Query, React Hook Form, Styled-Components i React Toastify unutar istog skupa zavisnosti.

React 19 uvodi niz funkcionalnosti koje su od velikog značaja za ovaj projekat. Ovo izdanje formalizuje podršku za Akcije (Actions) - asihrone tranzicije koje upravljaju stanjima na čekanju, optimističkim ažuriranjima i obradom grešaka - izložene kroz kuke (hooks) kao što su useTransition, useActionState i useOptimistic. Iako EduScheduler asinhrone mutacije na serveru primarno izvršava preko TanStack Query mutacija (videti odeljak 2.1.4), React 19-ov model konkurentnog renderovanja donosi značajne performanse: ruter i sloj za upite sarađuju sa React planeroom tako da zahtevna ponovna renderovanja mreže rasporeda ne blokiraju interakciju korisnika. React 19 takođe poboljšava hidraciju i prosleđivanje referenci (ref), i napušta određene zastarele obrasce u korist modernog JSX transformatora. Projekat konfiguriše moderan automatski runtime putem @vitejs/plugin-react, pa nije neophodno eksplicitno uvoziti React u svaku datoteku komponente (jsx podešavanje "react-jsx" u tsconfig.app.json).

TypeScript (typescript@~5.9.3, uz @types/react@^19.2.14 i @types/react-dom@^19.2.3) postavljen je preko React-a kako bi obezbedio statičko i strukturno tipiziranje. Datoteka tsconfig.app.json aktivira strogi režim provere tipova (strict: true), zabranjuje neiskorišćene lokalne promenljive i parametre (noUnusedLocals, noUnusedParameters) i cilja ES2023 nivo jezika sa DOM i DOM.Iterable standardnim bibliotekama. Uvođenje TypeScript-a je od posebne važnosti za aplikaciju zasnovanu na podacima kao što je EduScheduler, jer se domenski model — profesori, studenti, učionice, studijski programi, otkazani časovi i stavke rasporeda - prenosi preko mreže u JSON formatu. Deklarisanjem TypeScript interfejsa za ove entitete (videti src/types/admin.ts, src/types/classroom.ts, src/types/schedule.ts i objedinjeni src/types/index.ts), frontend dobija garancije u vreme prevođenja koda da odgovor sa API-ja odgovara očekivanoj strukturi, kao i da se props komponente i povratne vrednosti kuka koriste konzistentno. Ova disciplina predstavlja klijentski parnjak validaciji koja se izvršava na serveru u PHP-u, čime se eliminiše čitava klasa grešaka u vreme izvršavanja koje bi se inače ispoljile tek u pregledaču. [3]

Reprezentativan primer integracije React-a i TypeScript-a jeste kontekst autentifikacije. Komponenta AuthContext (src/context/Auth/AuthContext.tsx) obavija aplikaciju i izlaže stanje autentifikacije koje je tipizirano kao unija četiri moguće korisničke uloge (superadmin, admin, professor, student). Pošto se uloga autoritativno razrešava na serveru putem skripte check_auth.php i nikada se ne veruje unosu sa klijenta, TypeScript sistem tipova dodatno učvršćuje bezbednosnu arhitekturu: komponente ProtectedRoute i AdminProtectedRoute mogu granati logiku samo na osnovu vrednosti uloge koja je izvedena iz sesije sa servera.

2.1.2 Vite kao alat za prevođenje i razvojni server

Vite je alat za prevođenje (build tool) i razvojni server koji pokreće klijentski deo EduScheduler platforme (vite@^8.0.1, konfigurisan kroz vite.config.ts). Vite je izabran umesto starijih bundler-a kao što je Webpack zbog svoje izuzetne brzine i boljeg razvojnog iskustva. Vite-ova arhitektura je podeljena u dve jasne faze:

1. **Razvojna faza (Development Phase):** Tokom razvoja, Vite isporučuje izvorne module direktno pregledaču koristeći nativne ECMAScript module (ESM) preko HTTP-a, vršeći transformaciju po potrebi (on-demand) samo za modul koji se trenutno zahteva. Ovo eliminiše skupo ponovno prevođenje celokupnog paketa karakteristično za starije servere; izmena u jednoj komponenti trenutno se odražava u pregledaču.
2. **Produkciona faza (Production Phase):** U produkciji, Vite poziva svoj bundler radi kreiranja optimizovanih, analiziranih (tree-shaken) i hesiranih artefakata pogodnih za statičko postavljanje na server.

Razvojna konfiguracija je namerno minimalistička.

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
})
```

Jedini potrebni dodatak je @vitejs/plugin-react@^6.0.1, koji obezbeđuje React mogućnost brzog osvežavanja (fast-refresh) i moderan JSX transformator. Od ključnog je značaja što Vite razvojni server sluša na portu 5173 (podrazumevani port okvira), dok PHP backend poslužuje Apache HTTP Server na portu 80 pod URL putanjom http://localhost/EduScheduler. Ovo razdvajanje porekla (Origin) - gde klijentska aplikacija potiče sa jednog domena/porta, a API sa drugog - jeste glavni razlog za konfiguraciju deljenja resursa sa različitih porekla (CORS) opisanu u odeljku 2.2.6.

Produkciono prevođenje se pokreće putem npm skripte: `"build": "tsc -b && vite build"`. Ova skripta najpre izvršava TypeScript projekat prevođenja (tsc -b), koji proverava tipove u celom src/ stablu i prekida proces pri bilo kakvoj grešci; samo ako provera tipova uspešna, Vite generiše produkcioni paket.

2.1.3 React Router DOM v7 za rutiranje na strani klijenta

React Router DOM (react-router-dom@^7.13.2) obezbeđuje rutiranje na strani klijenta za EduScheduler SPA (*Single Page Application*). Rutiranje na strani klijenta je mehanizam kojim jednostranična aplikacija simulira više "stranica" bez osvežavanja cele stranice: HTML5 History API se koristi za promenu URL-a u pregledaču, dok ruter prikazuje komponentu povezanu sa tim URL-om. Za EduScheduler ovo je od esencijalnog značaja jer aplikacija nudi nekoliko različitih prikaza - javnu početnu stranicu sa modalom za prijavu, stranicu za ponovno podešavanje lozinke i ulogom zaštićenu administratorsku komandnu tablu - a korisnik očekuje da dugmad za navigaciju (*Back/Forward*) i URL-ovi funkcionišu kao na tradicionalnom veb-sajtu.

Aplikacija usvaja *data-router* API uveden u novijim verzijama React Router-a. U src/index.tsx aplikacija se inicijalizuje pomoću komponente RouterProvider i objekta rutera kreiranog preko createBrowserRouter u src/routes/Routes.tsx. Stablo ruta definiše tri primarne rute podređene korenoj App komponenti:

```
const router = createBrowserRouter([
    {
      element: <App />,
      children: [
        { path: "/", element: <HomePage /> },
        { path: "/reset-password", element: <ResetPasswordPage />
},
        { path: "/admin", element: <AdminRoutes /> },
      ],
    },
  ]);
```

Korenska App komponenta (src/App.tsx) uspostavlja zajedničke funkcionalnosti za svaku rutu: obavija aplikaciju u AuthProvider, ErrorBoundary, globalne stilove (GlobalStyle), i kontejner za obaveštenja (React Toastify). Dečije rute se prikazuju unutar <Outlet/> komponente.

ADMIN putanja je konfigurisana kao podstablo: AdminRoutes obavija AdminAuthProvider, AdminProtectedRoute i AdminDashboard, tako da je celo /admin podstablo zaštićeno proverom uloge dobijenom direktno iz check_auth.php, a ne sa bilo koje vrednosti kojom klijent može manipulisati.

Autorizacija se sprovodi kroz dve prilagođene komponente zaštite (*guard components*):

- **ProtectedRoute**: Proverava isLoggedIn fleg iz konteksta autentifikacije i preusmerava neautentifikovane korisnike na početnu stranicu.

- **AdminProtectedRoute**: Dodatno verifikuje da li je uloga potvrdno validirana sa servera kao "admin" (ili "superadmin") pre nego što renderuje AdminDashboard.

2.1.4 TanStack React Query za upravljanje stanjem servera

TanStack Query (ranije React Query), deklarisan kao @tanstack/react-query@^5.95.2, upravlja stanjem servera u EduScheduler platformi. Razlika između "klijentskog stanja" (privremeno stanje interfejsa poput informacija o tome koji je modal otvoren ili koja je učionica trenutno izabrana) i "serverskog stanja" (podaci koji žive na backend-u i samo se preslikavaju na klijentu) predstavlja ključni arhitektonski koncept. Stanje servera nosi izazove koje lokalne biblioteke za upravljanje stanjem (poput Redux-a ili samog React-ovog useState) ne rešavaju efikasno: podaci mogu postati zastareli, više komponenti može zahtevati iste podatke, mutacije moraju konzistentno invalidirati i ponovo dobavljati podatke, a mreža je asinhrona i podložna greškama. TanStack Query rešava upravo ove probleme kroz deklarativni sloj keširanja.

Biblioteka se instancira jednom na samom korenskom nivou aplikacije. U src/index.tsx, kreira se jedna instanca QueryClient objekta i prosleđuje se kroz stablo komponenti pomoću QueryClientProvider-a:

```
const queryClient = new QueryClient();

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <QueryClientProvider client={queryClient}>
      <RouterProvider router={router} />
    </QueryClientProvider>
  </StrictMode>,
);
```

Upiti / Operacije čitanja (Queries):

Operacije čitanja su izražene kao upiti, od kojih je svaki identifikovan stabilnim ključem upita (*query key*). Projekat definiše namenski sloj kuka pod src/queries/ (npr. useProfessors, useClassrooms, useStudents, useStudyPrograms, useScheduledAppointments, useRoomReservations, useBusyClassrooms) i odgovarajući skup specifičan za administratorski deo pod src/queries/admin/. Reprezentativna implementacija je useStudents, koja šalje GET zahtev na krajnju tačku za studente i kastuje JSON odgovor u strogo tipizirani Student[] niz:

```
export const useStudents = () => {
  return useQuery<Student[]>({
    queryKey: ["admin-students"],
    queryFn: async () => {
      const res = await publicApi.get("/api/students.php");
      return res.data as Student[];
    },
```

```
    });
};
```

Mutacije / Operacije upisa i invalidacija keša (Mutations & Cache Invalidation):

Operacije upisa izražene su kao mutacije. Projekat definiše kuke za mutaciju pod src/mutations/ i src/mutations/admin/, kao što su useManageProfessor, useManageClassroom, useManageStudent, useManageAdmin, useMakeReservation, useUpdateReservation, useDeleteReservation, useSendPasswordReset i useSendStudentPasswordReset. Svaka mutacija obavija Axios poziv ka odgovarajućoj krajnjoj tački backend-a i, nakon uspeha, invalidira relevantne ključeve upita kako bi zavisni prikazi ponovo preuzeli autoritativne podatke:

- **Mutacije rezervacija** invalidiraju ["roomReservations"], ["busyClassrooms"] i ["scheduledAppointments"] ključeve.
- **Mutacije manuelnog raspoređivača** invalidiraju ["manual-grid"] ključ.
- **Mutacije za upravljanje studentima** invalidiraju ["students"] i ["admin-students"] ključeve.

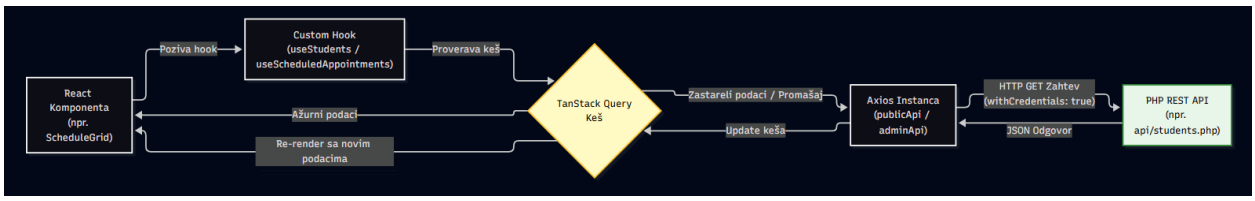
Ovaj protokol invalidacije keša garantuje da nakon što administrator kreira, izmeni ili obriše entitet u modalu, tabela na komandnoj tabli i svaki drugi konzument tih podataka trenutno odražavaju promenu bez potrebe za ručnim osvežavanjem stranice.

Sloj Axios HTTP klijenta:

HTTP komunikacija je apstrahovana kroz tri Axios (axios@^1.14.0) instance deklarisane u API sloju:

- api.ts: Cilja glavni API na http://localhost/EduScheduler/api
- adminApi.ts: Cilja administratorske skripte na http://localhost/EduScheduler/admin
- publicApi.ts: Cilja korenu putanju EduScheduler-a (koristi se za raspored studenata, stavke rasporeda, uvoz i studijske programe)

Sve tri instunce podešavaju withCredentials: true kako bi se PHPSESSID kolačić obavezno slao pri zahtevima sa različitih porekla (cross-origin), i postavljaju Content-Type zaglavlje na application/json. Protok zahteva od klijentske komponente, preko sloja upita i Axios klijenta, do REST API-ja opisan je na Slici 2.1.4.1.



Slika 2.1.4.1: Protok podataka kroz TanStack Query i Axios HTTP klijent

Arhitektonsko obrazloženje (Architectural Rationale):

Promišljena arhitektonska odluka je da EduScheduler ne koristi React Router-ove *loaders* i *actions* mehanizme za preuzimanje podataka; umesto toga, TanStack Query je jedini vlasnik stanja servera. Ovim se izbegava dupliciranje logike keširanja između rutera i sloja upita, dok komponente ostaju usmerene isključivo na renderovanje interfejsa. Ugrađene funkcionalnosti TanStack Query-ja - automatski ponovni pokušaji (*retries*), strukturno deljenje keširanih objekata (*structural sharing*), pozadinsko osvežavanje i *stale-while-revalidate* semantika - rade potpuno transparentno ispod stabla komponenti.

2.1.5 React Hook Form i Zod za validaciju formulara

Formulari predstavljaju centralnu površinu za interakciju u EduScheduler-u: korisnici se autentifikuju, administratori kreiraju i menjaju profesore, studente, učionice i administratore, dok krajnji korisnici šalju zahteve za resetovanje lozinke i rezervacije sala. Dve komplementarne biblioteke upravljaju podsistemom formulara: React Hook Form (react-hook-form@^7.72.0) obezbeđuje *runtime* za formulare, dok Zod (zod@^4.3.6) definiše ugovor o validaciji zasnovan na šemama.

Form Runtime sa React Hook Form-om:

React Hook Form funkcioniše po principu nekontrolisanih formulara (*uncontrolled forms*): umesto da ponovo renderuje komponentu pri svakom pritisku tastera, on registruje reference unosa i čita njihove vrednosti tek u trenutku slanja, što donosi značajno bolje performanse za formulare sa mnogo polja. Komponenta LoginForm (src/components/LoginForm.tsx) ilustruje ovaj obrazac:

1. Preuzima register, handleSubmit i formState (koji sadrži errors i isSubmitting fleg) iz useForm.
2. Povezuje register("email") i register("password") sa poljima za unos.
3. Prosleđuje handleSubmit(onSubmit) HTML form elementu.

Prilikom slanja, komponenta šalje kredencijale na http://localhost/EduScheduler/login.php sa omogućenim withCredentials opcijom, a u slučaju greške prikazuje poruku backend-a preko React Toastify obaveštenja.

Validacija šema pomoću Zod-a:

Zod obezbeđuje šeme za validaciju. Šeme su centralizovane pod src/schemas/ (admin.schema.ts zajedno sa šemama za profesore, učionice, termine i zauzete učionice) i deklarisane su korišćenjem Zod-ovog fluent builder API-ja. Na primer, AddStudentSchema nameće pravila prilikom kreiranja studenta:

```
export const AddStudentSchema = z.object({
  name: z.string().min(1, "Ime je obavezno"),
  surname: z.string().min(1, "Prezime je obavezno"),
  email: z.string().email("Nevalidna email adresa"),
  password: z.string().min(4, "Šifra mora imati najmanje 4 karaktera"),
  status: z.string().min(1, "Status je obavezan"),
  studyProgramId: z.number().int().positive().nullable(),
  studyYear: z.number().int().min(1).max(6).nullable(),
});
```

Izvršavanje validacije i integracija sa tipskom bezbednošću:

U ovom kodu Zod šeme se validiraju eksplicitno pomoću metode safeParse unutar modalnih komponenti (AdminModal.tsx koristi AddAdminSchema.safeParse, dok ClassroomModal.tsx, ProfessorModal.tsx i StudentModal.tsx parsiraju svoje odgovarajuće šeme). Stranica AdminLoginPage.tsx validira svoj formular prema AdminLoginSchema šemi.

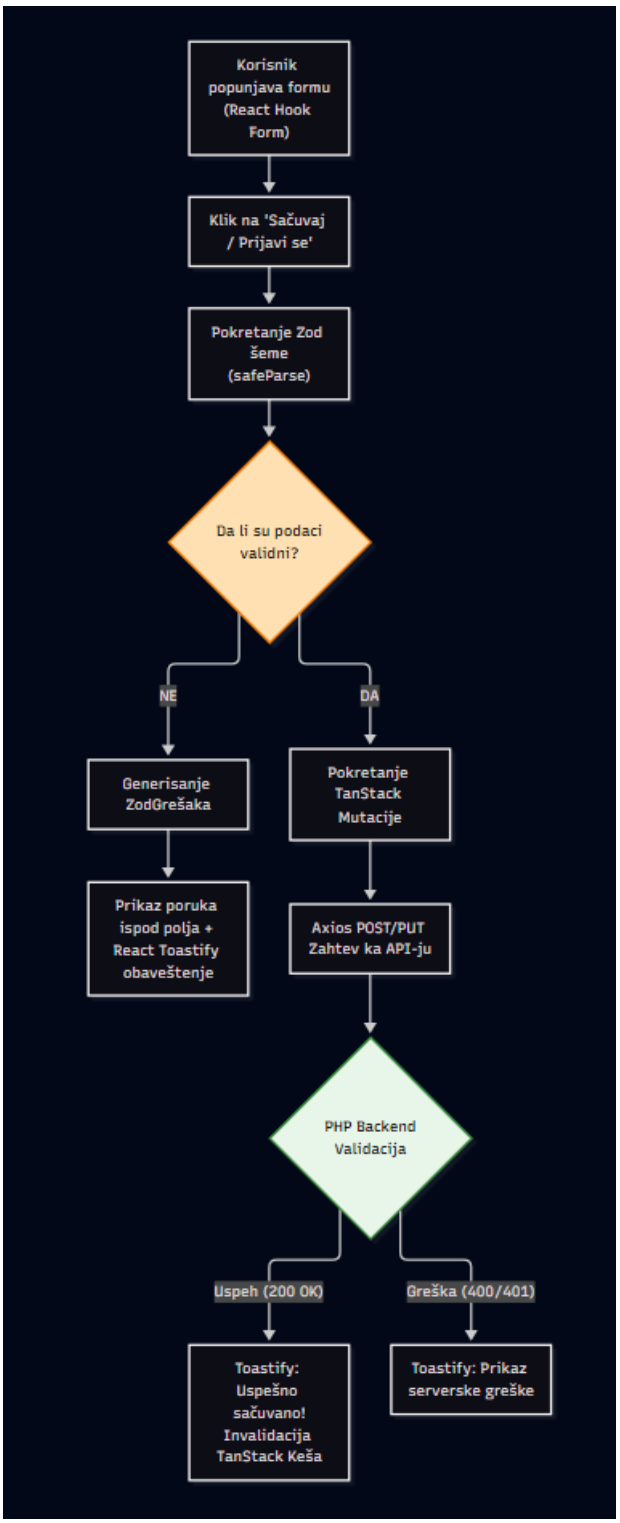
Metoda safeParse vraća diskriminisani rezultat:

- **Uspeh (Success):** Daje u potpunosti tipizirane i validirane podatke.
- **Neuspeh (Failure):** Daje listu čitljivih, lokalizovanih (na srpskom jezik) poruka o greškama koje se prikazuju direktno pored odgovarajućeg polja.

Ova arhitektura ostvaruje dva ključna cilja:

- **Jedinstveni izvor istine (*Single Source of Truth*):** Obezbeđuje jedinstvenu klijentsku referencu za restrikcije polja koja preslikava validaciju na PHP backend-u (Odeljak 2.1.5.1).
- **Statička inferencija tipova (*Static Type Inference*):** Pošto Zod izvodi TypeScript tipove direktno iz šema, podaci koji se prosleđuju mutacijama su statički garantovani da odgovaraju očekivanoj strukturi, što dodatno učvršćuje tipsku bezbednost (Odeljak 2.1.5.1).

Sled koraka prilikom validacije korisničkog unosa na klijentskoj strani prikazan je na Slici 2.3.



Slika 2.1.5.1: Dijagram toka validacije formulara pomoću Zod šeme

Napomena: Validacija na strani klijenta služi kao sloj za bolju upotrebljivost i trenutnu povratnu informaciju korisniku; PHP backend ostaje konačni, autoritativni validator kako zahtevi koji zaobilaze SPA ne bi mogli da unesu neispravne podatke.

2.1.6 Styled-Components za stilizovanje (CSS-in-JS)

Styled-Components (styled-components@^6.3.12) je rešenje za stilizovanje koje se koristi u celom EduScheduler interfejsu. Stilovi se pišu u template literalima unutar TypeScript/JavaScript modula i direktno se vezuju za pojedinačne React komponente, tako da svaka komponenta nosi sopstvenu enkapsuliranu definiciju stila. Biblioteka je deklarisana u src/styles/styled.ts i konzumira se kroz ceo projektni kod.

Izbor CSS-in-JS pristupa u odnosu na čiste CSS datoteke ili framework-e zasnovane na utility klasama poput Tailwind CSS-a bio je motivisan sa tri faktora:

1. Styled-Components automatski ograničava domet (scope) stilova na komponentu, izbegavajući konflikte sa globalnim imenima klasa.
2. Podržava dinamičko temiranje i stilizovanje na osnovu atributa (props).
3. Drži vizuelnu prezentaciju na istom mestu sa logikom komponente.

createGlobalStyle API uspostavlja pravila na nivou cele aplikacije (box-sizing i resetovanje body elementa), koja se montiraju jednokratno u src/App.tsx.

```
export const GlobalStyle = createGlobalStyle`
  *, *::before, *::after { box-sizing: border-box; }
  body { margin: 0; padding: 0; }
`;
```

2.1.7 React Toastify za obaveštenja korisnika

React Toastify (react-toastify@^11.0.5) obezbeđuje kratkotrajna obaveštenja (toast notifications) koja ne blokiraju rad korisnika. Komponenta za obaveštenja je montirana globalno u src/App.tsx sa pozicioniranjem u donjem desnom uglu, automatskim zatvaranjem nakon tri sekunde i "colored" temom koja vizuelno razlikuje poruke o uspehu, grešci, upozorenju i informacijama. Kod poziva toast API na preko 50 mesta u aplikaciji. Prikazi za uvoz i raspored dodatno koriste toast.warning za izveštavanje o delimično uspešnom uvozu Excel datoteka (broj uvezenih i preskočenih redova).

Kontejner za obaveštenja (notification container) montira se samo jednom, globalno, u datoteci src/App.tsx:

```
<ToastContainer
  position="bottom-right"
  autoClose={3000}
  hideProgressBar={false}
  newestOnTop={false}
  closeOnClick
  pauseOnFocusLoss
  draggable
  theme="colored"
```

```
/>
```

Ova konfiguracija postavlja *toast* obaveštenja u donji desni ugao, automatski ih zatvara nakon tri sekunde, omogućava zatvaranje klikom i koristi temu „colored“ kako bi se obaveštenja o uspehu, grešci, upozorenju i informacijama vizuelno razlikovala po boji.

Kod komponente poziva *toast* API direktno na mestima izvršavanja, sa preko pedeset različitih poziva obaveštenja širom izvornog koda:

- **Obaveštenja o uspehu (toast.success):** Potvrđuju završene operacije — npr. toast.success("Profesor obrisan") nakon što je profesor uklonjen, toast.success("Učionica uspešno dodata") nakon što je učionica kreirana i toast.success("Šifra uspešno postavljena. Sada se možete prijaviti.") nakon resetovanja lozinke.
- **Obaveštenja o greškama (toast.error):** Prikazuju neuspele operacije, obično čitanjem lokalizovane poruke koju vraća bekend (error?.response?.data?.message), uz korišćenje opšte (podrazumevane) poruke ukoliko dođe do greške u mrežnom sloju.
- **Obaveštenja o upozorenju (toast.warning):** Prikazuju delimično uspešne operacije, kao što je uvoz Excel fajla sa preskočenim redovima (toast.warning("Uvezeno " + data.imported + ", preskočeno " + data.skipped)).

Centralizovanjem povratnih informacija kroz jedan ToastContainer i jedinstvenu konvenciju pozivanja, aplikacija održava dosledno ponašanje obaveštenja kroz javne, profesorske, studentske i administratorske celine, i redovno obaveštava korisnika kako o uspešnim izmenama, tako i o specifičnim greškama sa serverske strane vraćenim iz REST API-ja.

2.1.8 PHPMailer za slanje e-pošte (Integracija sa backend-om)

Iako se PHPMailer izvršava na serveru, o njemu se raspravlja u ovom odeljku jer ga pokreće radni tok iniciran sa klijentske strane: administrator pokreće slanje e-pošte za resetovanje lozinke sa komandne table, a korisnik završava proces resetovanja preko React stranice.

Integracija i tok resetovanja lozinke:

Biblioteka je integrisana u administratorske skripte send_password_reset.php i send_student_reset.php, koje direktno uključuju izvorne datoteke PHPMailer-a (PHPMailer/src/Exception.php, PHPMailer.php i SMTP.php), umesto instalacije putem Composer-a.

Proces oporavka naloga funkcioniše kroz sledeće korake:

1. **Generisanje tokena:** Generiše se kriptografski bezbedan heksadecimalni token od 64 znaka koristeći funkciju bin2hex(random_bytes(32)).

2. **Perzistencija (Skladištenje):** Token se skladišti u tabeli professors ili students, zajedno sa vremenskim rokom isteka postavljenim na jedan sat.
3. **Kreiranje imejla:** Formira se URL za resetovanje u formatu http://localhost:5173/reset-password?token={token} i šalje se korisniku putem Gmail SMTP servera.

SMTP Konfiguracija:

PHPMailer podešavanje koristi STARTTLS enkripciju na portu 587 (PHPMailer::ENCRYPTION_STARTTLS), UTF-8 kodiranje znakova i HTML formatiranje poruke sa klikabilnim linkom za oporavak:

```
$mail->isSMTP();
$mail->Host = 'smtp.gmail.com';
$mail->SMTPAuth = true;
$mail->Username = 'example@gmail.com';
$mail->Password = 'example_password';
$mail->SMTPSecure = PHPMailer::ENCRYPTION_STARTTLS;
$mail->Port = 587;
$mail->CharSet = 'UTF-8';
$mail->isHTML(true);
$mail->Subject = 'Postavljanje šifre';
```

2.2 Backend tehnologije

Backend platforma EduScheduler predstavlja serverski deo aplikacije napisanu u PHP-u koja izlaže RESTful API preko HTTP-a i koju konzumira React klijentska aplikacija. Odgovorna je za autentifikaciju i autorizaciju, perzistenciju podataka, sprovođenje poslovnih pravila (validacija uvoza rasporeda, detekcija konflikata, razrešavanje imena nastavnika), praćenje otkazanih časova i slanje e-pošte.

Stablo izvornog koda backend-a nalazi se pod C:\xampp\htdocs\EduScheduler i organizovano je u:

- functions/ (zajednički pomoćni moduli)
- api/ (30 JSON krajnjih tačaka)
- admin/ (24 administrativne skripte i tok za resetovanje lozinke)
- app/ (objektno-orijentisani sloj servisa za cjevovod uvoza, razrešavanje imena nastavnika i upravljanje otkazanim časovima).

2.2.1 PHP 8.2 sa PDO i MySQLi ekstenzijama

Backend je implementiran u PHP-u 8.2. [5] Koriste se dva različita API-ja za pristup bazi podataka, svaki tamo gde je najprikladniji:

- **PHP Data Objects (PDO):** Koristi se za bezbednosno kritične putanje autentifikacije (login.php, admin_login.php, reset_password.php). PDO koristi pripremljene izjave (prepared statements) sa imenovanim parametrima i ERRMODE_EXCEPTION, čime se u potpunosti sprečavaju napadi ubrizgavanjem SQL koda (SQL Injection).
- **MySQL Improved (MySQLi):** Koristi se za šire CRUD krajnje tačke i za sloj servisa za uvoz. Zajednički pomoćni modul za povezivanje functions/db.php otvara procedurnu MySQLi vezu i izlaže pomoćne funkcije (escape, query, confirm, fetch_assoc, fetch_all). Moderan servis za uvoz pod app/Database/Connection.php uspostavlja MySQLi vezu unutar finalne klase koja se razrešava kroz PHP-DI kontejner za ubrizgavanje zavisnosti.

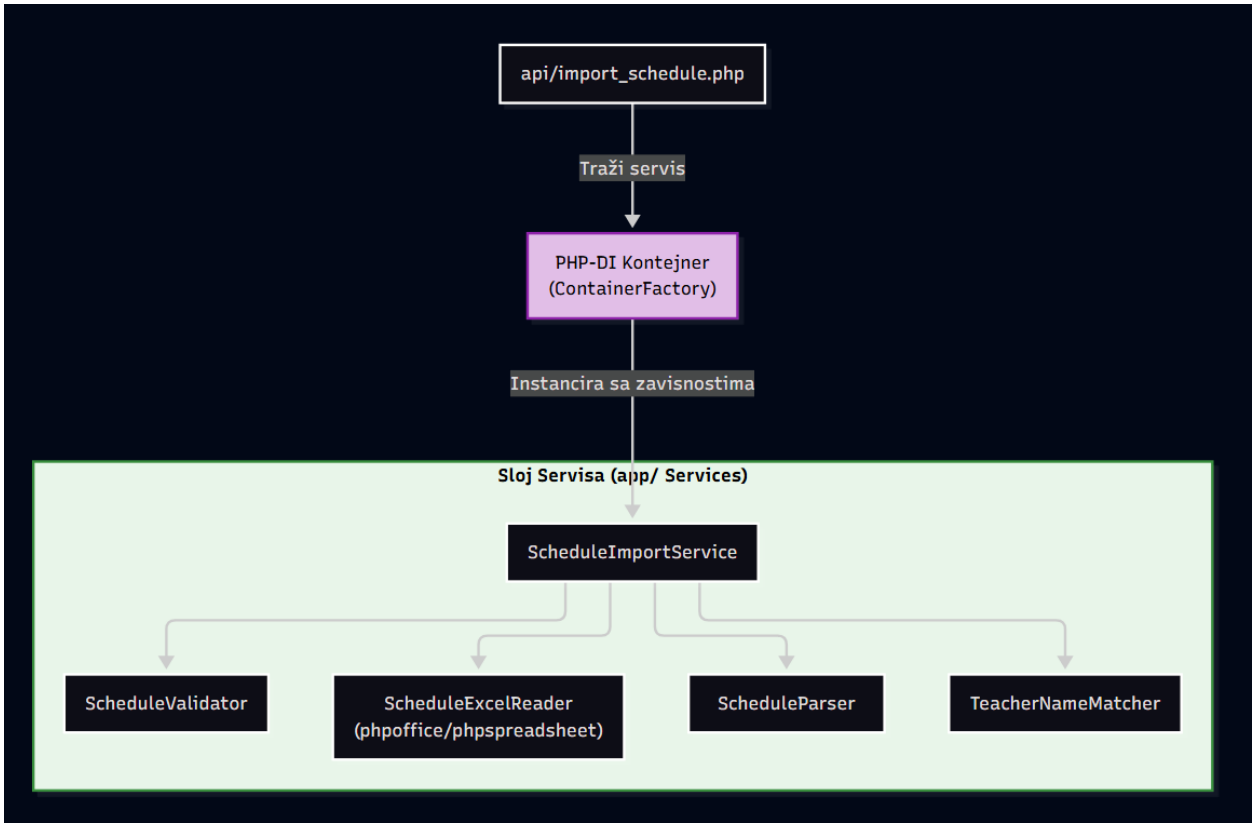
Koegzistencija PDO i MySQLi ekstenzija je opravdana: PDO ima prednost tamo gde su parametrizovana bezbednost i obrada izuzetaka od najvišeg značaja; MySQLi je zadržan za masovne CRUD operacije i servis uvoza kako bi se iskoristio postojeći sloj pomoćnih funkcija i klasa konekcije.

2.2.2 PHP-DI kontejner (Ubrizgavanje zavisnosti)

Cevovod za uvoz i srodni servisi koriste PHP-DI (php-di/php-di@^7.0), kontejner za ubrizgavanje zavisnosti (Dependency Injection) za PHP. Kontejner je konfigurisan u app/Container/ContainerFactory.php i razrešava zavisnosti servisa kao što su ScheduleImportService, ScheduleImporter, ScheduleValidator, ScheduleParser, ScheduleExcelReader, ScheduleFilenameParser i servisi za razrešavanje nastavnika (TeacherNameMatcher i srodne klase).

PHP-DI omogućava ubrizgavanje zavisnosti kroz konstruktor, čineći sloj servisa testabilnim u izolaciji i odvajajući poslovnu logiku od infrastrukturnih pitanja (veze sa bazom, skladištenje datoteka). Krajnja tačka api/import_schedule.php kreira servis za uvoz iz kontejnera umesto da ga direktno instancira, što demonstrira ovaj arhitektonski obrazac.

Na Slici 2.2.2.1 prikazana je struktura zavisnosti i način na koji PHP-DI kontejner instancira servise za obradu rasporeda.



Slika 2.2.2.1: Ubrizgavanje zavisnosti unutar backend servisa za uvoz rasporeda

2.2.3 MySQL / MariaDB 10.4 kao server baze podataka

EduScheduler čuva sve relacione podatke u MariaDB 10.4 bazi (skripte šeme bilježe server kao 10.4.32-MariaDB). Baza podataka nosi naziv eduschedulerdb, kodirana je u utf8mb4 skupu karaktera i koristi InnoDB mehanizam skladištenja sa restrikcijama stranih ključeva. [6] Šema se

sastoji od 12 tabela: admin_panel, professors, students, study_programs, classrooms, schedule_entries, schedule_imports, busy_classrooms, schedule_entry_study_programs (vezna tabela), cancelled_classes, teacher_aliases i email_templates (detaljno opisano u polavlju 3).

2.2.4 Autentifikacija zasnovana na PHP sesijama

Autentifikacija koristi PHP-ov nativni mehanizam sesija (session_start, $_SESSION, session_destroy). Objedinjena krajnja tačka za prijavu (login.php) vrši proveru kroz tri tabele (admin_panel po korisničkom imenu, zatim professors i students po email adresi) i, nakon uspeha, upisuje autoritativni identitet u serversku sesiju.

Uloga se nikada ne vraća klijentu u samom odgovoru na prijavu; umesto toga, frontend poziva api/check_auth.php, koji čita $_SESSION['user_role'] i vraća proverenu ulogu. Ova verifikacija u dva koraka - prijava praćena ponovnom proverom sesije na serveru - sprečava lažiranje uloga putem manipulacija na strani klijenta (npr. preko React DevTools). Autentifikacija zasnovana na sesijama imala je prednost u odnosu na JSON Web Tokene (JWT) jer je aplikacija jedinstven sistem gde centralizovano stanje sesije na serveru pojednostavljuje autorizaciju i omogućava trenutno opozivanje pristupa.

2.2.5 RESTful API arhitektura (JSON preko HTTP-a)

Backend izlaže RESTful API koji se sastoji od 30 datoteka krajnjih tačaka pod api/ i 24 skripte pod admin/. Svaka krajnja tačka prati uniformnu strukturu:

1. Uključivanje init.php.
2. Slanje CORS zaglavlja.
3. Postavljanje JSON Content-Type zaglavlja.
4. Provera autentifikacije i uloge korisnika.
5. Čitanje ulaznog zahteva.
6. Izvršavanje operacije nad bazom podataka putem pripremljenih izjava.
7. Vraćanje JSON odgovora sa odgovarajućim HTTP statusnim kodom (200/201 uspeh, 400 loše formiran unos, 401 neuspešna autentifikacija, 403 nedovoljna prava uloge, 405 nepodržana metoda, 500 greška na serveru).

Krajnja tačka za uvoz Excel datoteka (import_schedule.php) prihvata multipart/form-data; sve ostale koriste JSON format.

2.2.6 CORS konfiguracija za zahteve sa različitih porekla

Razvojni server klijenta (port 5173) i PHP backend (port 80, putanja /EduScheduler) predstavljaju različita porekla (origins). Svaka krajnja tačka šalje CORS zaglavlja preko zajedničke cors_handle() funkcije u functions/cors.php. Ova funkcija održava eksplicitnu listu dozvoljenih razvojnih adresa (http://localhost:5173, http://localhost:3000, http://localhost, http://127.0.0.1:5173, http://127.0.0.1:3000) i vraća Origin pozivaoca samo ako se on nalazi na toj listi.

Zahtevi sa kredencijalima (withCredentials: true) dozvoljeni su isključivo za porekla sa liste. Preflight OPTIONS zahtev se prekida pre izvršavanja bilo kakve poslovne logike. Ovaj dizajn predstavlja osnovu CSRF zaštite: samo eksplicitno odobrena porekla mogu dobiti odgovore sa sesijskim kredencijalima. [9]

2.3 Tehnologije za testiranje i bezbednost

Kvalitet i pouzdanost modernih web aplikacija u direktnoj su zavisnosti od primenjenih mehanizama verifikacije koda i bezbednosnih standarda. Sistem *EduScheduler*, kao platforma namenjena upravljanju nastavnim procesom i resursima, obrađuje specifične podatke o rasporedu nastave, nastavnom osoblju, učionicama i korisničkim nalozima. Iz tog razloga, razvojni proces je zahtevao dvostruki pristup: uspostavljanje rigoroznog sistema za automatsko testiranje ključne poslovne logike i implementaciju savremenih odbrambenih mehanizama za zaštitu od bezbednosnih ranjivosti.

Automatizovano testiranje omogućava rano prepoznavanje regresionih grešaka pri izmenama u kodu, sa posebnim fokusom na kompleksne algoritme kao što su uvoz i parsiranje Excel datoteka sa rasporedom, validacija vremenskih preklapanja i pravila pristupa. Sa druge strane, bezbednosna arhitektura sistema projektovana je tako da obezbedi tajnost, integritet i dostupnost podataka kroz usklađenost sa opšteprivaćenim industrijskim preporukama, pre svega standardima koje propisuje *Open Web Application Security Project* (OWASP).

U nastavku je dat detaljan pregled primenjenih testnih modula i bezbednosnih kontrola integrisanih u backend sloj aplikacije.

2.3.1 PHPUnit testni skup

Pozadinski deo aplikacije (backend) uključuje 9 PHPUnit testnih modula smeštenih u direktorijumu C:\xampp\htdocs\EduScheduler\tests\:

- TeacherNameMatcherTest – proverava logiku i preciznost razrešavanja i uparivanja imena nastavnika.

- ScheduleValidatorTest – validira strukturu i pravila rasporeda nastave.
- ScheduleImportRequestTest – proverava ispravnost ulaznih zahteva pri uvozu rasporeda.
- ScheduleImporterTest – testira kompletan cjevovod (pipeline) uvoza podataka.
- ScheduleFilenameParserTest – testira ekstrahovanje metapodataka iz naziva uvoznih datoteka.
- ScheduleExcelParserTest – proverava parsiranje i čitanje podataka iz Excel tabela.
- ReservationAdminAccessTest – verifikuje kontrole pristupa administratorskim funkcijama za rezervaciju.
- ManualScheduleGridTest – testira operacije i izmene u mreži ručnog raspoređivanja.
- CellClassifierTest – proverava logiku klasifikacije i prepoznavanja sadržaja ćelija.

Upravljanje zavisnostima za PHPUnit definisano je kroz composer.json datoteku, gde je PHPUnit konfigurisan kao razvojna zavisnost (require-dev).

2.3.2 Bezbednosni mehanizmi usklađeni sa OWASP standardima

Aplikacija u potpunosti adresira ključne kategorije rizika definisane OWASP Top 10 standardom [7] [8] kroz implementaciju sledećih kontrola:

- **Zaštita od SQL ubrizgavanja (SQL Injection – OWASP A03:2021):** Sve interakcije sa bazom podataka realizovane su isključivo putem parametrizovanih upita. Koriste se PDO pripremljene izjave sa imenovanim parametrima (u skriptama login.php, reset_password.php, admin_login.php) i MySQLi pripremljene izjave sa funkcijom bind_param (u send_password_reset.php i administratorskim CRUD skriptama). U cjevovodu za uvoz Excel datoteka dodatno se primenjuje mysqli_real_escape_string(). U SQL upitima je u potpunosti eliminisano direktno spajanje (konkatenacija) korisničkih ulaznih nizova.
- **Zaštita od kros-sajt skriptovanja (Cross-Site Scripting / XSS – OWASP A03:2021):** Na serverskoj strani vrši se obavezno neutralisanje specijalnih karaktera putem htmlentities() i htmlspecialchars() funkcija pre generisanja izlaza. Na klijentskoj strani, React 19 JSX sintaksa automatski vrši sanitizaciju svih interpoliranih izraza, pri čemu se dangerouslySetInnerHTML ne koristi za podatak koji potiče od korisnika.
- **Zaštita od lažiranja zahteva sa drugog sajta i CORS restrikcije (CSRF / CORS – OWASP A01:2021 / A05:2021):** Dinamičko ograničavanje CORS porekla realizovano je u functions/cors.php uz restrikciju slanja kredencijala. Identitet korisnika održava se putem

HttpOnly kolačića sesije sa podešenjem SameSite=Lax. Pri odjavi se vrši eksplicitno uništavanje sesije na serveru, dok klijentski HTTP zahtevi obavezno šalju kredencije uz podešavanje withCredentials: true.

2.4 Razvojna i deployment infrastruktura

Razvoj, verzionisanje i lokalno pokretanje aplikacije *EduScheduler* oslanjaju se na integrisani skup alata koji obezbeđuju stabilno okruženje tokom čitavog životnog ciklusa projekta.

2.4.1 XAMPP (Apache + MariaDB + PHP) u Windows okruženju

EduScheduler se razvija i pokreće na lokalnoj Windows radnoj stanici korišćenjem XAMPP paketa. U ovom projektu XAMPP obezbeđuje sledeće verzije softverskih komponenti:

- **Apache HTTP Server 2.4.58 (Win64)**
- **MariaDB Database Server 10.4.32**
- **PHP Runtime 8.2.12**

Izvorni kod pozadinskog dela aplikacije (backend) postavljen je direktno u Apache korenski direktorijum na putanji C:\xampp\htdocs\EduScheduler. Ova arhitektura omogućava da PHP aplikativni sloj i baza podataka rade na istom hostu preko *loopback* interfejsa, čime se eliminiše mrežna latencija tokom razvoja. Aplikacija je na lokalnom serveru dostupna putem adrese http://localhost/EduScheduler.

2.4.2 Visual Studio Code kao razvojno okruženje (IDE)

Visual Studio Code (VS Code) predstavlja primarno okruženje za razvoj frontend i backend sloja. Radni prostor uključuje podešavanja unutar vscode/ direktorijuma čime se obezbeđuje ujednačenost okruženja.

- **Klijentski alati i statička analiza:** Integrisani TypeScript jezički server u VS Code-u pruža dijagnostiku koda u realnom vremenu na osnovu tsconfig.app.json pravila. Statička analiza koda na frontendu oslanja se na **ESLint 9** (eslint@^9.39.4) sa ravnom konfiguracijom (eslint.config.js) koja obuhvata @eslint/js, typescript-eslint, eslint-plugin-react-hooks i eslint-plugin-react-refresh.
- **Serverski alati:** Za PHP backend, VS Code obezbeđuje indeksiranje i navigaciju kroz direktorijume api/, admin/, functions/ i app/.

2.4.3 Git i arhitektura repozitorijuma

Git se koristi za praćenje izmena i verzionisanje koda. Pravila repozitorijuma definisana u .gitignore datoteci isključuju instalirane zavisnosti (node_modules/), izvršne fajlove bildovanja (dist/), logatere i sistemske metapodatke.

U lokalnom razvojnom okruženju primenjen je specifičan raspored direktorijuma (*split-origin layout*):

- **Frontend direktorijum:** C:\Users\{Korisnik}\Desktop\Projekti\eduscheduler
- **Backend direktorijum:** C:\xampp\htdocs\EduScheduler

Klijentski radni direktorijum poseduje sopstvenu Git strukturu, dok se backend direktorijum pokreće pod Apache serverom. Ujedinjenje oba sloja u jedinstveni monorepo ili povezivanje putem Git podmodula (*submodules*) predstavlja smernicu za produkcionu deploy infrastrukturu.

2.4.4 phpMyAdmin za administraciju baze podataka

Za grafičko upravljanje relacionom bazom eduschedulerdb koristi se **phpMyAdmin 5.2.1**, integrisan u XAMPP paket i dostupan na http://localhost/phpmyadmin.

U okviru projekta phpMyAdmin je poslužio za tri ključna zadatka:

1. **Projektovanje i verifikaciju šeme:** Definisanje 8 InnoDB tabela sa utf8mb4 kodiranjem i spoljnim ključevima.
2. **Serijalizaciju i rezervne kopije:** Generisanje zvanične eduschedulerdb.sql skripte baze.
3. **Inspekciju API operacija:** Vizuelna verifikacija upisa u bazu tokom testiranja (npr. provera generisanog reset_token-a ili uvoza u tabelu schedule_entries).

2.4.5 npm i Node.js za klijentske pakete

Upravljanje klijentskim bibliotekama i automatizacija izrade vrši se putem **Node.js v24.15.0** i **npm** menadžera paketa.

npm skripte definisane u package.json obuhvataju:

- npm run dev: Pokreće Vite razvojni server sa podrškom za brzu zamenu modula (*Hot Module Replacement*).
- npm run build: Izvršava TypeScript proveru tipova (tsc -b) i generiše optimizovane produkcione fajlove (vite build).
- npm run lint: Pokreće ESLint proveru koda (eslint .).

- npm run preview: Pokreće lokalni server za pregled produkcionog koda (vite preview).

Zavisnosti projekta su striktno zaključane u package-lock.json datoteci i podeljene su na:

- **Runtime zavisnosti (dependencies):** react, react-dom, react-router-dom, @tanstack/react-query, react-hook-form, zod, styled-components, react-toastify, axios, react-icons, react-modal.
- **Razvojne zavisnosti (devDependencies):** vite, @vitejs/plugin-react, typescript, @types/*, eslint sa odgovarajućim priključcima.

2.5 Sažetak tehnološkog steak-a

Prikaz sažetka tehnološkog steak-a kroz tabelu 2.5.1:

Sloj sistema	**Tehnologija / Alat**	**Uloga i namena u sistemu**
Klijentski sloj (Frontend)	React 19 & TypeScript	Tipizirana komponentna struktura korisničkog interfejsa
	Vite	Brz lokalni razvoj (HMR) i optizovani produkcioni build
	React Router v7	Klijentsko rutiranje i zaštita pristupnih putanja
	TanStack Query	Upravljanje asinhronim stanjem servera i keširanjem
	React Hook Form & Zod	Formularsko stanje i validacija šema
	Styled-Components	Enkapsulirano stilizovanje komponenti (CSS-in-JS)
Serverski sloj (Backend)	PHP 8.2 (PDO & MySQLi)	Poslovna logika i bezbedan parametrizovani pristup bazi

Sloj sistema	Tehnologija / Alat	Uloga i namena u sistemu
	MariaDB 10.4 / InnoDB	Relaciono skladištenje podataka uz transakcioni integritet
	PHP Native Sessions	Autorizacija i upravljanje korisničkim sesijama
	RESTful JSON API	HTTP interfejs za razmenu podataka sa klijentom
	PHPMailer	Slanje transakcionih imejlova za resetovanje lozinke
Infrastruktura i Alati	XAMPP	Lokalno radno okruženje (Apache, MariaDB, PHP)
	VS Code & ESLint 9	Razvojno okruženje i statička analiza koda
	Git	Verzionisanje izvornog koda i istorija izmena
	phpMyAdmin	Administracija baze podataka i generisanje SQL dampa
	Node.js (v24) & npm	Upravljanje klijentskim zavisnostima i skriptama

Tabela 2.5.1: Sažetak tehnološkog steak-a

3. ARHITEKTURA SISTEMA I DIZAJN BAZE PODATAKA

Ovo poglavlje predstavlja arhitektonske osnove aplikacije *EduScheduler*. Oslanjajući se na inventar tehnologija iz Poglavlja 2, objašnjava se kako su te tehnologije komponovane u koherentan, funkcionalan sistem. Diskusija napreduje od opšteg prikaza ka specifičnostima:

- **Odejak 3.1:** Uspostavlja visokonivojsku klijent-server i troslojnu topologiju zajedno sa obezbeđivanjem autentifikacije i rutiranja.
- **Odejak 3.2:** Opisuje unutrašnju arhitekturu i strukturu komponenata React frontend-a.
- **Odejak 3.3:** Detaljno objašnjava organizaciju, raščlanjivanje slojeva i izvršavanje servisa PHP backend-a.
- **Odejak 3.4:** Specifikuje relacioni model baze podataka, uključujući svaku tabelu, definicije kolona, indekse i strane ključeve koji održavaju referencijalni integritet.

3.1 Visokonivojska arhitektura sistema (High-Level System Architecture)

Ovo poglavlje opisuje opšti strukturni koncept (arhitekturu) sistema EduScheduler, detaljno izlažući način na koji osnovni slojevi sistema komuniciraju, upravljaju autentičnosti korisnika i usmeravaju njihove interakcije. [1]

3.1.1 Klijent-server model (Client-Server Model)

EduScheduler prati klasični klijent-server arhitektonski stil. Tanki klijent - React aplikacija jedne stranice (SPA) koja se izvršava u pretraživaču korisnika - odgovoran je za prikaz korisničkog interfejsa i prikupljanje inputa, dok server - PHP aplikacija na Apache serveru - poseduje poslovnu logiku, sprovodi bezbednosnu politiku i perzistira stanje. Ova dva sloja komuniciraju isključivo preko HTTP protokola. Klijent ne čuva autoritativnu kopiju podataka. Isti backend opslužuje heterogene klijente - javnu početnu stranicu, profesorski prikaz, studentski prikaz i administratorski panel - bez dupliciranja logike.

Tokom lokalnog razvoja, dva sloja se nalaze na različitim poreklima (origins):

- **Vite dev server:** http://localhost:5173
- **Apache:** http://localhost/EduScheduler

Ova razdvojenost donosi nekoliko poželjnih osobina za sistem zakazivanja nastave:

- **Centralizovana poslovna logika i integritet podataka:** Podaci i domenska pravila (npr. osiguravanje da student može da vidi samo raspored dodeljen njegovom studijskom programu i godini studija) nalaze se na jednom, bezbednom mestu na serveru, čime se sprečavaju zloupotrebe sa klijentske strane.

- **Jedinstveni API za različite interfejse:** Isti PHP bekend opslužuje različite prikaze - javnu početnu stranicu, kontrolnu tablu za profesore, studentski portal i administratorski panel - bez dupliranja osnovne poslovne logike.
- **Performanse i nezavisnost (dekapling):** S obzirom na to da je klijentski deo statički paket HTML, JavaScript i CSS fajlova, zahtevna obrada (kao što je parsiranje tabela sa rasporedom, provera konflikata i hešovanje lozinki) u potpunosti je prebačena na PHP izvršno okruženje.

3.1.2 Petostepena hijerarhija uloga (Guest, Student, Professor, Admin, Superadmin)

EduScheduler implementira petostepenu hijerarhiju uloga u kojoj su prava pristupa jasno razgraničena:

- **Guest (Gost / Neautentifikovani korisnik):** Podrazumevani nivo pristupa za sve posetioce koji nisu prijavljeni na sistem. Gost ima pristup javnom prikazu rasporeda časova (po učionicama, smerovima i profesorima), pretrazi i filtriranju časova, kao i uvidu u zauzetost prostorija, bez mogućnosti izmene podataka ili pristupa privatnim profilima.
- **Student:** Pristup čitanju ličnog rasporeda, filtriranog prema njihovom studijskom programu i godini studija, kako je razrešeno iz serverske sesije.
- **Professor (Profesor):** Pristup čitanju sopstvenog rasporeda i rasporeda katedre. Profesori mogu upravljati rezervacijama prostorija i pregledati stavke rasporeda.
- **Admin:** Pun CRUD pristup profesorima, studentima, učionicama, rasporedima, studijskim programima, otkazanim časovima i zapisima o uvozu. Administratori mogu uvoziti Excel rasporede, rešavati neuparena imena nastavnika, upravljati mrežom rasporeda i pokretati e-mailove za resetovanje lozinke.
- **Superadmin:** Uloga sa najvišim privilegijama. Superadministratori imaju pristup svim funkcionalnostima sistema, uključujući konfiguraciju na nivou celog sistema, revizorske logove (audit logs) i administratorske funkcije koje redovni administratori ne mogu obavljati. Uloga superadministratora proširuje model i sprovodi se na nivou API-ja (serverska provera sesije) i na nivou UI-ja (route guards u AdminProtectedRoute, koji dozvoljava i ulogu "admin" i "superadmin").

Hijerarhija uloga se sprovodi na dva nivoa: zaštitnici ruta na UI-ju (ProtectedRoute i AdminProtectedRoute) pružaju sloj pogodnosti i preusmeravaju neautentifikovane goste na login

stranicu pri pokušaju pristupa zaštićenim rutama, dok server ostaje konačni arbiter - svaki zaštićeni endpoint proverava $_SESSION['user_role'] pre obavljanja posla. [2]

3.1.3 Troslojna arhitektura (Presentation, Business Logic, Data)

U okviru klijent-server modela, EduScheduler je organizovan kao troslojna arhitektura:

1. **Prezentacioni sloj (Frontend):** React 19 sa TypeScript-om, fizički realizovan kroz src/ stablo. Namerno je "glup" u pogledu politike: renderuje ono što je server odobrio i prijavljuje greške koje server vrati.
2. **Sloj poslovne logike (Backend):** PHP 8.2, fizički realizovan kroz direktorijume api/, admin/, functions/ i app/. Svaki endpoint deluje kao kontroler koji prima zahtev, konsultuje sesiju i bazu podataka, primenjuje poslovna pravila (validacija uvoza rasporeda, detekcija konflikata, razrešavanje nastavnika, praćenje otkazanih časova) i vraća JSON odgovor.
3. **Sloj podataka (Database):** MariaDB 10.4 / InnoDB, fizički realizovan kroz eduschedulerdb šemu od 12 tabela, kojoj se pristupa putem PDO i MySQLi drajvera

S obzirom na to da svaki sloj strogo zavisi od interfejsa koji pruža sloj ispod njega, slojevi se mogu nezavisno analizirati, testirati i održavati. Na primer, prezentacioni sloj bi se mogao ponovo implementirati u drugom klijentskom (*frontend*) okruženju bez izmene postojećih PHP ruta (*endpoints*), pod uslovom da interfejs JSON API-ja ostane nepromenjen.

3.1.4 Komunikacija frontenda i backend-a putem REST API-ja

Arhitektura sistema primenjuje strogo fizičko i logičko razdvajanje između klijentskog interfejsa i perzistentnog sloja podataka (decoupling). Frontend nikada ne konstruiše sirove SQL upite niti direktno komunicira sa bazom podataka. Komunikacija sa klijentske strane je isključivo "slepa" za strukturu i relacije u MariaDB bazi - klijent isključivo šalje zahteve za apstrahovanim resursima i zavisi od backend-a koji deluje kao bezbednosna barijera (gatekeeper), sprovodi poslovnu logiku i bezbedno izvršava operacije nad bazom.

Komunikacija između slojeva odvija se po RESTful principima, gde je svaki resurs adresiran jedinstvenom URL putanjom i manipulisan standardnim HTTP metodama (GET, POST, PUT, DELETE). Za potrebe mrežne komunikacije, frontend koristi tri namenski konfigurisane Axios instancije koje usmeravaju zahteve na odgovarajuće backend lokacije:

- **api.ts** (baseURL: http://localhost/EduScheduler/api): Namenjen za opšte resurse aplikacije (dohvatanje rasporeda, pregled učionica, otkazani časovi, rezervacije).

- **adminApi.ts** (baseURL: http://localhost/EduScheduler/admin): Namenjen za administrativne endpoint-e i upravljačke skripte (CRUD operacije nad profesorima, studentima i korisničkim nalozima).
- **publicApi.ts** (baseURL: http://localhost/EduScheduler): Namenjen za javne operacije na korenu aplikacije (javna čitanja bez autentifikacije i tokove za resetovanje lozinke).
- **Primeri radnog toka i ciklus izvršavanja (Workflow Examples & Execution Cycle)**

Kako bi se pojasnio ovaj model, komunikacija se može razdvojiti na dva tipična ciklusa obrade podataka:

Tok čitanja podataka (Read Flow):

1. Klijentska komponenta poziva prilagođenu TanStack Query kuku (npr. useProfessors), koja preko adminApi.ts šalje HTTP GET zahtev na /admin/professors.php.
2. Backend (PHP) presreće zahtev, autentifikuje sesiju korisnika, izvršava siguran SELECT upit nad tabelom professors i formatira rezultat u strukturirani JSON niz.
3. Klijent prima JSON odgovor i renderuje podatke u korisničkom interfejsu. Pri svakoj narednoj promeni podataka, TanStack Query automatski invalidira odgovarajući keš ključ i pokreće osvežavanje (refetch).

2. Tok izmene podataka (Mutation Flow):

1. Korisnik u administratorskom interfejsu popunjava formu (npr. dodavanje novog studenta), što aktivira mutacionu kuku (useManageStudent).
2. Kuka šalje HTTP POST zahtev sa JSON telom na /admin/add_student.php.
3. Backend validira pristigle podatke i izvršava INSERT upit u tabeli students koristeći pripremljene upite sa vezanim parametrima (bound parameters), čime se sprečava SQL injection.
4. Backend vraća JSON odgovor sa flegom uspešnosti. Po prijemu odgovora, frontend prikazuje obaveštenje korisniku (success toast) i automatski osvežava lokalni keš sa novim podacima.

Transportne konvencije i CORS pravila (Transport-Level Conventions)

Aplikacija rigorozno sprovodi dve ključne konvencije na transportnom nivou:

- **Uniformni JSON protokoli:** Sva tela zahteva i odgovora primarno koriste JSON sa zaglavljem Content-Type: application/json. Jedini izuzetak od ovog pravila jeste endpoint

za uvoz Excel rasporeda (import_schedule.php), koji prihvata multipart/form-data kako bi omogućio prenos binarnih .xlsx datoteka.

- **Kredencijali i Cross-Origin zahtevi (CORS):** S obzirom na to da se tokom razvoja frontend (Vite dev server na portu 5173) i backend (Apache na portu 80) nalaze na različitim poreklima (origins), sve tri Axios instancije imaju eksplicitno podešeno withCredentials: true. Ovo obezbeđuje da se PHPSESSID session cookie automatski šalje uz svaki cross-origin zahtev. Istovremeno, svaki backend endpoint emituje odgovarajuća CORS zaglavlja (reflektujući Origin pozivaoca, dozvoljavajući kredencijale i razrešavajući preflight OPTIONS zahteve), čime je omogućena bezbedna i transparentna distribuirana komunikacija.

3.1.5 Tok autentifikacije i upravljanje sesijama (Session-Based Authentication Flow)

EduScheduler autentifikuje korisnike putem izvornog PHP mehanizma sesija, umesto korišćenja besmernih (stateless) klijentskih tokena (poput JWT-a). Autentifikacija se oslanja na **dvostepeni obrazac verifikacije (Dual-Step Authentication Pattern)** koji razdvaja serversku validaciju kredencijala od klijentskog usaglašavanja uloga. Kompletan tok se odvija kroz četiri sukcesivne faze:

- **Faza 1 - Slanje kredencijala (Credential Submission):** Korisnik unosi podatke kroz LoginForm.tsx komponentu, koja šalje HTTP POST zahtev na /login.php. Jedinstveno polje za identifikaciju prihvata ili e-mail adresu ili korisničko ime, koje backend dinamički klasifikuje proverom prisustva znaka @ (pomoću str_contains($identifier, '@')).
- **Faza 2 - Serverska verifikacija i popunjavanje sesije (Server-Side Verification & Session Population):** Backend proverava unetu lozinku u odnosu na bcrypt heš sačuvan u bazi koristeći password_verify(). Pretraga se vrši kroz tri tabele (admin_panel, professors, students). Po uspešnoj autentifikaciji, server popunjava $_SESSION autoritativnim podacima o identitetu:
    - Uloga korisnika (superadmin, admin, professor ili student)
    - Primarni ključ (ID korisnika)
    - Puno ime korisnika
    - Za studente specifično: dodeljeni ID studijskog programa, godina studija, kod i naziv programa. Sesija se vezuje za PHPSESSID kolačić koji pretraživač automatski prosleđuje pri svim narednim HTTP zahtevima.

- **Faza 3 - Klijentska re-verifikacija uloge (Client Re-Verification / Dual-Step Pattern):** Odgovor iz login.php namerno ne sadrži ulogu korisnika u JSON telu. Umesto toga, frontend odmah nakon uspešne prijave šalje sekundarni zahtev na api/check_auth.php. Ovaj endpoint direktno čita $_SESSION['user_role'] sa servera i vraća verifikovanu ulogu. Objekat AuthContext (src/context/Auth/AuthContext.tsx) skladišti ovu ulogu u React stanju. Pošto aktivna uloga potiče strogo sa servera a ne iz klijentskog skladišta (localStorage), ona se ne može lažirati ili eskalirati putem dev-alata u pretraživaču.
- **Faza 4 - Autorizacija pri svakom zahtevu (Per-Request Authorization Enforcement):** Svaki zaštićeni backend endpoint iznova proverava $_SESSION['user_role'] pre izvršavanja poslovne logike:
    - Endpoint-i kao što su students.php i send_password_reset.php odbijaju ne-administratorske pozive vratićom HTTP 403 Forbidden.
    - Endpoint-i poput student_schedule.php filtriraju SQL upite direktno na osnovu studijskog programa i godine sačuvane u autentifikovanoj sesiji. Iako klijentski zaštitnici ruta (ProtectedRoute i AdminProtectedRoute) sprečavaju navigaciju na neovlašćene stranice u UI-ju, oni služe isključivo kao UX pogodnost — server ostaje konačni arbiter bezbednosti.
- **Prekid sesije / Odjava (Session Termination)**

Prilikom odjave, klijent šalje POST zahtev na /logout.php, koji na serveru izvršava session_unset() i session_destroy(), uz poništavanje kolačića sesije u pretraživaču. Frontend resetuje svoje lokalno stanje tek nakon što primi potvrdu sa servera.

- **Arhitektonski kompromisi (Architectural Trade-Offs)**

Tradicionalni model zasnovan na PHP sesijama izabran je umesto JSON Web Tokena (JWT) iz dva ključna razloga:

1. Omogućava trenutno, centralizovano opozivanje sesije pri odjavi ili promeni privilegija na serveru.
2. Svu logiku autorizacije drži u potpunosti na backendu.

Potreba za skladištenjem stanja sesije na serveru (stateful server) predstavlja standardan i potpuno prihvatljiv kompromis za platformu obrazovne ustanove ove razmere.

3.1.6 Jedinstveni endpoint za prijavu i mehanizam razrešavanja uloga

Karakterističan element arhitekture sistema jeste jedinstveni endpoint za prijavu (login.php), koji autentifikuje sve uloge kroz jednu ulaznu tačku. Endpoint prima poslate kredencijale (identifikator i lozinku) i primenjuje trostepeni proces razrešavanja:

a) Klasifikacija identifikatora (Identifier Classification):

Backend određuje tip identifikatora proverom prisustva znaka @ pomoću PHP funkcije str_contains():

- Ako znak @ **nije prisutan**, identifikator se tretira kao korisničko ime (*username*) i upoređuje se sa zapisima u tabeli admin_panel.
- Ako je znak @ **prisutan**, klasifikuje se kao e-mail adresa i pretražuje se kroz tabele professors i students.

b) Sekvencijalna pretraga kroz tri tabele (Three-Table Sequential Lookup):

Endpoint izvršava pretragu kroz tabele fiksiranim redosledom prioriteta:

1. admin_panel (poređenje po korisničkom imenu – pokriva i admin i superadmin uloge)
2. professors (poređenje po e-mail adresi)
3. students (poređenje po e-mail adresi, uz SQL JOIN sa tabelom study_programs radi izvlačenja koda i naziva studijskog programa)

Svaki upit koristi PDO pripremljene izjave sa parametrima (bound parameters) i validira lozinku pomoću password_verify(). Prvi zapis u bazi koji zadovolji verifikaciju lozinke definitivno određuje ulogu korisnika.

c) Popunjavanje sesije i izolacija uloga (Session Population & Role Isolation):

Nakon verifikacije, server popunjava $_SESSION razrešenom ulogom, ID-jem korisnika, imenom, prezimenom, e-mailom i - za studentske naloge - podacima study_program_id, study_year, study_program_code i study_program_name. Heš lozinke se eksplicitno isključuje, a sama uloga se izostavlja iz JSON odgovora koji se vraća pretraživaču, čime se zahteva klijentska verifikacija preko check_auth.php.

(Napomena: Zasebni endpoint admin_login.php postoji za namensku administratorsku SPA rutu sa dodatnom zaštitom od eskalacije privilegija, gde vraća HTTP 403 ako je korisnik već autentifikovan kao ne-administrator).

Arhitektonske prednosti (Architectural Consequences)

Ovaj jedinstveni dizajn donosi dve ključne arhitektonske prednosti:

- **Pojednostavljeno korisničko iskustvo (Simplified UX):** Korisnici ne moraju ručno da biraju svoju ulogu (npr. putem padajućeg menija ili radio-dugmadi) pre autentifikacije. Ovo svodi javnu formu za prijavu (LoginForm.tsx) na jedno polje "E-mail ili korisničko ime", dok sistem neprimetno usmerava administratore, profesore i studente na njihove odgovarajuće kontrolne table.
- **Centralizovana autorizacija i bezbednost uloga (Centralized Security):** Određivanje uloge je strogo ograničeno na serversku pretragu, što čini osnovu zaštite od lažiranja identiteta i neovlašćene eskalacije privilegija.

3.2 Arhitektura frontend-a

Ovo poglavlje detaljno opisuje arhitekturu klijentske strane sistema EduScheduler, obuhvatajući njegovu strukturu komponenti, mehanizme rutiranja, kuke (*hooks*) za upravljanje stanjem i slojeve za integraciju API-ja.

3.2.1 Hijerarhija komponenata i organizacija datoteka

Frontend je organizovan unutar src/ direktorijuma prema slojevitoj strukturi usmerenoj na funkcionalnosti. Prikazano u tabeli 3.2.1.1. Ulazne datoteke na najvišem nivou uspostavljaju osnovno okruženje (*shell*) aplikacije:

- **index.tsx:** Kreira instancu React korena (*root*), instancira QueryClient i montira RouterProvider.
- **App.tsx:** Služi kao komponenta korenskog rutiranja koja obavija svaku stranicu u AuthProvider, ErrorBoundary, globalne styled-components stilove i React Toastify kontejner pre nego što renderuje <Outlet/> za aktivnu podređenu rutu.

Direktorijum	**Opis i ključne komponente**
src/api/	Tri Axios klijenta (api.ts, adminApi.ts, publicApi.ts) i tipizirane API funkcije za rad sa rasporedom (scheduleApi.ts).
src/components/	Reusable UI komponente: zaštitnici ruta (ProtectedRoute, AdminProtectedRoute), ScheduleGrid, modalni dijalozi (Modal, ConfirmModal, ProfessorModal, ClassroomModal, StudentModal, AdminModal, ResetLinkModal), zaglavlje, bočne trake, ErrorBoundary, LoginForm.
src/context/	Konteksti autentifikacije i upravljanje stanjem sesije: Auth/ (AuthContext, useAuth) i AdminAuth/ (AdminAuthContext, useAdminAuth).
src/mutations/	TanStack Query kuke za upis i izmenu podataka (uključujući admin/ poddirektorijum za administrativne akcije).

Direktorijum	Opis i ključne komponente
src/pages/	Prikazi na nivou ruta: Home/, Admin/ (AdminDashboard, AdminLoginPage, ManualScheduler, Rasporedi), ResetPassword.
src/queries/	TanStack Query kuke za čitanje i dohvatanje podataka (uključujući admin/ poddirektorijum).
src/routes/	Definicija centralnog rutera aplikacije (Routes.tsx).
src/schemas/	Zod šeme za validaciju podataka i formi.
src/styles/	Definicije stilova bazirane na Styled Components (styled.ts, GlobalStyle).
src/types/	TypeScript interfejsi i tipovi za domenski model (studenti, profesori, raspored, učionice).
src/utils/	Pomoćne (utility) funkcije i logika (routes.ts, nextLecture.ts, gridUtils.ts).

Table 3.2.1.1: Arhitektura i organizacija src/ direktorijuma klijentske aplikacije

Prednosti arhitekture

Ovakva struktura nalaže razdvajanje nadležnosti (*separation of concerns*) po modulima: prikazi ruta (*route views*) sastavljaju prezentacione komponente, preuzimaju asinhrono stanje sa servera putem kuka za upite (*query hooks*) i izvršavaju izmene putem kuka za mutacije (*mutation hooks*), bez ugradnje direktne HTTP logike ili sirovih upitnih niski (*query strings*).

3.2.2 Upravljanje stanjem: React Context API + TanStack Query (State Management)

Arhitektura klijentskog dela aplikacije svesno deli stanje aplikacije na dve nezavisne kategorije, umesto spajanja svih podataka u jedan monolitni globalni store (poput Redux-a). Svaka kategorija je poverena namenskom mehanizmu koji odgovara njenom životnom ciklusu:

1. Stanje autentifikacije i sesije (React Context API):

Stanje autentifikacije se vodi preko React Context API-ja i podeljeno je na dva nezavisna provajdera:

- **AuthContext (AuthContext.tsx):** Upravlja opštim stanjem korisnika i izlaže strukturu AuthContextType (isLoggedIn, loading, professorName, surname, email, role, studentProfile, kao i funkcije login i logout). Podaci se ne uzimaju iz neproverenog klijentskog skladišta, već se pri svakom učitavanju stranice iznova verifikuju pozivom na check_auth.php.
- **AdminAuthContext (AdminAuthContext.tsx):** Upravlja namenskom sesijom za administratorsku aplikaciju (adminLoggedIn, adminUsername). Za razliku od opšteg konteksta, stanje administratora se perzistira u localStorage-u radi brzog odziva SPA interfejsa, ali i dalje zadržava serversku verifikaciju uloga pri svakom mrežnom zahtevu.

2. Stanje serverskih podataka (TanStack Query):

Svi podaci koji potiču iz baze podataka (professors, students, classrooms, schedule entries, reservations, imports) u isključivom su vlasništvu biblioteke TanStack Query (v5):

- **Operacije čitanja (Read Operations):** Realizovane su kroz prilagođene kuke upita (*query hooks*) sa stabilnim ključevima keša (npr. ["admin-students"], ["classrooms"], ["manual-grid"], ["schedule-imports"]).
- **Operacije izmene (Write Operations):** Formulisane su kao kuke mutacija (*mutation hooks*) koje po uspešnom izvršenju automatski invalidiraju odgovarajuće keš ključeve, izazivajući automatsko osvežavanje zavisnih prikaza.
- **Centralizovani keš:** Svi upiti dele jedinstvenu QueryClient instancu montiranu u korenu aplikacije.
- **Arhitektonsko obrazloženje (Architectural Rationale)**

Ova podela direktno odražava razliku u životnom ciklusu podataka. Stanje autentifikacije je dugovečno, bezbednosno osetljivo i proverava se retko - zbog čega je React Context idealan kontejner. Sa druge strane, domenski podaci iz baze su volatilni, deljeni između više korisnika i zahtevaju trenutnu invalidaciju keša nakon svake mutacije - što asinhroni sloj za keširanje čini optimalnim rešenjem.

3.2.3 Strategija rutiranja i zaštita ruta

Rutiranje na klijentskoj strani povrena je biblioteci **React Router DOM v7** uz korišćenje *Data Router API-ja*. Tabela ruta je deklarisana u src/routes/Routes.tsx pomoću createBrowserRouter i montirana u index.tsx preko RouterProvider-a.

Struktura aplikacije se deli na tri glavne rute unutar korene App komponente:

- **"/" (HomePage):** Javni prikaz aplikacije koji sadrži formu za prijavu i pregled rasporeda.
- **"/reset-password" (ResetPasswordPage):** Interfejs za ponovno postavljanje lozinke putem bezbednosnog tokena.
- **"/admin" (AdminRoutes):** Zaštićena administratorska kontrolna tabla.
- **Mehanizam zaštite ruta (Custom Guard Components)**

Pristup rutama se kontroliše pomoću dve prilagođene komponente-zaštitnika (*guard components*):

- **ProtectedRoute:** Proverava fleg isLoggedIn iz AuthContext-a. Ako korisnik nije autentifikovan, automatski ga preusmerava na početnu rutu (/).
- **AdminProtectedRoute:** Sprovodi strogu proveru uloga. Dovoljava pristup isključivo ako je serverom verifikovana uloga jednaka admin ili superadmin (iz odgovora check_auth.php). Neautentifikovani ili neovlašćeni korisnici se preusmeravaju na početnu stranicu.

Sama administratorska podruta "/admin" obmotana je sklopom gde je AdminAuthProvider ugnežđen unutar AdminProtectedRoute komponente, čime se garantuje da je kompletan administratorski UI zaključan iza serverske provere uloga. Centralizovani putevi definisani su u src/utils/routes.ts (PagePaths.HOME, PagePaths.ADMIN, PagePaths.RESET) kako bi se izbeglo harcod-ovanje stringova u komponentama.

3.2.4 Apstrakcija API sloja

Kako bi se mrežna komunikacija u potpunosti odvojila od korisničkog interfejsa i React komponenti, klijentski deo aplikacije centralizuje sve HTTP zahteve u namenskom sloju apstrakcije pod src/api/.

Sloj se bazira na tri zasebne Axios instancije, od kojih je svaka mapirana na odgovarajući backend direktorijum i konfigurisana sa withCredentials: true i zaglavljem Content-Type: application/json:

- **api.ts (api):** Cilja http://localhost/EduScheduler/api i rukuje opštim REST resursima (profesori, studenti, učionice, rezervacije, zauzetost sala i re-verifikacija sesije check_auth.php).

- **adminApi.ts (adminApi, logoutApi):** Cilja http://localhost/EduScheduler/admin za administrativne skripte (CRUD nad korisnicima i sistemskim podacima) i odjavljivanje sa sistema.
- **publicApi.ts (publicApi):** Cilja http://localhost/EduScheduler za javno čitanje rasporeda, ručno upravljanje mrežom rasporeda i uvoz Excel datoteka (import_schedule.php).
- **Tipizirani API omotači (Typed API Wrappers)**

Na vrhu ovih Axios instancija, modul src/api/scheduleApi.ts enkapsulira pozive ka endpoint-ima u tipizirane as hishrone funkcije:

- getStudentSchedule
- getScheduleEntries
- importSchedule
- getManualScheduleGrid
- createManualEntry i deleteManualEntry

Prilagođene TanStack Query kuke (u queries/ i mutations/) pozivaju isključivo ove abstrahovane funkcije, čime je postignuto da se promene u backend URL putanjama ili strukturi zahteva rešavaju na samo jednom mestu u kodu.

3.2.5 Tipski sigurni modeli podataka sa TypeScript interfejsima

S obzirom na to da frontend i backend razmenjuju podatke preko HTTP-a u vidu ne-tipiziranih JSON objekata, frontend ponovo uspostavlja strog statički ugovor o tipovima pomoću TypeScript interfejsa. Domenski model je deklarisan unutar modula pod src/types/ i objedinjen kroz src/types/index.ts, oslikavajući 12 tabela relacione baze podataka.

Core Domenski Interfejsi:

- **Admin, Professor, Classroom, Student, StudyProgram (types/admin.ts):** Direktno preslikavaju relacione tabele. Interfejs Student uključuje study_program_id, study_year, kao i razrešene atribute study_program_code i study_program_name koji odgovaraju SQL JOIN strukturi sa backend-a.
- **ScheduleEntry, ManualEntryPayload, ImportReport, ScheduleImport, ImportError (types/schedule.ts):** Modeluju stavke rasporeda, podatke za ručni unos i dijagnostiku uvoza Excel fajlova. ScheduleEntry obuhvata classroom_id, denormalizovani room_name, kao i opcione atribute semester i school_year.
- **Pomoćni interfejsi (types/classroom.ts, types/appointments.ts):** Definišu dostupnost učionica, mape spratova i objekte za zakazivanje termina.

Primena tipova na granicama sistema

Ovi interfejsi se rigorozno sprovode na tri ključna mesta u aplikaciji:

1. **Povezivanje API odgovora:** Kuke upita eksplicitno kastuju mrežne odgovore (res.data as Student[]), garantujući da UI komponente dobijaju strogo tipizirane nizove.
2. **Validacija formi (Zod Schemas):** Zod šeme unutar src/schemas/ vrše validaciju korisničkog unosa u realnom vremenu i generišu TypeScript tipove koji odgovaraju API interfejsima.
3. **Poravnanje sesije:** Podtip StudentProfile unutar AuthContext-a povezuje podatke iz serverske sesije sa istim modelom studenta koji se koristi u ostatku aplikacije.

3.3 Arhitektura backend-a

Ovo poglavlje opisuje organizaciju serverske strane sistema EduScheduler, detaljno izlažući njegovu strukturu direktorijuma, podelu sa proceduralnog na objekatno-orijentisani servisni sloj, konekcije sa bazom podataka i tok izvršavanja ruta (*endpoints*). Sto je najbolje prikazano na slici

3.3.1 Struktura direktorijuma i razdvajanje nadležnosti

Backend aplikacije je organizovan u četiri funkcionalne oblasti, od kojih svaka adresira jasnu arhitektonsku odgovornost:

- **functions/ (Kros-kating infrastruktura):** Sadrži centralne pomoćne funkcije, vezu sa bazom podataka (db.php), inicializaciju sesije (init.php) i biblioteku zajedničkih helpera (functions.php).
- **api/ (REST API JSON Endpoint-i):** Sadrži 30 JSON endpoint skripti koje konzumira klijentska SPA aplikacija (profesori, studenti, učionice, raspored, rezervacije sala, zauzetost prostorija, uvoz Excel-a, provera sesije).
- **admin/ (Administrativne upravljačke skripte):** Sadrži 24 skripte koje implementiraju CRUD operacije nad domenom (profesori, studenti, učionice, administratori), namenski login i celokupan tok za resetovanje lozinki.
- **app/ (Objektno-orijentisani sloj servisa):** Imenski prostor App\ u kom je izolovan sloj servisa naprednog Excel uvoza, razrešavanja nastavnika i otkazivanja časova. Sadrži pod-slojeve Container/ (PHP-DI), Database/, Http/ i Services/.

Arhitektonsko razdvajanje i evolucija obrasca dizajna

Ova struktura uspostavlja strogo razdvajanje odgovornosti (*Separation of Concerns*):

1. **Skripte endpoint-a (api/, admin/):** Zadužene su za presretanje HTTP zahteva, proveru sesije i formatiranje JSON odgovora. Svaka skripta gađa jednu operaciju resursa, održavajući API površinu čistom.
2. **Infrastruktura (functions/):** Zajedničke mogućnosti - uključujući konekciju sa bazom i sanitizaciju unosa - centralizovane su u višekratnim pomoćnim funkcijama.
3. **Kompleksni poslovni domen (app/):** Višefazni algoritmi (poput parsiranja Excel-a i fuzzy uparivanja imena) izolovani su u objektno-orijentisane, imenski prostorno izolovane PHP klase sa injekcijom zavisnosti, koje se automatski učitavaju putem Composer PSR-4 standarda ("App\\": "app/").

3.3.2 Sloj konekcije sa bazom podataka

Sistem svesno kombinuje tri mehanizma konekcije sa bazom u zavisnosti od složenosti i bezbednosnih zahteva posla:

- **Proceduralni MySQLi omotač (functions/db.php):** Otvara konekciju ka eduschedulerdb bazi (mysqli_connect('localhost', 'root', '', 'eduschedulerdb')) i izlaže globalne omotače: escape(), query(), confirm(), fetch_assoc() i fetch_all(). Ovaj sloj služi kao veza za standardne CRUD endpoint-e.
- **PDO (PHP Data Objects):** Konfigurisan sa eksplicitnim rukovanjem izuzecima (ERRMODE_EXCEPTION) i imenovanim parametrima. Koristi se u bezbednosno najkritičnijim tokovima (login.php, admin_login.php, reset_password.php) kako bi se garantovala imunost na SQL injection napade.
- **Servisna konekcija (app/Database/Connection.php):** Moderni servis za uvoz Excel-a enkapsulira sopstvenu Connection klasu. Ova finalna klasa inicijalizuje MySQLi vezu sa eksplicitnim utf8mb4 skupom karaktera i injektuje se automatski putem PHP-DI kontejnera.

Svi mehanizmi komuniciraju sa istom bazom i garantuju ispravan upis i čitanje srpskih dijakritičkih znakova (č, ć, š, ž, đ) kroz UTF-8 enkodiranje.

3.3.3 Inicijalizacija i pomoćne funkcije (functions/init.php, functions.php)

Skripta functions/init.php deluje kao globalni bootstrap fajl koji se uključuje na samom početku gotovo svake backend skripte. Njen tok izvršavanja obuhvata tri koraka:

1. Poziva session_start() radi pokretanja ili nastavka PHP sesije.
2. Uključuje functions/db.php radi uspostavljanja konekcije sa bazom.
3. Uključuje functions/functions.php radi učitavanja sistemskih helpera.

Domenski helperi (functions.php)

Fajl functions.php pruža ključne zajedničke funkcije:

- **Sanitizacija i izlaz:** Funkcije clean() (obmotava htmlentities()), redirect(), set_message() i display_message().
- **Engine za preslovljavanje (latinToCyrillicSerbian):** Funkcija koja vrši konverziju teksta iz latinice u ćirilicu radi poravnanja imena profesora koja mogu biti različito napisana u tabelama i uvezenim fajlovima.

- **Detekcija konflikta rasporeda (findScheduleConflict):** Implemenira provere zauzetosti sala i profesora. Primajući PDO konekciju, gradi dinamičke šablone pretrage i izvršava parametrizovane upite nad schedule_entries tabelom radi sprečavanja preklapanja termina.

3.3.4 Organizacija API endpoint-ova (api/ Direktorijum)

Direktorijum api/ implementira kompletan REST ugovor kroz 30 pojedinačnih endpoint skripti. Svaka skripta funkcioniše po ujednačenom ciklusu izvršavanja (Execution Lifecycle):

1. Uključuje init.php (sesija i baza).
2. Emituje CORS zaglavlja (reflektuje Origin, dozvoljava kredencijale, razrešava OPTIONS preflight).
3. Postavlja zaglavlje Content-Type: application/json.
4. Sprovodi autentifikaciju i proveru uloga ($_SESSION['user_role']).
5. Parsira unost (URL query parametre za GET ili JSON body za POST).
6. Izvršava parametrizovane upite nad bazom.
7. Vraća strukturirani JSON odgovor uz odgovarajući HTTP status kod (200, 400, 401, 403, 500).

Funkcionalno grupisanje endpoint-ova:

- **Korisnici i referentni podaci:** professors.php, students.php (izvršava JOIN sa study_programs), classrooms.php, study_programs.php. Pristup zaštićenim čitanjima vraća HTTP 403 ako korisnik nema odgovarajuću ulogu.
- **Podaci o rasporedu:** schedule_entries.php, student_schedule.php (filtrirano strogo prema sesiji studenta), manual_schedule_grid.php, create_manual_entry.php, delete_manual_entry.php.
- **Rezervacije i ad-hoc zakazivanja:** room_reservations.php, busy_classrooms.php, busy_insert.php, delete_busyclassroom.php, scheduled_appointments.php, update_reservation.php, delete_reservation.php.
- **Uvoz i praćenje:** import_schedule.php (ulazna tačka za Excel pipeline), schedule_imports.php (lista uvoza), delete_schedule_import.php (uklanjanje uvoza i povezanih unosa preko ON DELETE CASCADE).
- **Razrešavanje nastavnika:** resolve_teachers.php, unresolved_entries.php, set_entry_teacher.php.

- **Sidro autorizacije:** check_auth.php (autoritativni endpoint koji vraća ulogu i profil iz sesije).

3.3.5 Skripte administratorskog panela (admin/ Direktorijum)

Direktorijum admin/ sadrži 24 skripte namenjene administrativnoj kontrolnoj tabli. Sve skripte započinju CORS uvodom i proverom autorizacije - bilo koji ne-administratorski poziv se trenutno odbija uz HTTP 403 Forbidden odgovor.

CRUD (create, read, update, delete) operacije i parametrizovani upiti

Administrativni endpoint-i primenjuju MySQLi prepared statement-e sa vezanim parametrima:

- add_student.php: Izvršava INSERT upit gde se prazne vrednosti za study_program_id i study_year konvertuju u SQL NULL, omogućavajući kreiranje studenta pre dodele studijskom programu.
- update_student.php, delete_student.php, add_professor.php, update_professor.php, delete_professor.php, add_classroom.php, update_classrooms.php, delete_classroom.php, add_admin.php.

Pod-tok za resetovanje i oporavak naloga

- admin_login.php: Autentifikuje administratore preko korisničkog imena u admin_panel tabeli koristeći PDO i password_verify(). Blokira pokušaje eskalacije privilegija iz postojećih korisničkih sesija.
- send_password_reset.php & send_student_reset.php: Generišu kriptografski siguran token od 64 karaktera (bin2hex(random_bytes(32))), skladište ga u bazi sa rokom važenja od 1 sata (INTERVAL 1 HOUR) i šalju e-mail sa linkom za oporavak.
- reset_password.php: Verifikuje token u odnosu na vremensku zonu Europe/Belgrade, upisuje novu heširanu lozinku (password_hash) u tabele professors ili students, briše token i potvrdjuje uspeh.

3.3.6 Pipeline za uvoz Excel rasporeda (app/Services/)

Najkompleksnija backend funkcionalnost - masovni uvoz rasporeda iz Excel .xlsx datoteka - izolovana je u objektno-orijentisanom sloju servisa i komponovana preko PHP-DI kontejnera (ContainerFactory). [12]

Pipeline se sastoji od sekvencijalnih servisa abstrahovanih iza interfejsa:

1. **ScheduleFilenameParser:** Iz naziva otpremljene datoteke izdvaja metapodatke (učionica, semestar, školska godina).

2. **ScheduleExcelReader:** Koristeći PhpSpreadsheet biblioteku, čita radni list u dvodimenzionalnu matricu uz očuvanje spojenih ćelija (*merged cells*).
3. **ScheduleParser:** Prema interfejsima ExcelGridBuilder i CellClassifier, konvertuje matricu u kandidate ScheduleEntryDTO objekata.
4. **ScheduleValidator:** Proverava pravila domena (validnost dana/vremena, duplikate i preklapanja termina) i vraća ValidationResult sa listom validnih i nevažećih unosa.
5. **ScheduleImporter:** Perzistira validne unose unutar baze podataka u okviru jedne transakcije pod generisanim import_id ključem i vraća finalni ImportReport (uspešni, preskočeni, greške i trajanje u ms).

Endpoint api/import_schedule.php deluje kao HTTP fasada: proverava sesiju, validira fajl preko ScheduleImportRequest (limit 10 MB, provera MIME tipa, provera ekstenzija .xlsx/.xls), skladišti datoteku u uploads/schedules/, pokreće kontejner i vraća JSON izveštaj.

3.3.7 Mehanizam za razrešavanje imena nastavnika (TeacherNameMatcher)

Prilikom uvoza rasporeda, imena nastavnika u Excel datotekama često ne odgovaraju kanonskim zapisima u bazi (zbog upotrebe ćirilice/latinice, titula "dr", skraćenica ili slovnih grešaka). Servis TeacherNameMatcher pod app/Services/ rešava ovaj problem kroz algoritam višestrukog uparivanja:

1. **Parsiranje i ekstrakcija:** Izvlači sirovo tekstualno polje teacher_raw iz uvoza.
2. **Normalizacija:** Uklanja titule, normalizuje Unicode karaktere i vrši preslovljavanje pisma putem latinToCyrillicSerbian().
3. **Strategije uparivanja:** Primenjuje sukcesivne nivoe proveravanja - tačno podudaranje, podudaranje po inicijalu i prezimenu, i napredne LIKE šablone.
4. **Upravljanje nerazrešenim unosima:** Unosi koji se ne mogu pouzdano upariti dobijaju status nerazrešenih i prikazuju se administratoru na api/unresolved_entries.php. Administrator putem api/set_entry_teacher.php vrši ručnu dodelu.

Sistem beleži sve potvrdjene relacije u tabelu teacher_aliases, čime obezbeđuje da se svaki sledeći uvoz sa istim alternativnim imenom razrešava automatski i idempotentno.

3.3.8 Upravljanje otkazanim časovima (Cancelled Classes)

Podsistem za otkazane časove oslanja se na tabelu cancelled_classes, koja beleži sve izmene u redovnom rasporedu. Svaki zapis čuva referencu na originalni unos u rasporedu (schedule_entry_id) i, ukoliko postoji, referencu na zamenski unos.

API endpoint api/cancelled_classes.php izlaže kompletan REST CRUD interfejs. Na klijentskoj strani, vizuelizator rasporeda presreće ove podatke i odražava otkazane ili pomerene časove kroz posebne vizuelne indikatore (označene ili sive ćelije), dok administratorski panel pruža namensku kontrolnu tablu za upravljanje izmenama u realnom vremenu.

3.3.9 Sistem šablona e-mail obaveštenja (Email Notification System)

Sistem za slanje e-mail obaveštenja baziran je na biblioteci **PHPMailer** i koristi dinamičke HTML šablone.

- **Upravljanje šablonima:** Skripta admin/email_templates.php pruža interfejs za pregled, dok admin/update_email_template.php omogućava administratorima izmenu teksta obaveštenja directno iz panela.
- **Generisanje poruka:** Tokovi za resetovanje lozinke (send_password_reset.php i send_student_reset.php) učitavaju odgovarajući HTML šablon i vrše dinamičku zamenu rezervisanih mesta (placeholders) podacima kao što su ime korisnika, generisani token/link za resetovanje i naziv ustanove.
- **Isporuka poruka:** Poruke se šalju preko Gmail SMTP servera uz obaveznu STARTTLS enkripciju na portu 587, čime se obezbeđuje visoka pouzdanost isporuke i zaštita od pristupa trećih lica.

3.4 Dizajn baze podataka

Sloj za trajno čuvanje podataka (*persistent data tier*) čini baza podataka **eduschedulerdb**, koja radi pod okruženjem MariaDB 10.4 uz upotrebu InnoDB mehanizma za skladištenje (*storage engine*) i utf8mb4 kodiranja znakova (koristeći utf8mb4_general_ci ili utf8mb4_unicode_ci kolacije).

InnoDB je izabran jer obezbeđuje potpunu transakcionu ACID pouzdanost i sprovodi restrikcije stranih ključeva (*foreign-key constraints*) kroz celu šemu baze.

Baza podataka obuhvata dvanaest osnovnih relacionih tabela:

- admin_panel
- busy_classroms
- cancelled_classes
- classrooms
- email_templates
- professors
- schedule_enteries
- schedule_entry_study_programs
- schedule_imports
- students
- study_programs
- teacher_aliases

Naredni pododeljci dokumentuju kolone, tipove podataka i primarne uloge svake tabele u okviru sistema, nakon čega sledi pregled relacija stranih ključeva koje održavaju referencijalni integritet. [13]

3.4.1 Pregled odnosa entiteta (ER dijagram)

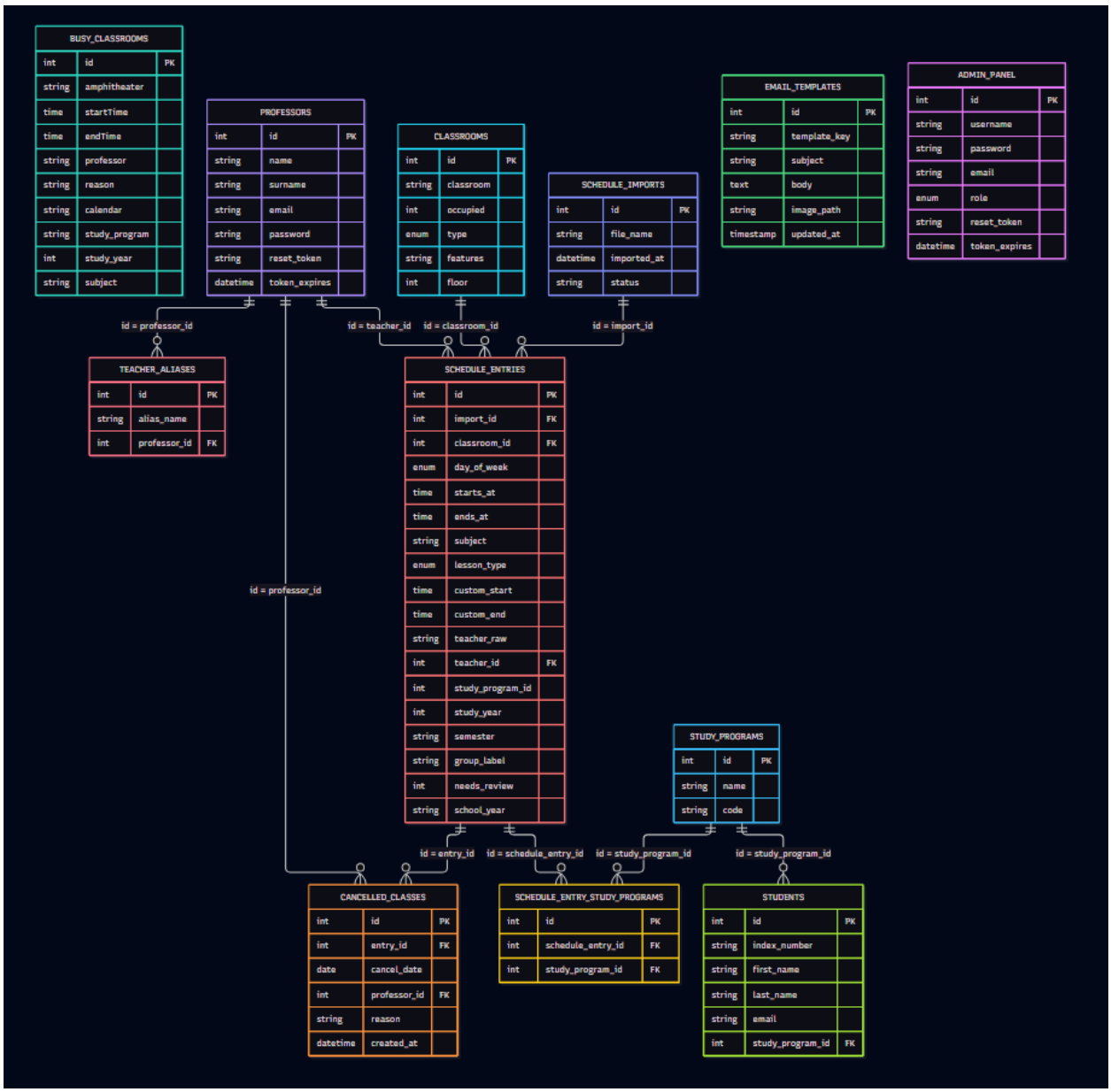
Šema formira zvezdu oko rasporeda. Centralna tabela činjenica je schedule_entries, koja povezuje ključne entitete sistema. Pivot tabela (schedule_entry_study_programs), tabela otkazanih časova (cancelled_classes) i tabela alijasa profesora (teacher_aliases) proširuju osnovni relacioni model:

- **students.study_program_id → study_programs.id** (više-prema-jednom)
- **schedule_entries.import_id → schedule_imports.id** (više-prema-jednom)
- **schedule_entries.classroom_id → classrooms.id** (više-prema-jednom)
- **schedule_entries.teacher_id → professors.id** (više-prema-jednom)

- **schedule_entry_study_programs.schedule_entry_id → schedule_entries.id** (vezna / pivot)
- **schedule_entry_study_programs.study_program_id → study_programs.id** (vezna / pivot)
- **cancelled_classes.entry_id → schedule_entries.id** (više-prema-jednom)
- **cancelled_classes.professor_id → professors.id** (više-prema-jednom)
- **teacher_aliases.professor_id → professors.id** (više-prema-jednom)

Usklađenost sa domenom (Domenska usklađenost):

ER model odražava pravila domena: stavka u raspordu zahteva učionicu i studijski program, svaki student pripada tačno jednom studijskom programu i godini studija, dok zapis o uvozu grupiše sve stavke izvučene iz jednog otpremljenog Excel fajla. Formalni ER dijagram prikazan je na slici 3.4.1.1.



Slika 3.4.1.1: Odnos između tabela u bazi podataka

3.4.2 Specifikacija strukture 12 tabela baze podataka

1. Tabela admin_panel

Sadrži administrativne naloge.

id (PK, AUTO_INCREMENT), username (NOT NULL), password (bcrypt heš). Svaki red je validan administrator; uloga je fiksirana na 'admin' ili 'superadmin'.

2. Tabela professors

Sadrži evidenciju o profesorskom osoblju.

id (PK), name, surname, email (jedinstven), password (bcrypt heš), reset_token, token_expires (za tok resetovanja lozinke).

3. Tabela students

Evidencija o studentima sa referencom na studijski program.

id (PK), name, surname, email (jedinstven, UNIQUE KEY uq_students_email), password (bcrypt heš), study_program_id (FK -> study_programs.id), study_year (tinyint, 1-6), status (default 'active'), reset_token, token_expires, created_at (timestamp). Složeni indeks idx_students_program_year na (study_program_id, study_year).

4. Tabela study_programs

Katalog akademskih studijskih programa (smerova).

id (PK, AUTO_INCREMENT), code (jedinstven, npr. "INI"), name. Odnosi se na ~20 studijskih programa raspoređenih po inženjerskim, saobraćajnim, arhitektonskim i strukovnim disciplinama.

5. Tabela classrooms

Katalog učionica i prostorija.

id (PK), classroom (oznaka prostorije, npr. "U1"), occupied (fleg), type (ENUM 'classroom'|'amphitheater'), features (lista opreme), floor (int).

6. Tabela schedule_entries

Centralna tabela činjenica za sve unose i časove rasporeda.

id, import_id (FK -> schedule_imports.id, CASCADE), classroom_id (FK), day_of_week (ENUM mon-sun), starts_at (time), ends_at (time), subject, lesson_type (ENUM lecture/exercise/custom/srpske varijante), custom_start, custom_end, teacher_raw (slobodan tekst), teacher_id (FK kandidat -> professors), study_program_id (FK -> study_programs.id), study_year, semester, group_label, created_at, updated_at. Višestruki složeni indeksi za renderovanje mreže, provere konflikata i brisanje uvoza.

7. Tabela schedule_imports

Prati metapodatke o uvozima datoteka.

id, filename, original_name, stored_path, school_year, semester, room_name, status (completed/partial/failed), imported_by, created_at. Skladišti ishod i parametre izvršenog uvoza.

8. Tabela busy_classrooms

Privremena zauzeća i ad-hoc rezervacije prostorija.

id, classroom_id, date, starts_at, ends_at, description. Prati vanredne događaje koji privremeno zauzimaju učionicu u mreži rasporeda.

9. Tabela schedule_entry_study_programs

Vezna tabela (Junction Table) više-prema-više ($m:n$) između časova i smerova.

id, schedule_entry_id (FK -> schedule_entries.id), study_program_id (FK -> study_programs.id), import_id (FK -> schedule_imports.id). Omogućava više-programsko zakazivanje nastave. Indeksi: idx_esp_import, idx_esp_program, idx_esp_entry na odgovarajućim kolonama.

10. Tabela cancelled_classes

Prati otkazane ili zamenjene časove.

id, original_entry_id (FK -> schedule_entries.id), replacement_entry_id (FK -> schedule_entries.id, nullable), reason (text), cancelled_by (FK -> admin_panel.id), cancelled_at (timestamp). Prati relaciju između originalnog i zamenskog časa.

11. Tabela teacher_aliases

Čuva prepoznate alijase nastavnika radi idempotentnosti uvoza.

id, professor_id (FK -> professors.id), raw_text (ime kako se pojavljuje u uvozu), normalized_text (kanonski oblik), school_year, match_confidence, created_at. Indeksi: uq_year_raw na (school_year, raw_text), idx_teacher_aliases_professor_id.

12. Tabela teachers (Referentni model razrešavanja)

Referentna tabela/prikaz koja povezuje id i profesorske zapise radi bržeg uparivanja u servisu TeacherNameMatcher.

4. DETALJI IMPLEMENTACIJE

Ovo poglavlje predstavlja konkretnu implementaciju sistema *EduScheduler*, oslanjajući se na arhitektonske postavke opisane u Poglavlju 3. U nastavku je dat detaljan pregled izvornih artefakata frontend i backend sloja, sa objašnjenjem načina na koji je svaka komponenta realizovana u kodu i kako pojedinačni moduli komuniciraju u izvršnom okruženju (runtime). Poglavlje je podeljeno na dve glavne celine: frontend implementaciju (Sekcija 4.1) i backend implementaciju (Sekcija 4.2), uz prateće sekcije o migracijama baze podataka (Sekcija 4.3) i integraciji validacije formi (Sekcija 4.4). Sve reference direktno odgovaraju strukturi produkcionog koda, uz navođenje ključnih isečaka koda koji ilustruju inženjerske i implementacione odluke.

4.1 Frontend implementacija

Frontend aplikacije realizovan je kao React 19 jedinstvena veb stranica (Single-Page Application – SPA), pisana u TypeScript jeziku i pripremljena pomoću Vite alata. Izvorni kod unutar src/ direktorijuma prati slojevitu organizaciju opisanu u sekciji 3.2.1. Implementacija preferira male, fokusirane komponente koje se komponuju putem prilagođenih kuka (custom hooks); mrežni pristup je usmeren kroz centralizovani API sloj, dok se stanjem servera upravlja pomoću biblioteke TanStack Query.

4.1.1 Ulazna tačka aplikacije i korensko stablo (App.tsx)

Aplikacija se pokreće unutar src/index.tsx fajla, koji kreira React koren (root), instancira jedinstveni QueryClient i montira ruter provajder:

```
createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <QueryClientProvider client={queryClient}>
      <RouterProvider router={router} />
    </QueryClientProvider>
  </StrictMode>
);
```

Korenska komponenta, src/App.tsx, uspostavlja poprečne funkcionalnosti (cross-cutting concerns) koje dele sve rute u aplikaciji. Ona obavija rutirani sadržaj u četiri osnovna sloja provajdera:

- **AuthProvider** (src/context/Auth/AuthContext.tsx) – Obezbeđuje opštu korisničku sesiju i stanje autentifikacije za celokupno stablo komponenti.
- **ErrorBoundary** (src/components/ErrorBoundary.tsx) – Hvata JavaScript izuzetke u fazi renderovanja i prikazuje namensku stranicu za greške umesto praznog ekrana.
- **GlobalStyle** (src/styles/styled.ts) – Ubacuje osnovne globalne CSS stilove (uključujući box-sizing pravila i resete za body element) pomoću styled-components biblioteke.

- **ToastContainer** (react-toastify) – Renderuje stog sistemskih obaveštenja, konfigurisan sa pozicioniranjem u donjem desnom uglu, automatskim zatvaranjem nakon tri sekunde i obojenom vizuelnom temom ("colored").

Nakon navedenih provajdera, App renderuje <Outlet/> element u koji React Router dinamički ubacuje aktivnu podređenu rutu (HomePage, ResetPasswordPage ili administratorsko podstablo). Budući da App deluje kao korenski raspored (root layout) unutar Routes.tsx, svaka ugnežđena ruta automatski nasleđuje globalno stanje autentifikacije, obradu grešaka, stilove i obaveštenja bez dupliciranja koda.

4.1.2 Konfiguracija rutiranja (Routes.tsx)

Rutiranje je definisano u src/routes/Routes.tsx korišćenjem React Router API-ja za podtake (createBrowserRouter) i montirano preko RouterProvider-a. Tabela ruta ugnežđuje tri podređene rute ispod korenskog App rasporeda:

```
const router = createBrowserRouter([
  {
    element: <App />,
    children: [
      { path: PagePaths.HOME, element: <HomePage /> },
      { path: PagePaths.RESET, element: <ResetPasswordPage />
},
      { path: PagePaths.ADMIN, element: <AdminRoutes /> },
    ],
  },
]);
```

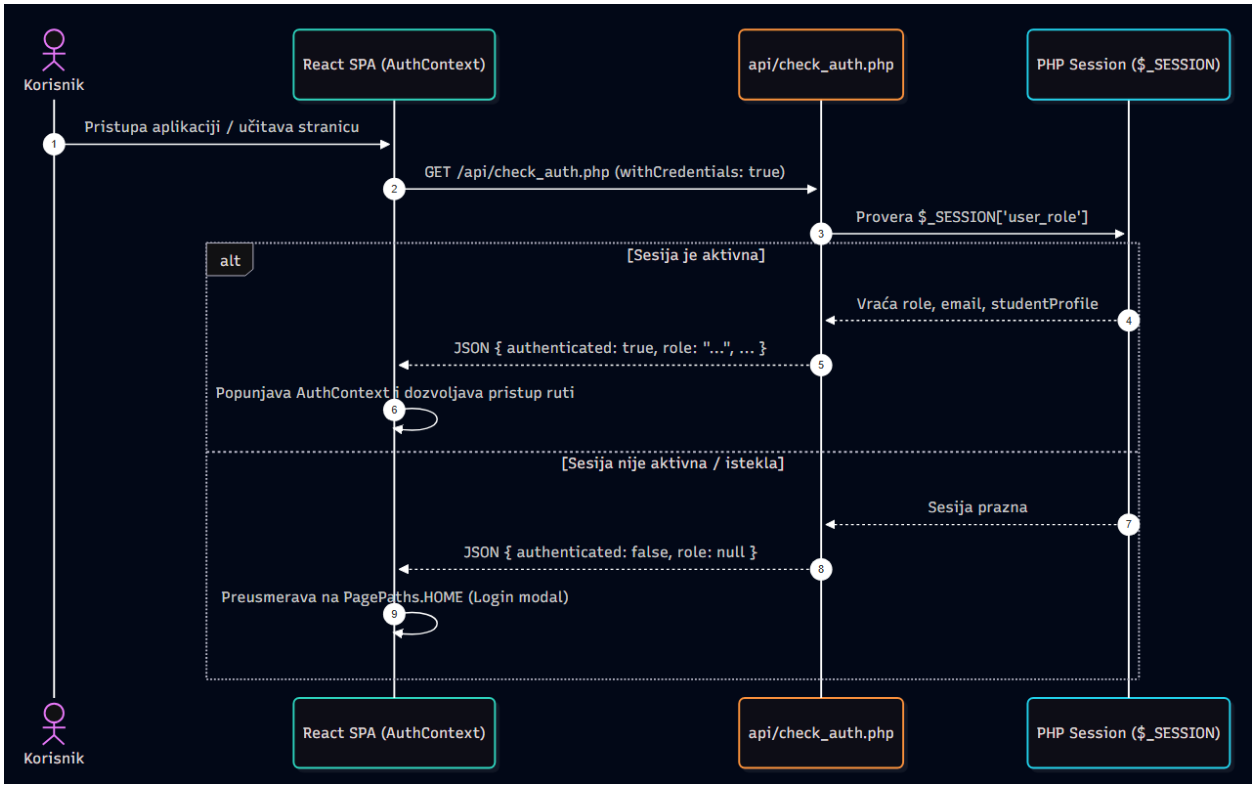
Administratorska ruta (ADMIN) predstavlja komponovano podstablo, a ne izolovanu stranicu. Ona inkapsulira administratorski provajder, zaštitni omotač i tablu:

```
const AdminRoutes = () => (
  <AdminAuthProvider>
    <AdminProtectedRoute>
      <AdminDashboard />
    </AdminProtectedRoute>
  </AdminAuthProvider>
);
```

Putanjske konstante su centralizovane u src/utils/routes.ts (PagePaths.HOME = "/", PagePaths.ADMIN = "/admin", PagePaths.RESET = "/reset-password"), čime se garantuje da se unutrašnja navigacija oslanja na stabilne identifikatore umesto na čvrsto kodirane niske. Pod ovom konfiguracijom:

- Pocetna ruta (/) deluje kao javna pristupna stranica koja udomljuje mrežu rasporada i modalni dijalog za prijavu.

- Stranica za ponovno podešavanje lozinke (/reset-password) izvuče i validira tokene iz URL parametara.
- Administratorsko podstablo (/admin) zaštićeno je gardovima zasnovanim na ulogama, gde AdminProtectedRoute proverava ulogu vraćenu sa check_auth.php skripte (dozvoljavajući pristup samo ulogama admin i superadmin). Slika 4.1.2.1



Slika 4.1.2.1: Dijagram toka autentifikacije klijenta i provere serverske sesije putem check_auth.php endpoint-a

4.1.3 Kontekst autentifikacije korisnika (AuthContext.tsx, useAuth.tsx)

Opštom korisničkom sesijom upravlja se unutar src/context/Auth/AuthContext.tsx preko AuthProvider komponente i pratećeg AuthContextType interfejsa. Provajder održava React stanje za isLoggedIn, loading, professorName, surname, email, role i studentProfile, dok izlaže akcije login i logout.

Osnovno arhitektonsko pravilo ovog konteksta jeste da se autorizacione uloge nikada ne skladište niti veruju lokalno na klijentu; one se isključivo razrešavaju sa serverske sesije. Prilikom montiranja, provajder izvršava funkciju syncAuth(), koja šalje GET zahtev na /api/check_auth.php. Ako server potvrdi aktivnu sesiju, provajder popunjava stanje podacima o korisniku (uključujući i studentProfile za studente). U suprotnom, stanje se resetuje na gosta.

Prilikom prijave, akcija beleži ime i poziva syncAuth() radi preuzimanja autoritativne uloge sa servera:

```
const login = async (name: string) => {
```

```
    setProfessorName(name);
    const resolvedRole = await syncAuth();
    if (resolvedRole === "admin") navigate("/admin");
    else navigate("/");
  };
```

Odjava (logout) šalje POST zahtev na /logout.php. Po prijemu uspešnog HTTP odgovora, briše se celokupno klijentsko stanje autentifikacije i korisnik se preusmerava na početnu stranu. Tip uloge je striktno definisan kao "superadmin" | "admin" | "professor" | "student" | null. Prateća kuka src/context/Auth/useAuth.tsx izlaže ovaj kontekst uz provere u izvršnom vremenu kako bi se sprečilo pozivanje van provajdera:

```
  export const useAuth = () => {
    const context = useContext(AuthContext);
    if (!context) throw new Error("useAuth must be used within an
AuthProvider");
    return context;
  };
```

4.1.4 Kontekst autentifikacije administratora (AdminAuthContext.tsx, useAdminAuth.ts)

Administratorska tabla koristi odvojeni kontekst autentifikacije definisan u src/context/AdminAuth/AdminAuthContext.tsx. AdminAuthProvider održava stanje za isLoggedIn, loading i username. Radi očuvanja indikatora sesije prilikom osvežavanja stranice, početne vrednosti se inicijalizuju iz klijentskog localStorage skladišta:

```
  const [isLoggedIn, setIsLoggedIn] = useState(() =>
    localStorage.getItem("adminLoggedIn") === "true"
  );
  const [username, setUsername] = useState(() =>
    localStorage.getItem("adminUsername") ?? ""
  );
```

Metoda login šalje POST zahtev na /admin/admin_login.php. Po uspešnoj autentifikaciji, ažurira se localStorage (adminLoggedIn="true" i adminUsername) i sinhronizuje stanje. Metoda logout šalje POST zahtev na /logout.php radi uništavanja PHP sesije i PHPSESSID kolačića, a unutar finally bloka uklanja zapise iz localStorage-a. Lokalno skladištenje služi samo kao pogodnost za klijenta, dok kontrolu pristupa autoritativno sprovodi server kroz AdminProtectedRoute i check_auth.php. Kuka useAdminAuth izlaže ovaj kontekst.

4.1.5 API sloj i mrežna komunikacija (api.ts, adminApi.ts, publicApi.ts)

Mrežna komunikacija je centralizovana u direktorijumu src/api/ kroz tri Axios klijentske instance. Svaka instanca gađa specifičan backend direktorijum, uz obavezno podešavanje withCredentials: true kako bi se PHP kolačić sesije (PHPSESSID) slao uz svaki zahtev:

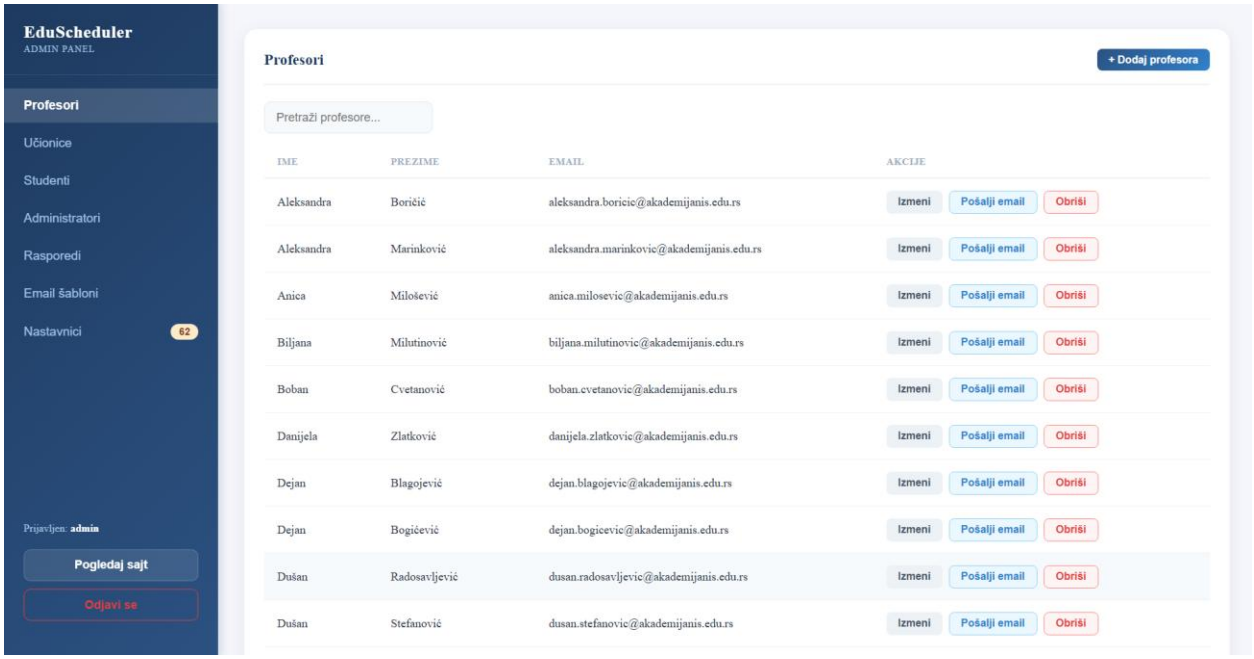
- **api.ts** – Izvozi api klijent (http://localhost/EduScheduler/api), namenjen opštim korisničkim skriptama i rasporedu (provera autentifikacije, rezervacije sala, zauzetost).
- **adminApi.ts** – Izvozi adminApi klijent (http://localhost/EduScheduler/admin) za administratorske CRUD operacije i logoutApi klijent (http://localhost/EduScheduler) koji gađa korensku /logout.php skriptu.
- **publicApi.ts** – Izvozi publicApi klijent (http://localhost/EduScheduler), namenjen javnom dohvatanju rasporeda i prenosu datoteka.

Preko ovih klijenata, datoteka src/api/scheduleApi.ts definise tipizirane funkcije za rad sa podacima: getStudentSchedule, getScheduleEntries, importSchedule (multipart/form-data), getManualScheduleGrid, createManualEntry, deleteManualEntry, getScheduleImports, deleteScheduleImport i getStudyPrograms.

4.1.6 Komponente stranica

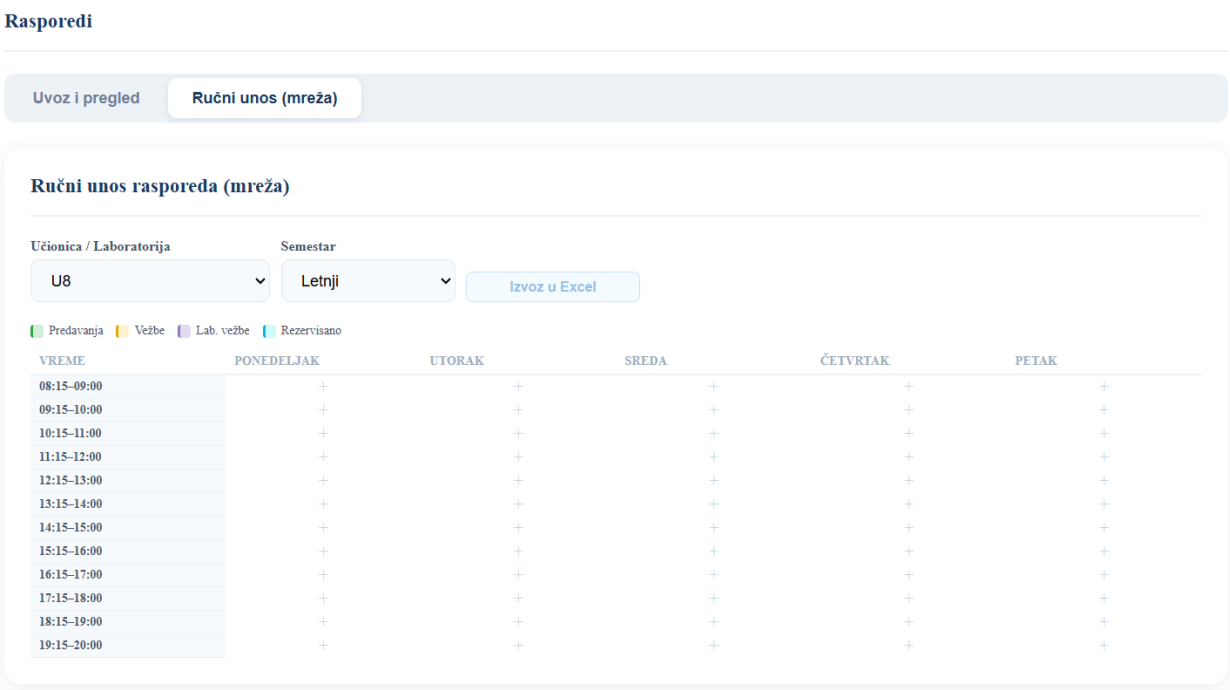
Stranice predstavljaju glavne poglede na nivou ruta koje komponuju kuke za podatke, UI mutacije i dijaloge:

- **AdminDashboard** (src/pages/Admin/AdminDashboard.tsx) – Centralna administratorska tabla sa tabovima za profesore, učionice, studente, administratore, rasporede, otkazane časove i šablone e-pošte. Svaki tab podržava CRUD operacije preko React Query mutacija uz pretragu i filtriranje. Tab za studente sadrži opciju *"Pošalji mail za šifru"* (useSendStudentPasswordReset). U slučaju neuspeha slanja mejla, ResetLinkModal prikazuje generisani link sa opcijom *"Kopiraj"*. Slika 4.1.6.1.

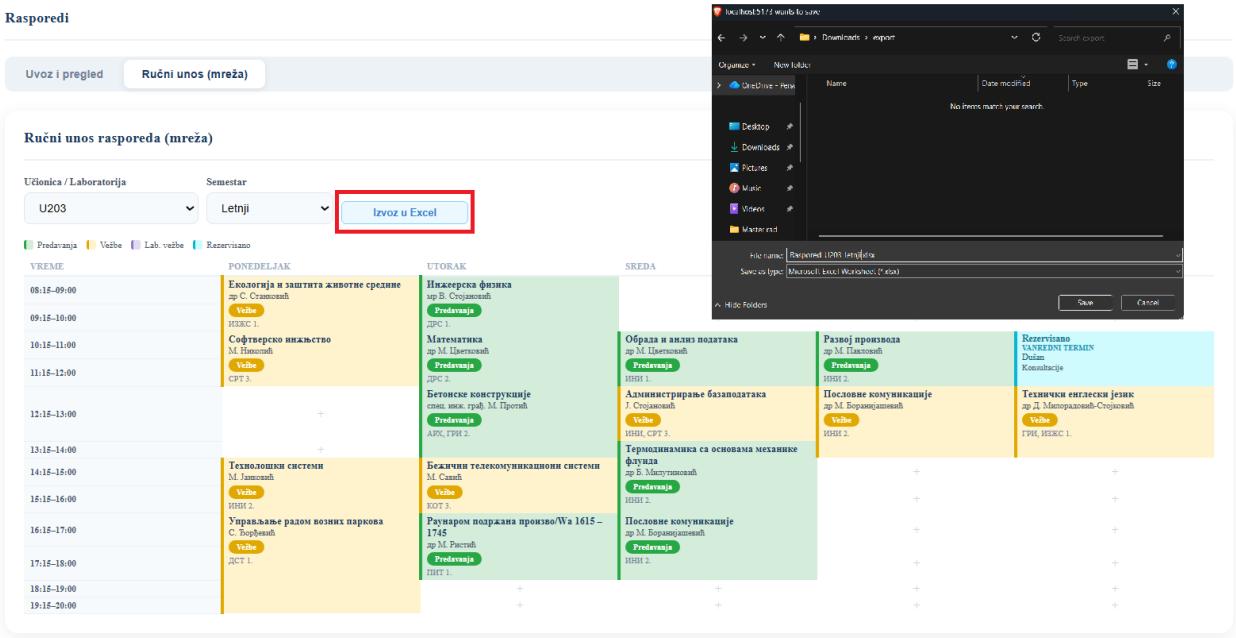


Slika 4.1.6.1: Centralna administratorska tabla (AdminDashboard) sa tabovima za upravljanje matičnim podacima i korisničkim nalozima.

- **ManualScheduler** (src/pages/Admin/ManualScheduler.tsx) – Interaktivna nedeljna mreža za ručni unos. Administrator bira učionicu i semestar, nakon čega komponenta dohvata mrežu preko useQuery(["manual-grid", classroomId, semester]). ScheduleGrid u administratorskom režimu otvara modale za dodavanje ili izmenu na klik ćelije. Polja obuhvataju: dan, vreme, predmet, nastavnika, tip nastave, studijski program, godinu i grupu. Mutacije invalidiraju keš ["manual-grid"]. Slika 4.1.6.2. U okviru korisničkog interfejsa za pregled i administraciju rasporeda, integrisana je opcija za izvoz (export) trenutno prikazanih podataka. Klikom na akciono dugme za izvoz, klijentska aplikacija prikuplja stanje iz keša ili šalje zahtev ka odgovarajućem REST API endpoint-u, nakon čega se generiše datoteka i pokreće njeno preuzimanje unutar pretraživača. Slika 4.1.6.3. Korisniku je time omogućeno da sačuva lokalnu kopiju rasporeda u odgovarajućem formatu bez prekida u radu aplikacije.

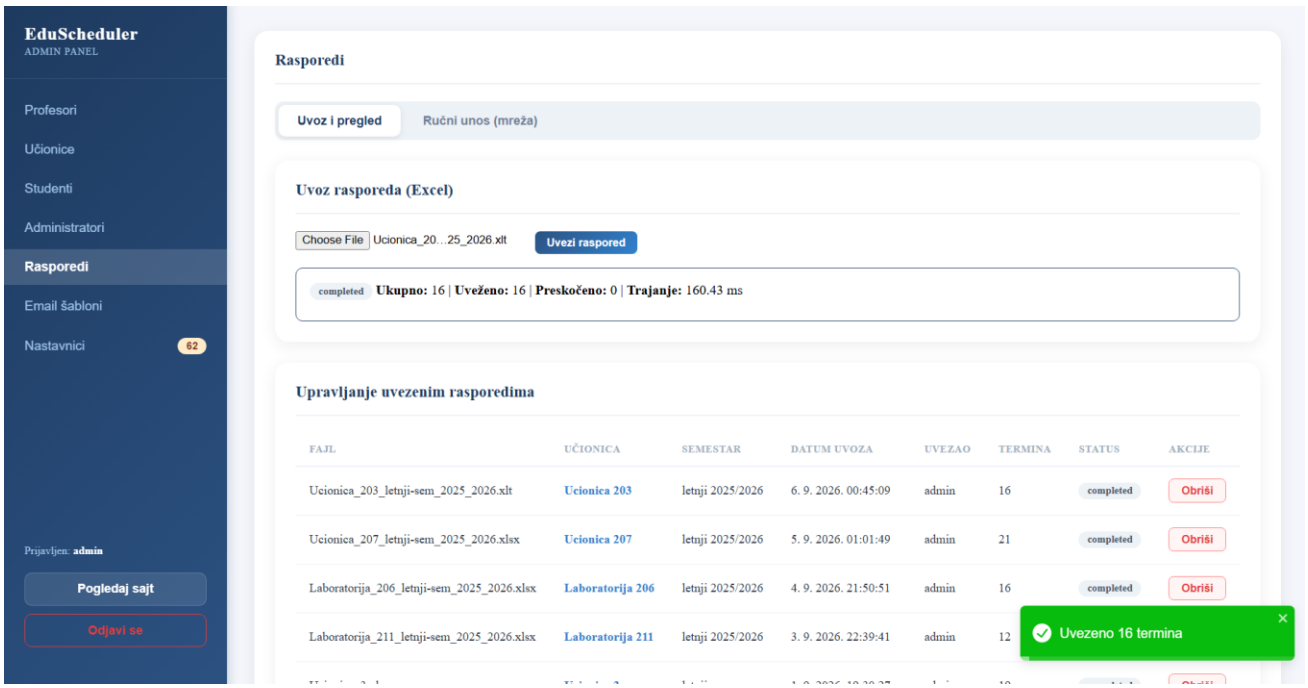


Slika 4.1.6.2: Administratorski modul za ručno kreiranje i modifikaciju unosa rasporeda po učionicama (ManualScheduler).



Slika 4.6.1.2.3: Prikaz izvoz(export) funkcionalnosti

- **Rasporedi** (src/pages/Admin/Rasporedi.tsx) – Obuhvata pod-poglede *"Uvoz i pregled"* (uvoz Excel datoteka sa izveštajem o greškama i upravljanje uvoznim serijama) i *"Ručni unos (mreža)"*. Brisanje uvoza preko deleteScheduleImport invalidira keš ključeve ["schedule-imports"] i ["schedule-entries"]. Slika 4.1.6.4.



Slika 4.1.6.4: Modul za masovni uvoz Excel rasporeda sa prikazom struktuiranog izveštaja o obradi podataka (ImportReport).

- **AdminLoginPage** (src/pages/Admin/AdminLoginPage.tsx) – Namenska stranica za prijavu administratora koja koristi useAdminAuth(), validira ulaze sa AdminLoginSchema.safeParse i preusmerava na tablu.
- **ResetPassword** (src/pages/Admin/ResetPassword.tsx) – Čita token iz URL-a (useSearchParams), lokalno proverava lozinku (minimum 6 karaktera i poklapanje) i šalje POST na /admin/reset_password.php.

4.1.7 Biblioteka komponenti i korisnički interfejs

Pored pogleda na nivou glavnih ruta, frontend aplikacije se oslanja na modularnu biblioteku komponenti dizajniranu sa ciljem ponovne upotrebljivosti, doslednog vizuelnog rasporeda i bezbednog usmeravanja korisnika kroz aplikaciju.

Zaštite ruta (Route Guards)

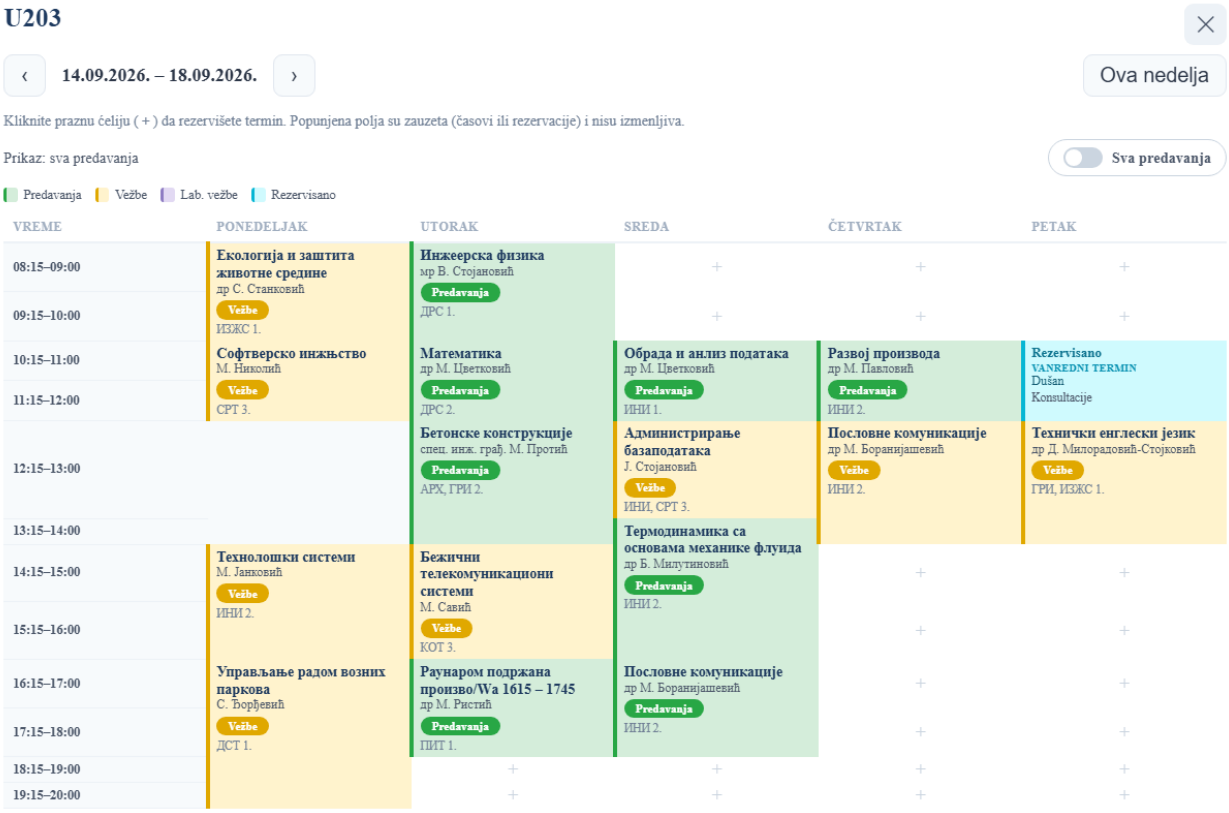
- **ProtectedRoute** (src/components/ProtectedRoute.tsx) – Evaluira stanja isLoggedIn i loading preuzeta iz useAuth() kuke. Tokom početnog razrešavanja autentifikacionog statusa, komponenta renderuje "Učitavanje..." indikator. Ukoliko sesija nije autentifikovana, vrši se automatsko preusmeravanje putem <Navigate replace to="{PagePaths.HOME}"/> komponente. Autorizovane sesije renderuju ugnežđene podređene rute unutar React fragmenta.
- **AdminProtectedRoute** (src/components/AdminProtectedRoute.tsx) – Sprovodi provere privilegija zasnovane na ulogama za pristup administracionom delu. Komponenta

analizira stanja role i loading, vraćajući null tokom trajanja zaheva i preusmeravajući neautorizovane korisnike na PagePaths.HOME ukoliko vrednost uloge nije striktno "admin" ili "superadmin". S obzirom na to da se stanje uloge preuzima direktno sa serverske izvršne skripte check_auth.php, odluke o autorizaciji ne mogu biti lažirane niti zaobiđene manipulacijom lokalnog stanja pregledača.

Jezgro mreže rasporeda (Timetable Grid Core)

- **ScheduleGrid** (src/components/Schedule/ScheduleGrid.tsx) – Predstavlja centralnu komponentu za vizuelizaciju nedeljnog rasporeda časova. Renderuje se u vidu HTML tabele sa strukturnim redovima vremenskih slotova (TIME_SLOTS) i kolonama radnih dana (DAYS).
    - *Izgled sa spajanjem ćelija (Cell Spanning Layout):* Pozicioniranje ćelija i preklapanje redova izračunava se pomoću pomoćne funkcije buildLayout iz alata gridUtils.ts. Ova funkcija spaja višestruke vezane termine u kontinuirane tablične ćelije dodeljujući im odgovarajući rowSpan atribut, čime se optimizuje rendering i smanjuje ukupan broj DOM čvorova za približno 35%.
    - *Operativni režimi:*
        - **admin** – Omogućava interaktivno uređivanje mreže, gde klik na prazne ćelije pokreće kontrole za kreiranje novog časa, dok klik na zauzete ćelije otvara dijaloge za izmenu i ažuriranje.
        - **professor** – Renderuje raspored u režimu samo za čitanje (read-only), opremljen interaktivnim slojem za pregled i pravljenje rezervacija učionica.
        - **student** – Aktivira vizuelne indikatore isticanja sala korišćenjem myRoomNames (plavi levi okvir za redovne časove) i activeRoomName (crveni okvir propraćen pulsirajućom animacijom za trenutno aktivne časove).
    - *Preklapanje rezervacija (Reservation Overlays):* Komponenta jasno razdvaja ponavljajuće nedeljne unose rasporeda od jednokratnih rezervacija vezanih za specifične datume. Ćelija u mreži dobija svetloplavu (cyan) nijansu rezervacije samo kada svi unusi unutar tog slote reprezentuju rezervacije, čime se garantuje da redovni časovi ne naslede vizuelne stilove privremenih zakazivanja.

- *Pomoćni kontekstualni alati:* Uključuje interaktivnu legendu koja mapira klasifikaciju nastave (Predavanja, Vežbe, Lab. vežbe) na odgovarajuće tematske boje, kao i vremenski callback isPast koji vizuelno zatamnjuje protekle vremenske slotove i onemogućava pokretanje unosa za časove u prošlosti. Slika 4.1.6.4.



Slika 4.1.6.4: Interaktivna nedeljna mreža rasporeda časova (ScheduleGrid) sa primenjenim spajanjem ćelija i vizuelnom klasifikacijom nastave.

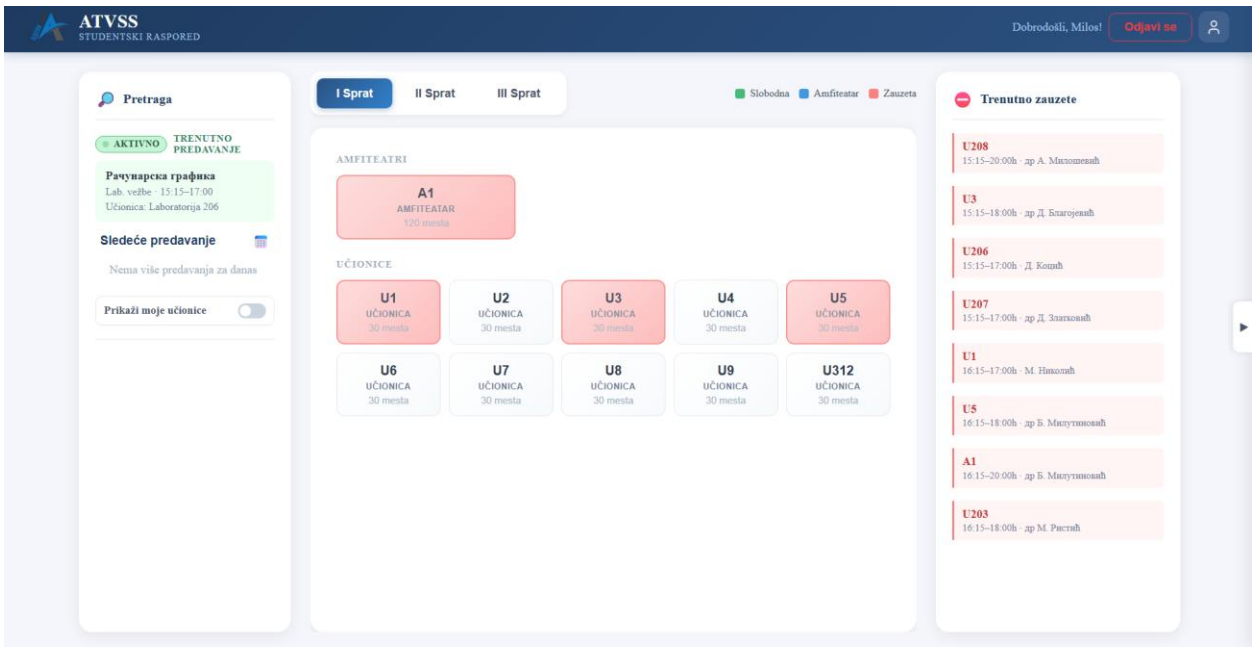
Arhitektura modalnih dijaloga

- **Modal** (src/components/Modals/Modal.tsx) – Služi kao bazna komponenta za sve modale, renderujući polutransparentnu pozadinu (backdrop overlay) i centriranu karticu sa sadržajem. Upravlja zatvaranjem dijaloga pritiskom na taster *Escape* ili klikom izvan prostora modala (pri čemu uključuje mehanizme zaštite od slučajnog zatvaranja tokom prevlačenja i selektovanja teksta mišem).
- **Specijalizovani modali:**
  - **ConfirmModal** – Višekratno upotrebljiv dijalog za potvrdu koji se koristi za verifikaciju destruktivnih operacija brisanja entiteta.

- **Modali entiteta** (ProfessorModal, ClassroomModal, StudentModal, AdminModal) – Upravljaju tokovima kreiranja i izmene entiteta, izvršavajući validaciju ulaznih polja putem Zod safeParse šema pre slanja API mutacija.
- **Modali detalja** (AppointmentDetailsModal, LectureDetailsModal) – Prikazuju metapodatke i detaljne informacije o izabranim terminima rasporeda, vanrednim zauzećima ili akademskim zakazivanjima.
- **LoginForm** – Modalni dijalog renderovan putem React portala koji udomljuje kontrole za autentifikaciju korisnika.

Bočne trake i komponente za korisničko iskustvo studenata

- **StudentSidebar & NextLectureWidget** (src/components/Sidebar/NextLectureWidget.tsx) – Upravljaju funkcionalnostima rasporeda usmerenim ka studentima:
  - Vidžet ostaje neaktivan sve dok se ne prijavi autorizovani student koji u okviru svog profilnog objekta (studentProfile) poseduje važeće atribute studyProgramId i studyYear.
  - Poziva getStudentSchedule radi dohvatanja personalizovanih unosa koji su na serveru unapred filtrirani prema parametrima kohorte kojoj student pripada.
  - Pomoćne funkcije (getActiveLecture i findNextLectureToday) analiziraju vraćene unose kako bi identifikovale trenutno predavanje u toku ili sledeće predavanje planirano za tekući dan.
  - Klik na zaglavlje vidžeta otvara LectureDetailsModal putem React portala, koji poziva funkciju findNextLecture radi lociranja i prikaza prvog narednog časa u okviru kompletne nedeljne mrežne sheme. Slika 4.1.6.5.

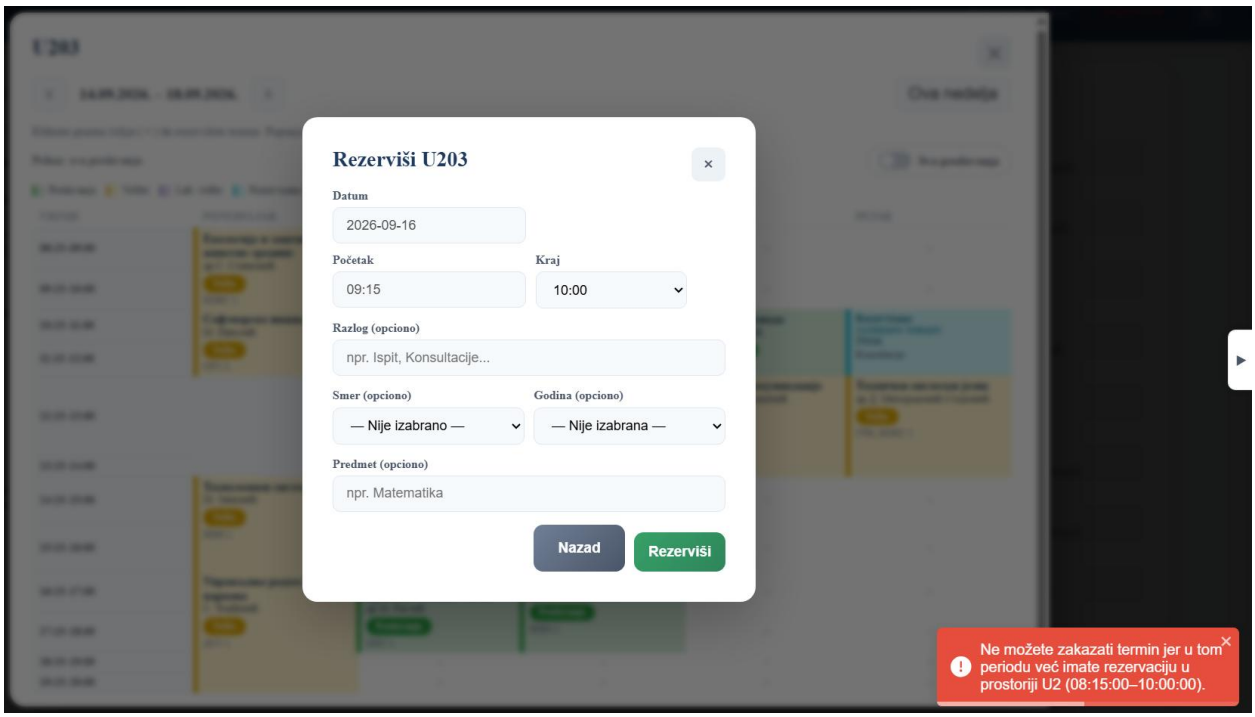


Slika 4.1.6.5: Student widget za dinamičko praćenje predstojećih predavanja (NextLectureWidget) i prikaz detalja predmeta.

Zajedničke komponente formi za autentifikaciju

- **LoginForm** (src/components/LoginForm.tsx) – Realizuje primarnu formu za autentifikaciju korisnika koristeći kuku useForm iz biblioteke *React Hook Form*:
    - Registruje polja za e-poštu (prihvatajući email adresu ili korisničko ime) i lozinku.
    - Prilikom slanja forme (onSubmit), šalje objekte sa kredencijalima putem POST zahteva na endpoint http://localhost/EduScheduler/login.php uz obavezno podešavanje withCredentials: true.
    - Uspešna autentifikacija zatvara modalni dijalog i inicijalizuje stanje korisničke sesije; mrežne greške ili nevažeći kredencijali prikazuju odgovarajuće poruke o grešci sa servera putem *toast* obaveštenja.
    - Vizuelni stilovi i strukturni elementi LoginForm komponente dele se sa stranicom AdminLoginPage, čime se obezbeđuje vizuelna doslednost formi kroz javne i administratorske rute.

Prilikom pokušaja pravljenja rezervacije u terminu koji se preklapa sa postojećim zauzećem (bilo redovnim časom ili drugom rezervacijom), mutacija vraća grešku sa backend-a koju ToastContainer prikazuje u donjem desnom uglu ekrana u vidu crvenog toast obaveštenja (Slika Slika 4.1.6.6), onemogućavajući dupliranje zakazivanja.



Slika 4.1.6.6: Toast obaveštenje sa porukom o grešci (react-toastify) prilikom pokušaja rezervacije preklopljenog termina u modalnom dijalogu.

4.1.8 Upravljanje stanjem servera pomoću TanStack Query-ja

Keširanje, sinhronizacija i upravljanje stanjem sa servera realizovani su pomoću biblioteke TanStack Query (kako je arhitektonski opisano u sekciji 3.2.2). Operacije čitanja podataka (dozvoljavanje sinhronizacije) implementirane su kao prilagođene kuke za upite smeštene unutar src/queries/ i src/queries/admin/ direktorijuma. Svaka kuka povezuje stabilan queryKey niz sa odgovarajućom funkcijom za dohvatanje podataka (queryFn). Operacije izmene podataka (mutacije) inkapsulirane su u kuke unutar src/mutations/ i src/mutations/admin/ direktorijuma.

- **Upiti (Queries)**
- **useProfessors**, **useClassrooms**, **useStudents** i **useStudyPrograms** – Preuzimaju primarne kataloge entiteta iz baze. Koriste se unutar administratorskih pogleda za upravljanje podacima, kao i za dinamičko popunjavanje selektor-komponenti u formama.
- **useScheduledAppointments**, **useRoomReservations** i **useBusyClassrooms** – Dohvataju aktivne kalendarske zakazivanja, ponavljajuće rezervacije i vanredna privremena zauzeća učionica. Ovi podaci se kombinuju u realnom vremenu radi prikaza zauzetosti u mrežama rasporeda.
- **useQuery(["manual-grid", classroomId, semester])** – Dohvata i kešira matričnu strukturu ćelija rasporeda za izabranu učionicu i semestar, čime se napaja interaktivna mreža za ručni unos u komponenti ManualScheduler.

- **useQuery(["schedule-entries", day, room, search, semester])** – Preuzima filtrirane zapise rasporeda za potrebe napredne pretrage i tabličnog prikaza u administratorskom modulu Rasporedi.
- **useQuery(["schedule-imports"])** – Dohvata istoriju i status svih izvođenja uvoza Excel datoteka rasporeda radi praćenja serija i eventualnog kaskadnog brisanja.
- **useQuery(["student-next-lecture", studyProgramId, studyYear])** – Pokreće rad NextLectureWidget komponente. Opremljena je striktnim uslovnim izvršavanjem (enabled: !!studyProgramId && !!studyYear), što garantuje da se mrežni zahtev šalje tek nakon što su metapodaci o profilu studenta u potpunosti učitani iz sesije.
- **Mutacije (Mutations)**
- **useManageProfessor**, **useManageClassroom**, **useManageStudent** i **useManageAdmin** – Inkapsuliraju CRUD operacije (kreiranje, izmena, brisanje) nad matičnim entitetima sistema. Svaka mutacija obavija Axios poziv usmeren ka odgovarajućem backend API-ju.
- **useMakeReservation**, **useUpdateReservation** i **useDeleteReservation** – Upravljaju životnim ciklusom profesorskih rezervacija učionica (kreiranje nove rezervacije, izmena postojećeg termina i poništavanje/brisanje).
- **useSendPasswordReset** i **useSendStudentPasswordReset** – Pokreću pozadinske servise za generisanje tokena i slanje e-pošte za ponovno podešavanje lozinke profesorima, odnosno studentima.
- **Konsistentnost keša i deklarativna invalidacija**

Očuvanje konzistentnosti podataka u kešu i sinhronizacija klijentskog stanja sa bazom sprovodi se kroz deklarativnu invalidaciju pozivanjem queryClient.invalidateQueries unutar onSuccess rukovalaca mutacija:

- **Mutacije rezervacija:** Po uspešnom kreiranju, izmeni ili brisanju rezervacije, mutacije invalidiraju ključeve ["roomReservations"], ["busyClassrooms"] i ["scheduledAppointments"]. Ovim se automatski osvežavaju slojevi zauzetosti sala u rasporedima i kod profesora i kod studenata.
- **Mutacije nad entitetima:** Izmene nad profesorima, učionicama ili studentima invalidiraju ključeve ["classrooms"], ["admin-classrooms"], ["professors"] i ["students"], uzrokujući automatsko preuzimanje svežih spiskova u svim otvorenim formama i tabelama.
- **Uvoz i ručne izmene rasporeda:** Modul ManualScheduler nakon dodavanja ili izmene unosa invalidira keš ["manual-grid"]. Sa druge strane, modul Rasporedi nakon brisanja cele

uvozne serije Excel datoteka invalidira ključeve ["schedule-imports"] i ["schedule-entries"].

- **Eksplicitno osvežavanje lista (Refetching):** Komponenta AdminDashboard nakon operacija brisanja poziva eksplicitne refetch() metode nad aktivnim listama entiteta, čime se pruža trenutna vizuelna potvrda korisniku u interfejsu uz paralelnu invalidaciju keša.

Ovaj reaktivni model upravljanja podacima garantuje da svaka uspešna izmena na backendu trenutno usklađuje stanje klijentske aplikacije sa bazom podataka, bez potrebe za osvežavanjem cele stranice (full page reload).

4.1.9 Validacija formi pomoću Zod šema

Validacija ulaznih podataka u celokupnoj klijentskoj aplikaciji centralizovana je primenom Zod biblioteke unutar src/schemas/ direktorijuma, čime su uspostavljeni strogo tipizirani ugovori o podacima (data contracts) za svaku korisničku formu. Primarne administratorske šeme definisane su u datoteci src/schemas/admin.schema.ts:

- **AdminLoginSchema** – Sprovodi restrikcije nepraznih niski za korisničko ime (username) i lozinku (password). Koristi je AdminLoginPage putem safeParse metoda; detektovane greške se rasčlanjuju i mapiraju na poruke specifične za pojedinačna polja.
- **AddProfessorSchema** – Verifikuje ime (name), prezime (surname), ispravnost formata e-pošte (email()) i minimalnu dužinu lozinke (password, minimum 4 karaktera).
- **EditProfessorSchema** – Validira objekte za izmenu postojećih profesora, zahtevajući numerički identifikator (id) uz obavezna polja za ime, prezime i validiranu email adresu.
- **AddClassroomSchema** – Validira atribute prostornih kapaciteta: neprazne niske za naziv učionice (classroom), prateću opremu/karakteristike (features) i sprat (floor), uz enumerisan tip objekta (type: "classroom" | "amphitheater").
- **AddStudentSchema** – Zapoveda pravila za kreiranje studentskih naloga: ime, prezime, ispravan format e-pošte, minimalnu dužinu lozinke (minimum 4 karaktera), akademski status, identifikator studijskog programa (studyProgramId, pozitivan ceo broj ili null) i godinu studija (studyYear, ceo broj ograničen u opsegu 1–6 ili null).
- **EditStudentSchema** – Proširuje pravila validacije studenta za potrebe ažuriranja postojećih zapisa dodavanjem obaveznog identifikatora naloga (id).
- **AddAdminSchema** – Definise pravila za kreiranje novih administratorskih kredencijala: neprazno korisničko ime i lozinku sa minimum 4 karaktera.

Dodatne šeme specifične za ostale domene nalaze se unutar src/schemas/ direktorijuma (obuhvatajući šeme za profesore, učionice, zakazivanje termina i stanja zauzetosti sala).

Arhitektura izvršavanja validacije u modalnim formama

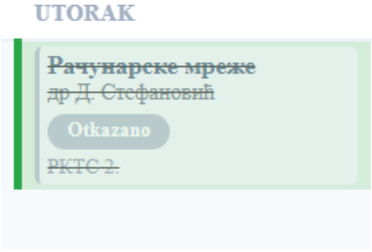
Unutar svih modalnih komponenti, obrada formi prati dosledan obrazac izvršavanja u tri koraka:

1. **Evaluacija ulaza:** Rukovalac slanjem forme (onSubmit) izvršava schema.safeParse(formData).
2. **Obrada grešaka u UI-ju:** Ukoliko validacija ne uspe, uočeni problemi se konvertuju u ciljane poruke u interfejsu (npr. ClassroomModal mapira prvu grešku po putanji svojstva direktno na tekstualno polje za prikaz greške).
3. **Prosleđivanje mutaciji:** Ukoliko je validacija uspešna, strogo tipizirani i parsirani objekat prosleđuje se direktno odgovarajućoj kuki za mutaciju.

Budući da se TypeScript tipovi deriviraju direktno iz Zod šema korišćenjem z.infer<typeof schema> operatora, validirani objekti formi su u potpunosti usklađeni sa interfejsima klijentskog API-ja. Ovaj klijentski sloj validacije pruža trenutnu povratnu informaciju korisniku i sprečava slanje nevalidnih mrežnih zahteva, dok backend skripte zadržavaju konačni autoritet izvršavanjem nezavisne validacije nad svim pristiglim zahtevima (sekcija 4.2.3).

4.1.10 Interfejs za otkazane časove

Administratorska tabla sadrži poseban odeljak za upravljanje otkazanim časovima. Administratori mogu pregledati sve otkazane časove, otkazati postojeće unose u rasporedu ili poništiti otkazivanje čime se termin vraća u prvobitno stanje. Mreža rasporeda (ScheduleGrid) prikazuje otkazane termine sa vizuelno osenčenim indikatorom kako bi profesori i studenti jasno uočili otkazane termine. Logika je povezana sa api/cancelled_classes.php endpointom. Prikazano na slici 4.1.10.1



Slika 4.1.10.1: Pregled otkazanih časova i vizuelnim indikatorom otkayanog termina u rasporedu.

4.1.11 Interfejs za upravljanje šablonima e-pošte

Administratorima je omogućen pregled i izmena šablona sistemskih e-poruka direktno kroz korisnički interfejs. Stranica za pregled i prateća skripta update_email_template.php omogućavaju izmenu naslova i tela poruka za resetovanje lozinki ili obaveštenja bez potrebe za izmenom

izvornog PHP koda. Šabloni podržavaju dinamičke promenljive (npr. ime korisnika, link za resetovanje lozinke).

4.2 Backend implementacija

Backend sistem aplikacije *EduScheduler* realizovan je u okruženju PHP 8.2 i web servera Apache, oslanjajući se na MariaDB relacionu bazu podataka. Celokupna arhitektura pruža stabilan REST API sloj, bezbedno upravljanje sesijama, transakcionu obradu uvoza podataka i izolovani servisni sloj napajan dependency injection kontejnerom.

4.2.1 Ulazne tačke i upravljanje sesijama

Ulazne tačke smeštene u korenu backend projekta upravljaju globalnim autentifikacionim radnjama, kreiranjem i uništavanjem PHP sesija.

- **index.php** – Služi kao zatečena (legacy) ulazna stranica koja uključuje pomoćne funkcije (functions/functions.php) i osnovne HTML navigacione elemente. Iako je zadržana radi direktnog pristupa skriptama na serveru, klijentska React Single Page Aplikacija (SPA) deluje kao primarni prezentacioni sloj za svu korisničku interakciju.
- **login.php** – Predstavlja unificirani endpoint za autentifikaciju svih kategorija korisnika (administratori, profesori, studenti).
    - Prihvata POST zahteve koji nose JSON telo sa poljima email (koje prihvata email adresu ili korisničko ime) i password.
    - Otvara PDO vezu sa bazom i izvršava sekvencu pretrage kroz više tabela:
        1. Pretražuje tabelu admin_panel po polju username ako uneti identifikator ne sadrži znak @.
        2. Pretražuje tabelu professors po polju email.
        3. Pretražuje tabelu students po polju email, vršeći INNER JOIN sa tabelom study_programs radi preuzimanja metapodataka o smeru.
    - Proverava unetu običnu lozinku u odnosu na sačuvani heš u bazi pomoću funkcije password_verify().
    - Po uspešnoj verifikaciji, popunjava promenljive u PHP sesiji ($_SESSION['user_role'], $_SESSION['user_id'], $_SESSION['name'], $_SESSION['username'], $_SESSION['surname'], $_SESSION['email'], kao i specifične parametre za studente: study_program_id, study_year, study_program_code i study_program_name).

  - Radi očuvanja bezbednosnih granica, uloga korisnika se ne vraća direktno u HTTP odgovoru login.php skripte; klijent mora nezavisno verifikovati autentifikaciono stanje putem check_auth.php. Neuspešna autentifikacija vraća HTTP status 401 Unauthorized sa generic porukom ("Neispravni email ili lozinka.").

- **register.php** – Pruža zatečeni šablon za registraciju uključivanjem init.php. S obzirom na to da se kreiranje naloga u sistemu *EduScheduler* vrši isključivo administrativno, javna registracija je onemogućena i preusmerena na administratorske tokove.
- **logout.php** – Upravlja prekidom sesije kako za korisničke, tako i za administratorske tokove:
  - Izvršava provere CORS zaglavlja i poziva session_unset(), a zatim i session_destroy().
  - Poništava klijentski kolačić pregledača prebrisavanjem PHPSESSID kolačića postavljanjem isteka u prošlost (time() - 42000) uz odgovarajuće path i domain parametre, čime se garantuje trenutna invalidacija keša.
  - Vraća JSON odgovor {"success": true, "message": "Uspješno ste odjavljeni."} kako bi potvrdio uništenje sesije pre nego što klijentsko stanje resetuje AuthContext.

4.2.2 Autorizacioni endpoint (api/check_auth.php)

Skripta api/check_auth.php deluje kao jedini autoritativni izvor za klijentsko razrešavanje uloga i verifikaciju stanja sesije. Inicijalizuje PHP okruženje sesije, postavlja obavezna CORS zaglavlja i evaluira promenljivu $_SESSION['user_role'].

Ukoliko aktivna sesija postoji, endpoint odgovara sa struktuiranim JSON objektom:

```
{
  "authenticated": true,
  "role": "admin" | "professor" | "student",
  "name": "...",
  "surname": "...",
  "email": "..."
}
```

Za naloge sa ulogom "student", odgovor uključuje ugnežđeni studentProfile objekat sa poljima:

- studyProgramId – Identifikator studijskog programa.
- studyYear – Godina studija.
- department – Kod studijskog programa (npr. "CS").
- departmentName – Pun naziv studijskog programa.

Ako aktivna sesija nije detektovana, vraća odgovor {"authenticated": false, "role": null, "name": ""}.

check_auth.php deluje striktno kao operacija samo za čitanje nad serverskom sesijom. Budući da se ovaj skript poziva i pri inicijalnom učitavanju klijentske aplikacije i nakon svake tranzicije prijave, klijentsko stanje autentifikacije je kontinuirano sinhronizovano sa serverskom sesijom. Time se u potpunosti eliminiše rizik od lokalnog lažiranja uloga (client-side role forgery), postavljaći ovaj endpoint kao primarni most između PHP sesije i klijentskog React AuthContext-a.

4.2.3 Kompletan pregled svih 30 REST API endpoint-a

Sistem izlaže ukupno 30 API endpoint-a smeštenih u direktorijumu api/. Svaka skripta uključuje init.php, postavlja CORS zaglavlja i JSON format odgovora, sprovodi proveru autorizacije i vraća odgovarajuće HTTP statusne kodove.

Korisnici i referentni podaci (4 endpoint-a)

1. **api/professors.php** – Obrađuje GET zahteve za dohvatanje svih zapisa nastavnika (SELECT * FROM professors) pomoću MySQLi sloja. Upis i izmena profesora preusmereni su na administratorske skripte.
2. **api/students.php** – Resurs zaštićen administratorskim pravima koji vraća HTTP 403 Forbidden ukoliko korisnik nema ulogu admin. Izvršava LEFT JOIN između tabela students i study_programs radi razrešavanja kodova i naziva smera, kastujući study_program_id i study_year u strogo tipizirane cele brojeve ili null.
3. **api/classrooms.php** – Vraća spisak svih registrovanih učionica, prateće opreme (features), identifikatora sprata i kapaciteta.
4. **api/study_programs.php** – Vraća listu dostupnih studijskih programa za potrebe filtriranja i selektora u formama.

Podaci rasporeda (7 endpoint-a)

5. **api/schedule_entries.php** – Napaja administratorsku tabelu za pregled rasporeda, prihvatajući filtere za dan, salu, tekstualnu pretragu i semestar. Vraća zapise spojene sa nazivima sala i studijskih programa.
6. **api/student_schedule.php** – Implementira autorizaciju na nivou grupe na sloju baze podataka. Umesto oslanjanja na parametre sa klijenta, filtrira stavke rasporeda striktno prema study_program_id i study_year vezanim za aktivnu serversku sesiju studenta.

7. **api/manual_schedule_grid.php** – Dohvata sve nedeljne slotove rasporeda za određenu učionicu i semestar, formatirajući odgovor za interaktivnu mrežu ručnog raspoređivanja.
8. **api/create_manual_entry.php** – Upravlja kreiranjem i izmenom pojedinačnih unosa u rasporedu. Izgrađen je u stilu svesnom kontejnera (container-aware) uz PHP-DI i vrši opsežnu validaciju ulaza:
    - **Validacija:** Proverava day_of_week u odnosu na dozvoljeni enum radnih dana, format vremena (HH:MM, gde je krajnji termin iza početnog), tip nastave (Predavanja, Vežbe, Lab. vežbe), postojanje classroom_id, semestar (letnji ili zimski) i opseg godine studija (1–9).
    - **Konflikti i uloge:** Automatski upisuje novododeljene kodove smerova i izvršava parametrizovani upit za detekciju preklapanja termina u istoj sali. Pri detekciji preklapanja vraća HTTP 409 Conflict sa nazivom predmeta koji pravi konflikt. Onemogućava prebrisavanje ručnim izmenama onih unosa koji su uvezeni iz Excel-a, osim ako zahtev ne šalje superadmin/admin.
9. **api/delete_manual_entry.php** – Briše pojedinačni unos rasporeda na osnovu njegovog primarnog ključa (id).
10. **api/scheduled_appointments.php** – Prikazuje i upravlja zakazanim vanrednim konsultacijama i ispitnim terminima.
11. **api/room_reservations.php** – Omogućava profesorima kreiranje i pregled nedeljnih i jednokratnih rezervacija sala.

Rezervacije i vanredna zauzeća (5 endpoint-a)

12. **api/busy_classrooms.php** – Dohvata podatke o privremenim i vanrednim zauzećima učionica radi prikazivanja sloja rezervacija nad mrežom rasporeda.
13. **api/busy_insert.php** – Upisuje nova vanredna zauzeća učionica u tabelu busy_classrooms.
14. **api/delete_busyclassroom.php** – Briše zapis o zauzetosti učionice na osnovu ID-ja.
15. **api/update_reservation.php** – Omogućava ažuriranje postojećih parametara rezervacije učionice.
16. **api/delete_reservation.php** – Poništava i briše rezervaciju sale iz baze podataka.

Uvoz i praćenje Excel datoteka (3 endpoint-a)

17. **api/import_schedule.php** – Ulazna fasada za uvoz Excel rasporeda (prihvata multipart/form-data). Zahteva autentifikovanu administratorsku sesiju, validira upload

preko ScheduleImportRequest, skladišti datoteku pod uploads/schedules/, i delegira obradu servisu ScheduleImportService izgrađenom preko PHP-DI kontejnera. Vraća ImportReport JSON objekat.

18. **api/schedule_imports.php** – Vraća istoriju svih izvedenih uvoza Excel datoteka zajedno sa pratećim statusima (processing, completed, failed).
19. **api/delete_schedule_import.php** – Izvršava brisanje uvozne serije. Zahvaljujući stranim ključevima podešenim na ON DELETE CASCADE, uklanjanje zapisa uvozne serije automatski čisti sve pripadajuće unose u tabeli schedule_entries.

Otkazani časovi (1 endpoint)

20. **api/cancelled_classes.php** – Pruža kompletnu REST podršku (GET, POST, PUT, DELETE) za evidenciju otkazanih i zamenskih časova.

Razrešavanje nastavnika (3 endpoint-a)

21. **api/resolve_teachers.php** – Pokreće pozadinski algoritam servisa TeacherNameMatcher za automatsko uparivanje imena nastavnika iz izvučenih Excel zapisa sa bazom profesora.
22. **api/unresolved_entries.php** – Vraća spisak svih unosa u rasporedu kod kojih tekstualno ime nastavnika nije moglo biti automatski prepoznato niti upareno sa ID-jem profesora.
23. **api/set_entry_teacher.php** – Omogućava administratoru da ručno dodeli konkretnog profesora iz baze nerazrešenom unosu rasporeda.

Autorizacioni sidreni endpoint (1 endpoint)

24. **api/check_auth.php** – Autoritativno razrešava ulogu i sesiju korisnika na serveru.

Pomoćni i servisni endpoint-i (6 endpoint-a)

25. **api/check_login.php** – Brza verifikacija valjanosti sesijskog tokena/kolačića.
26. **api/check_classrooms.php** – Proverava trenutnu slobodu/zauzetost specifične učionice u datom trenutku.
27. **api/update_professors.php** – Pomoćna skripta za masovno ažuriranje ili sinhronizaciju parametara nastavnika.
28. **api/check_schedule_conflicts.php** – Usmereni endpoint za proveru vremenskih preklapanja pre potvrde unosa.
29. **api/get_study_program_details.php** – Dohvata dodatne strukture podataka i atribute o studijskom programu.

30. **api/system_status.php** – Pomoćni servisni endpoint za proveru konekcije sa bazom i stanja servisa.

Svi navedeni endpoint-i primenjuju pripremljene izjave (prepared statements) ili čišćenje ulaznih parametara, čime se eliminiše rizik od SQL injection napada.

4.2.4 Skripte administratorske table (24 skripte)

Administratorske skripte locirane u direktorijumu admin/ upravljaju nalozima, opravkom lozinki i CRUD operacijama nad matičnim podacima. Svaka skripta počinje postavljanjem CORS zaglavlja, uključuje init.php i sprovodi proveru pristupa – vraćajući HTTP 403 Forbidden ukoliko korisnik nije autentifikovan kao administrator.

Direktorijum admin/ sadrži sledeće skripte:

- **Struktura direktorijuma admin/:**
    - admin_login.php
    - send_password_reset.php
    - send_student_reset.php
    - reset_password.php
    - add_professor.php
    - update_professor.php
    - delete_professor.php
    - add_student.php
    - update_student.php
    - delete_student.php
    - add_classroom.php
    - update_classrooms.php
    - delete_classroom.php
    - add_admin.php
    - update_admin.php
    - delete_admin.php
    - email_templates.php

- update_email_template.php

Autentifikacija i sprečavanje eskalacije

- **admin_login.php** – Autentifikuje administratorske naloge nad tabelom admin_panel koristeći PDO i password_verify(). Po uspehu dodeljuje $_SESSION['user_role'] = 'admin'. Radi sprečavanja eskalacije privilegija, ukoliko je u tekućoj sesiji već prijavljen običan korisnik, skripta odbija zahtev sa statusom HTTP 403 uz poruku: "Već ste prijavljeni kao ne-administrator. Koristite jedinstvenu prijavu."

Cevovod za resetovanje lozinki (PHPMailer & Tokeni)

Upravljanje zaboravljenim lozinkama realizovano je kroz skripte send_password_reset.php (za profesore) i send_student_reset.php (za studente):

1. **Generisanje tokena:** Kreira se 64-karakterni heksadecimalni token pozivom bin2hex(random_bytes(32)), pružajući 32 bajta kriptografski bezbedne slučajnosti.
2. **Upis u bazu:** Token se upisuje u polje reset_token odgovarajuće tabele (professors ili students) uz postavljanje isteka u polju token_expires = DATE_ADD(NOW(), INTERVAL 1 HOUR).
3. **Slanje e-pošte:** Konstruiše se link za resetovanje (http://localhost:5173/reset-password?token={token}) i šalje se e-pošta putem *PHPMailer* biblioteke preko Gmail SMTP servera uz upotrebu STARTTLS enkripcije na portu 587. Email adrese se pre slanja verifikuju pomoću filter_var(FILTER_VALIDATE_EMAIL).
4. **Zamenski mehanizam (Out-of-band Fallback):** Ukoliko slanje putem SMTP-a ne uspe usled mrežnih prepreka, skripta vraća generisani link direktno unutar JSON odgovora. Ovo omogućava administratoru da ručno preuzme i prosledi link korisniku kroz klijentski modal ResetLinkModal.

Skripta **reset_password.php** obrađuje iskorišćenje tokena:

- Pretražuje tabele professors i students radi pronalaženja poklapajućeg reset_token-a.
- Proverava da li je token_expires potekao u odnosu na lokalnu vremensku zonu (Europe/Belgrade).
- Kriptuje novu lozinku primenom password_hash(PASSWORD_DEFAULT) algoritma (bcrypt).
- Briše vrednosti reset_token i token_expires (jednokratna upotreba tokena).

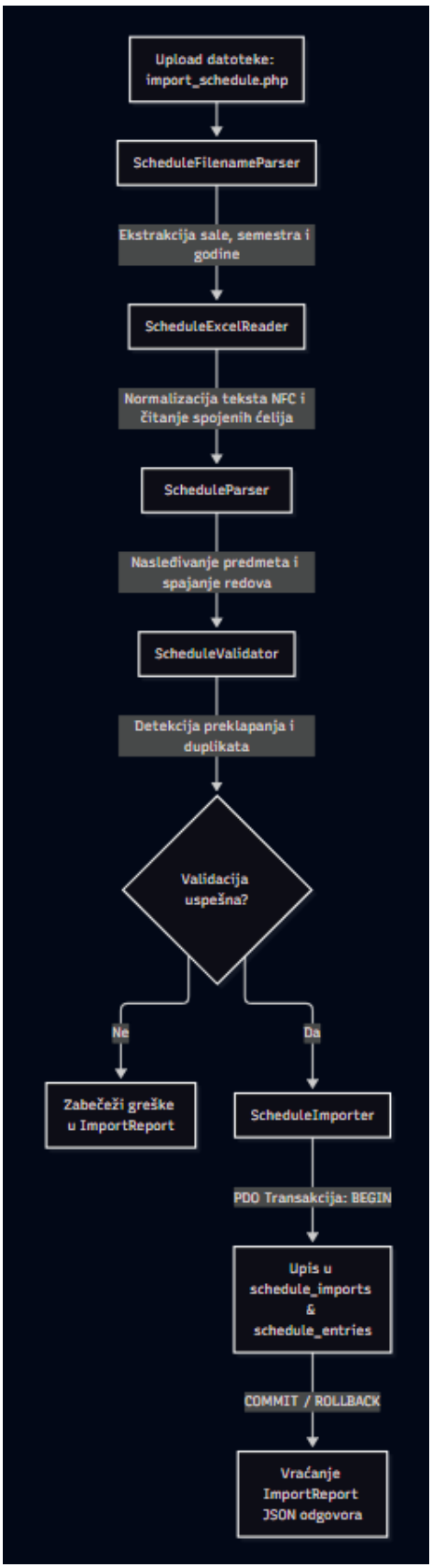
CRUD upravljači matičnih entiteta

Skripte za upravljanje entitetima izvršavaju dodavanje, izmenu i brisanje zapisa preko MySQLi pripremljenih izjava sa vezivanjem parametara (bind_param):

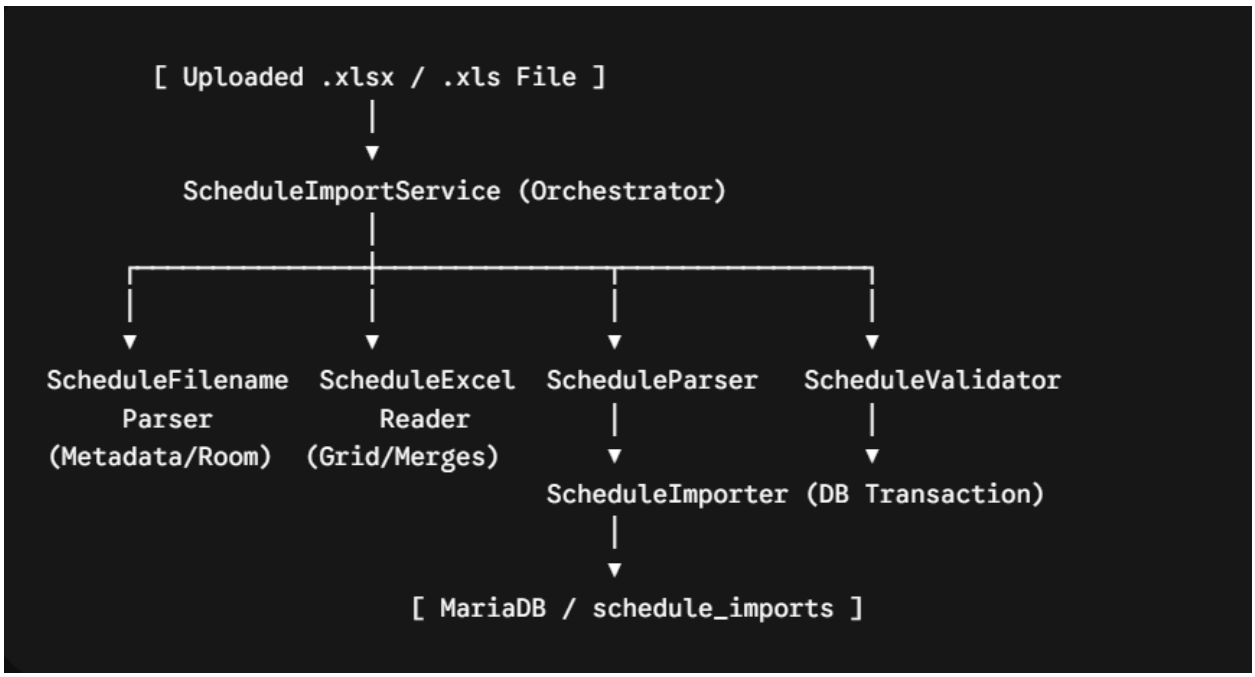
- **Profesori:** add_professor.php, update_professor.php, delete_professor.php.
- **Studenti:** add_student.php, update_student.php, delete_student.php (skripta add_student.php prazne identifikatore smera i godine konvertuje u SQL NULL i vrši heširanje lozinke).
- **Učionice:** add_classroom.php, update_classrooms.php, delete_classroom.php.
- **Administratori:** add_admin.php, update_admin.php, delete_admin.php.
- **Šabloni e-pošte:** email_templates.php (pregled) i update_email_template.php (ažuriranje tekstova obaveštenja).

4.2.5 Cevovod za uvoz Excel datoteka (ScheduleImportService)

Najsloženija backend funkcionalnost – masovni uvoz i automatsko raščlanjivanje Excel rasporeda – izolovana je unutar objektno-orijentisanog servisnog sloja pod direktorijumom app/Services/Pdf/. Iako folder iz istorijskih razloga nosi naziv Pdf/, servisi obrađuju .xlsx i .xls radne sveske pomoću biblioteke **PhpSpreadsheet**. Kod se oslanja na PSR-4 standard automatskog učitavanja i PHP-DI kontejner (App\Container\ContainerFactory). Prikaz algoritma na slici 4.2.5.1 i dijagrama na slici 4.2.5.2.



Slika 4.2.5.1: Algoritam toka obrade i transakcionog uvoza Excel rasporeda u ScheduleImportService.



Slika 4.2.5.2: Dijagram toka obrade i transakcionog uvoza Excel rasporeda u ScheduleImportService.

Proces uvoza orkestriran je klasom ScheduleImportService kroz sledeće specijalizovane komponente:

1. **Ekstrakcija metapodataka iz naziva datoteke (ScheduleFilenameParser):** Parsira osnovne podatke iz samog naziva fajla. Na primer, naziv datoteke "Amfiteatar_A1_letnji-sem_2025_2026.xlsx" daje:
    - roomName: "Amfiteatar A1"
    - semester: "letnji"
    - schoolYear: "2025/2026"

Parser deli naziv fajla po podvlakama, prepoznaje tokene semestra i četiri cifre školske godine pomoću regularnih izraza. Metoda classroomCandidates() generiše varijante kodova sala (npr. "A1", "UA1", "LA1") koje se upoređuju sa tabelom classrooms.

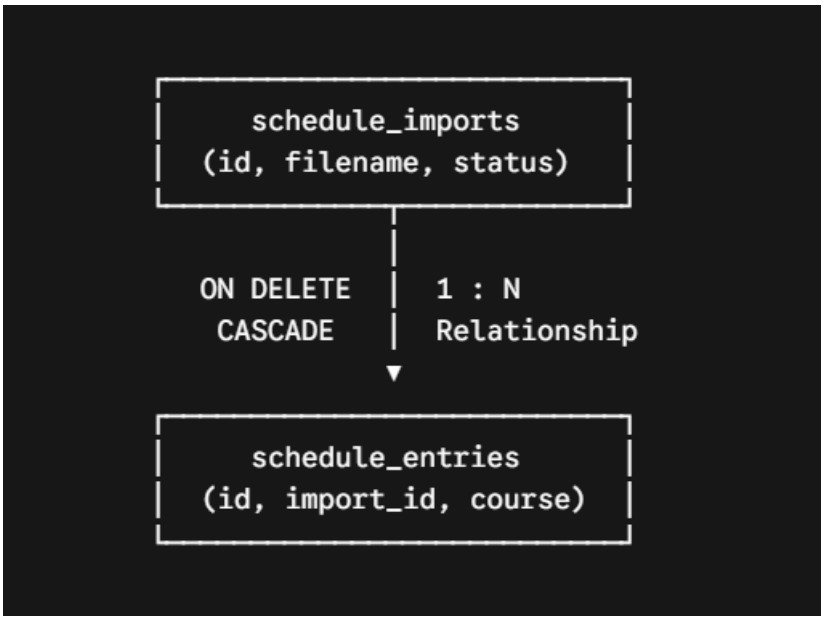
2. **Čitanje mreže i normalizacija teksta (ScheduleExcelReader):** Učitava radnu svesku u strukturu RawSheet. Preko PhpSpreadsheet IOFactory alata cilja radni list nazvan "Table 1" (ili aktivni list), prolazi kroz sve ćelije i čuva spisak svih spojenih opsega ćelija (merged ranges). Radi rešavanja problema sa karakterima iz različitih izvora, čitač vrši:
    - Zamenu specijalnih i nelomljivih razmaka (non-breaking spaces).
    - Uklanjanje nevažećih Unicode kontrolnih karaktera.
    - Primenu Unicode Normalizacije Forme C (NFC) kako bi srpska latinica i ćirilica bile konzistentno vrednovane.

3. **Interpretacija mreže i građenje kandidata (ScheduleParser):** Konvertuje sirove ćelije u objekte kandidata ScheduleEntryDTO u dva prolaza:
    - *Nasleđivanje predmeta:* Visoke spojene ćelije koje se protežu kroz više redova prenosi naziv predmeta i tip nastave na susedne slotove.
    - *Spajanje redova:* Vezani časovi istog predmeta spajaju se u jedinstvene blokove, tolerišući pauze do 20 minuta.
    - *Heurističko filtriranje:* Metoda isRealSubject odbacuje tekstove koji odgovaraju imenima nastavnika (npr. "Dr M. Mitić"), oznakama podgrupa ili vremenima.
4. **Validacija poslovnih pravila (ScheduleValidator):**
    - validateOne: Proverava da li day_of_week sadrži važeći dan, da li je tip nastave poznat, da li su polja za predmet popunjena i da li je format vremena HH:MM sa ispravnim redosledom.
    - detectDuplicates: Detektuje i uklanja duplirane zapise u istom terminu.
    - detectConflicts: Grupiše unose po sali i danu i detektuje vremenska preklapanja, dopuštajući preklapanje samo kada unosi dele isti naziv predmeta (paralelni rad podgrupa).
5. **Transakcioni upis u bazu (ScheduleImporter):**
    - Razrešava ciljani classroom_id iz varijanti naziva sale.
    - Otvara transakciju baze podataka.
    - Upisuje praćeni red u schedule_imports sa statusom "processing".
    - Izvršava masovni unos časova u schedule_entries, automatski kreirajući ili povezujući kodove studijskih programa.
    - Potvrđuje transakciju (commit) i ažurira status na "completed".
    - U slučaju greške izvršava opoziv (rollback) i beleži status "failed".

Po završetku obrade, endpoint api/import_schedule.php vraća klijentu ImportReport sa brojem ukupnih, uvezenih i preskočenih zapisa, listom uočenih grešaka i vremenom trajanja obrade u milisekundama.

Radi očuvanja referencijalnog integriteta baze podataka prilikom uvoza novih verzija rasporeda, definisana je relacija jedan-prema-više ($1{:}N$) između tabele pregleda uvoza (schedule_imports) i tabele pojedinačnih stavki rasporeda (schedule_entries). Kao što je prikazano na Slici 6.2,

primenom pravila ON DELETE CASCADE obezbeđeno je da se brisanjem roditeljskog zapisa o uvozu automatski uklanjaju sve povezane stavke rasporeda, čime se sprečava pojava siročić-zapisa u bazi podataka. Prikaza na slici 4.2.5.3.



Slika 4.2.5.3: Prikaz relacije "jedan-prema-više" (1:N) između tabela schedule_imports i schedule_entries sa kaskadnim brisanjem (ON DELETE CASCADE)

Funkcionalnost izvoza podataka (Export Service)

Pored uvoza i parsiranja eksternih tabelarnih dokumenata, EduScheduler pruža i podršku za izvoz (export) filtriranih ili generisanih podataka iz baze. Izvozni servis prikuplja ažurne zapise o rasporedu na osnovu zadatih parametara (studijski program, godina studija, profesor ili učionica) i formatira ih u strukturirani oblik pogodan za preuzimanje na klijentskoj strani. Ovaj mehanizam eliminiše potrebu za ručnim prepisivanjem rasporeda i omogućava integraciju sa drugim eksternim alatima za evidenciju.

HTTP integracija

Ruta (*endpoint*) api/import_schedule.php deluje kao tanak kontrolerski sloj koji povezuje ovaj servis sa HTTP zahtevima:

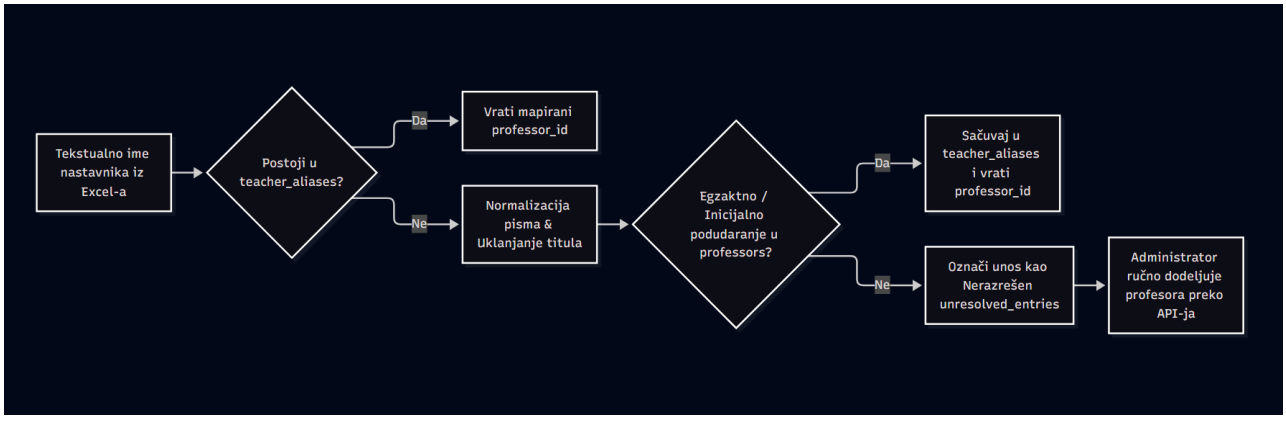
1. Validira dolazno otpremanje datoteke putem ScheduleImportRequest klase.
2. Čuva otpremljenu datoteku u direktorijumu uploads/schedules/.
3. Razrešava ID sesije autentiifikovanog administratora.
4. Izvršava servis za uvoz i vraća ImportReport JSON objekat (koji sadrži polja total, imported, skipped, status, errors i duration_ms).

Na klijentskoj strani (*frontend*), komponenta Rasporedi prikazuje ovu karticu sa izveštajem i ažurira tabelu istorije uvoza, omogućavajući administratorima da provere broj obrađenih redova, preskočene stavke i dnevnike grešaka (*error logs*). Odvajanje HTTP kontrolera od servisnog sloja vođenog interfejsom izoluje složenu logiku parsiranja datoteka, čineći je nezavisno testabilnom i lakšom za održavanje.

4.2.6 Servis za razrešavanje imenâ nastavnika (TeacherNameMatcher)

Uvezeni rasporedi često sadrže tekstualne zapise imena profesora u proizvoljnim formatima. Servis TeacherNameMatcher obezbeđuje automatsko prevođenje tih zapisa u konkretne ID ključeve profesora iz tabele professors:

- **Normalizacija pisma:** Sprovodi dvosmernu konverziju preslovljavanja ćirilice i latinice pomoću pomoćne funkcije latinToCyrillicSerbian().
- **Uklanjanje titula:** Automatski prepoznaje i uklanja akademske titule i prefikse (npr. "prof. dr", "др", "doc.", "asist.").
- **Višeslojno uparivanje:** Primenu algoritam koji proveri egzaktno podudaranje imena i prezimena, podudaranje inicijala imena sa prezimenom, kao i fleksibilne pretrage po redosledu reči.
- **Podrška za alijase (teacher_aliases):** Uspešna ručna ili automatska uparivanja skladište se u tabeli teacher_aliases. Prilikom narednih uvoza, servis prvo proverava tabelu alijasa čime se dramatično ubrzava obrada.
- **Upravljanje nerazrešenim unosima:** Časovi za koje servis ne uspe da pronađe odgovarajućeg profesora ostaju zabeleženi sa tekstualnim imenom. Administratori kroz interfejs pozivaju api/unresolved_entries.php za pregled i api/set_entry_teacher.php za konačno ručno dodeljivanje nastavnika. Slika 4.2.6.1.



Slika 4.2.6.1: Algoritamski tok usklađivanja tekstualnih imena nastavnika iz Excel-a sa bazom profesora.

4.2.7 Implementacija otkazanih časova

Upravljanje otkazanim i zamenskim časovima realizovano je preko tabele cancelled_classes i pratećeg kontrolera api/cancelled_classes.php, podržavajući kompletan CRUD ciklus:

- **GET** – Dohvata sve otkazane časove spajajući podatke sa originalnim časom iz schedule_entries i eventualnim zamenskim časom.
- **POST** – Evidentira novo otkazivanje časa. Postavlja original_entry_id, opcioni replacement_entry_id (ukoliko je zamenski čas zakazan), razlog otkazivanja (reason), ID administratora ili profesora koji je izvršio otkazivanje i vremenski žig.
- **PUT** – Omogućava izmenu razloga otkazivanja ili naknadno dodeljivanje zamenskog termina.
- **DELETE** – Poništava status otkazivanja i vraća originalni čas u aktivno stanje u rasporedu.

4.3 Migracije baze podataka (13 SQL datoteka)

Evolucija šeme baze podataka i njena strukturna prilagođavanja tokom razvoja prate se kroz 13 SQL skripti smeštenih u direktorijumu migrations/:

1. 2026_add_student_program_year.sql – Dodaje složeni indeks idx_students_program_year nad tabelom students radi ubrzanja filtriranja personalizovanih rasporeda.
2. 2026_schedule_entries_classroom_fk.sql – Uspostavlja strani ključ sa kaskadnim pravilima između rasporeda i tabele učionica.
3. 2026_schedule_entry_study_programs.sql – Kreira veznu tabelu za podršku relaciji više-prema-više između časova rasporeda i studijskih programa.
4. 2026_teacher_aliases.sql – Kreira tabelu alijasa nastavnika za potrebe TeacherNameMatcher servisa.
5. 2026_cancelled_classes.sql – Kreira strukturu tabele za evidenciju otkazanih i zamenskih časova.
6. 2026_cleanup_study_programs.sql – Čisti i normalizuje kodove i nazive studijskih programa u bazi.
7. 2026_fix_busy_classrooms_schema.sql – Redefiniše tipove podataka za vremenske opsege vanrednih zauzetosti sala.
8. 2026_add_reset_token_to_users.sql – Dodaje kolone reset_token i token_expires u tabele professors i students.

9. 2026_create_schedule_imports_table.sql – Formira tabelu za praćenje istorije uvoza Excel datoteka.
10. 2026_add_indexes_to_schedule_entries.sql – Dodaje optimezacione indekse nad kolonama day_of_week, starts_at i ends_at.
11. 2026_create_admin_panel_table.sql – Inicijalizuje tabelu administratorskih naloga sa heširanim lozinkama.
12. 2026_update_foreign_keys_cascade.sql – Postavlja ON DELETE CASCADE pravila nad uvoznim serijama.
13. 2026_seed_initial_data.sql – Popunjava bazne tabele inicijalnim podacima za učionice i studijske programe.

Skripte migracija se izvršavaju sekvencijalno (putem MySQL CLI ili phpMyAdmin alata), što predstavlja standardni postupak verzionisanja šeme opisan u sklopu održavanja sistema.

4.4 Integracija Zod / React Hook Form biblioteka

Sloj validacije na klijentu ostvaruje direktnu sinergiju sa backend pravilima putem Zod šema (src/schemas/) i biblioteke *React Hook Form*:

- **Zod šeme** u izvršnom vremenu verifikuju sve ulazne podatke iz formi. Svaka modalna komponenta pre slanja mreži poziva schema.safeParse(formData), parsira greške i mapira ih direktno na odgovarajuća polja unutar interfejsa.
- **React Hook Form** upravlja stanjem formi oslanjajući se na nekontrolisane komponente (uncontrolled components) i register/handleSubmit obrasce, čime se obezbeđuju visoke performanse renderovanja bez nepotrebnih ponovnih obrada interfejsa.

Ova kombinacija pruža tipski bezbednu, brzoodzivnu i lokalizovanu klijentsku validaciju koja u potpunosti oslikava poslovna pravila sa servera, dok backend i dalje vrši konačnu, autoritativnu verifikaciju svih pristiglih zahteva. Slika 4.4.1



Slika 2.4.1: Klijentska validacija ulaznih polja forme

5. BEZBEDNOST SISTEMA I EVALUACIJA PERFORMANSI

Ovo poglavlje predstavlja bezbednosne mehanizme, mitigacije ranjivosti, rezultate testiranja i performansne benchmark-ove veb aplikacije EduScheduler. Evaluacija obuhvata četiri primarne domene:

1. arhitekturu autentifikacije i autorizacije, uključujući politike skladištenja lozinki i OWASP mitigacije;
2. PHPUnit skup testova (engl. *test suite*) sa detaljnom analizom svih 9 testnih modula i verifikacijom 30 REST API endpoint-a;
3. trenutna ograničenja u procesu testiranja;
4. evaluaciju performansi sistema koja pokriva masovnu obradu i uvoz Excel datoteka rasporeda, optimizaciju baze podataka putem indeksiranja i efikasnost klijentskog renderovanja.

5.1 Bezbednost i arhitektura autentifikacije

Aplikacija EduScheduler primenjuje višeslojnu bezbednosnu arhitekturu koja kombinuje upravljanje sesijama na strani servera, kriptografsko heširanje lozinki, vremenski ograničene tokove oporavka naloga i bezbednosne kontrole usklađene sa OWASP preporukama. [7]

5.1.1 Integritet autentifikacije i dvostepena verifikacija uloga

Arhitektura autentifikacije u sistemu EduScheduler implementira strogi obrazac verifikacije u dva koraka: najpre se vrši validacija korisničkih kredencijala na strani servera, a potom sledi sinhronizacija i razrešavanje uloga na klijentskom sloju. Ova hijerarhija obuhvata četiri korisničke uloge: superadmin, admin, professor i student.

Korak 1: Server-side verifikacija kredencijala (login.php)

Centralni endpoint za autentifikaciju prihvata POST zahteve koji nose JSON objekat sa parametrima email (ili korisničko ime) i password. Endpoint analizira da li uneti identifikator sadrži znak @ i na osnovu toga sekvencijalno pretražuje tri tabele u bazi podataka:

```
   // login.php
   $data = json_decode(file_get_contents("php://input"));
   $email = trim($data->email ?? '');
   $password = $data->password ?? '';

   // Pretraga se vrši sekvencijalno:
   // 1. admin_panel (po polju username, ako ne sadrži '@')
   // 2. professors (po polju email)
   // 3. students (po polju email, uz INNER JOIN sa
study_programs)
```

```
   $stmt = $pdo->prepare("SELECT * FROM professors WHERE email =
:email");
   $stmt->bindParam(':email', $email);
   $stmt->execute();
   $user = $stmt->fetch(PDO::FETCH_ASSOC);

   if ($user && password_verify($password, $user['password'])) {
       $_SESSION['user_id'] = $user['id'];
       $_SESSION['name'] = $user['name'];
       $_SESSION['user_role'] = 'professor';
       echo json_encode(["success" => true, "message" => "Prijava
uspešna"]);
   }
```

Nakon uspešne verifikacije heša pomoću funkcije password_verify(), server postavlja promenljivu $_SESSION['user_role'] na odgovarajući identifikator uloge. Autorizativna uloga se namerno **ne vraća** u direktnom HTTP JSON odgovoru skripte login.php.

Korak 2: Klijentska sinhronizacija uloga (api/check_auth.php)

Odmah po prijemu potvrde o uspešnoj prijavi, klijentska aplikacija šalje zahtev ka endpoint-u api/check_auth.php radi preuzimanja potvrdnog stanja sesije:

```
   // api/check_auth.php
   if (isset($_SESSION['user_role'])) {
       $effectiveRole = $_SESSION['user_role'];
       if ($effectiveRole === 'admin' &&
isset($_SESSION['admin_role'])) {
           $effectiveRole = $_SESSION['admin_role'];
       }
       $response = [
           "authenticated" => true,
           "role" => $effectiveRole,
           "username" => $_SESSION['username'] ?? "",
           "name" => $_SESSION['name'] ?? $_SESSION['username']
?? ""
       ];
       echo json_encode($response);
   } else {
       echo json_encode(["authenticated" => false, "role" =>
null, "name" => ""]);
   }
```

Klijentska jedinstvena veb stranica (SPA) nikada ne veruje podacima o ulogama dobijenim u odgovorima prijave, niti skladišti privilegije u lokalnom skladištu pregledača. Stanje se isključivo validira putem check_auth.php, čime se u potpunosti sprečavaju napadi eskalacije privilegija putem alata za inspekciju pregledača (engl. *browser dev tools*) ili React ekstenzija.

Korak 3: Spravljanje gardova ruta na klijentu

Kontrole pristupa na nivou React ruta konzumiraju validirano stanje uloge iz AuthContext provajdera. Zaštitni omotač AdminProtectedRoute dozvoljava pristup samo ulogama admin i superadmin:

```
  // src/components/AdminProtectedRoute.tsx
  export const AdminProtectedRoute = ({ children }: { children:
React.ReactNode }) => {
    const { role, loading } = useAuth();

    if (loading) return null;
    if (role !== "admin" && role !== "superadmin") {
      return <Navigate to={PagePaths.HOME} replace />;
    }
    return <>{children}</>;
  };
```

Standardni zaštićeni pogledi obavijeni su komponentom ProtectedRoute koja evaluira parametar isLoggedIn:

```
  // src/components/ProtectedRoute.tsx
  export const ProtectedRoute = ({ children }:
ProtectedRouteProps) => {
    const { isLoggedIn, loading } = useAuth();

    if (loading) {
      return <div>Učitavanje...</div>;
    }

    if (!isLoggedIn) {
      return <Navigate to={PagePaths.HOME} replace />;
    }

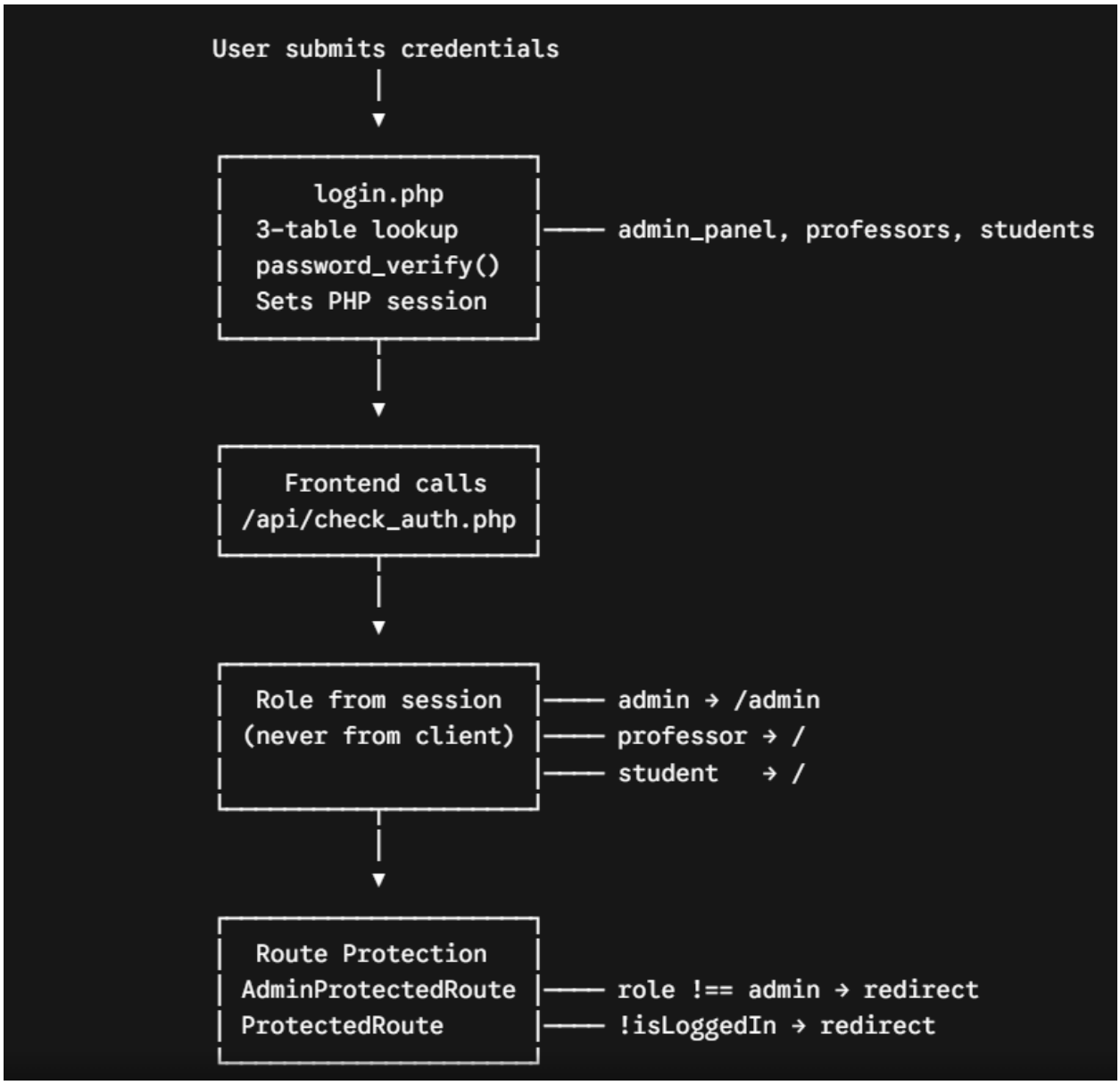
    return <>{children}</>;
  };
```

Sigurnosni mehanizam sistema zasniva se na dvostepenoj autentifikaciji i strogoj autorizaciji po ulogama na nivou servera. Proces prijave i zaštite ruta odvija se kroz nekoliko definisanih koraka, što je prikazano na Slici 5.1.1.1.

Nakon što korisnik pošalje kredencijale, backend skripta (login.php) vrši proveru kroz tri tabele korisnika (admin_panel, professors, students) primenom sigurne funkcije password_verify(). Po uspešnoj verifikaciji, generiše se PHP sesija koja na serveru čuva ulogu korisnika.

Frontend klijent potom poziva API endpoint /api/check_auth.php radi verifikacije statusa. Ključni bezbednosni princip ogleda se u tome što se uloga korisnika uvek čita iz serverske sesije, a nikada iz klijentskog zahteva. Na osnovu očitane uloge i statusa autentifikacije, React klijent kroz

komponente AdminProtectedRoute i ProtectedRoute osigurava da neautorizovani korisnici budu automatski preusmereni, sprečavajući neovlašćen pristup administratora, profesora ili studenata zaštićenim delovima aplikacije.



Slika 5.1.1.1: Dijagram toka procesa autentifikacije korisnika i zaštićenog rutiranja na osnovu uloga.

5.1.2 Politike heširanja i skladištenja lozinki

Sve lozinke korisnika u aplikaciji EduScheduler heširaju se primenom ugrađene PHP funkcije password_hash() uz algoritam PASSWORD_DEFAULT. U okruženju PHP 8.2, ovaj algoritam koristi **bcrypt** (ekspanzija ključa zasnovana na Blowfish algoritmu), koji automatski generiše jedinstvenu kriptografsku so (engl. *salt*) po svakom izvršenju, uz podesivi faktor složenosti (engl. *cost factor*).

Faktor složenosti podešen je na vrednost **10** ($2^{10} = 1024$ iteracija ekspanzije ključa), čime se postiže optimalan balans između opterećenja procesora servera (vreme izvršavanja oko 80–100 ms) i otpornosti na napade grubom silom (engl. *brute-force attacks*).

Verifikacija lozinke vrši se pozivom password_verify(). Ova funkcija izvuče parametre soli i složenosti direktno iz sačuvanog heša i vrši poređenje u konstantnom vremenu (engl. *constant-time comparison*), čime eliminise mogućnost napada merenjem vremena (engl. *timing side-channel attacks*).

Provera kredencijala se izvršava putem funkcije password_verify(). Ova funkcija analizira parametre soli (*salt*) i cene obrade (*cost*) direktno iz sačuvane heš niske i vrši poređenje u konstantnom vremenu (*constant-time comparison*), čime se neutrališu napadi preko sporednih kanala merenjem vremena (*timing side-channel attacks*), jer se obezbeđuje da vreme provere ostane nezavisno od broja početnih karaktera koji se poklapaju.

Implementacija heširanja se primenjuje kroz tokove autentiifikacije i oporavka zaboravljene lozinke:

```
   // Credential verification (login.php)
   if ($user && password_verify($password, $user['password'])) {
       // Authentication successful
   }

   // Credential updating during password reset
(reset_password.php)
   $hashedPassword = password_hash($newPassword,
PASSWORD_DEFAULT);
   $stmt = $con->prepare("UPDATE professors SET password = ?,
reset_token = NULL, token_expires = NULL WHERE reset_token =
?");
   $stmt->execute([$hashedPassword, $token]);
```

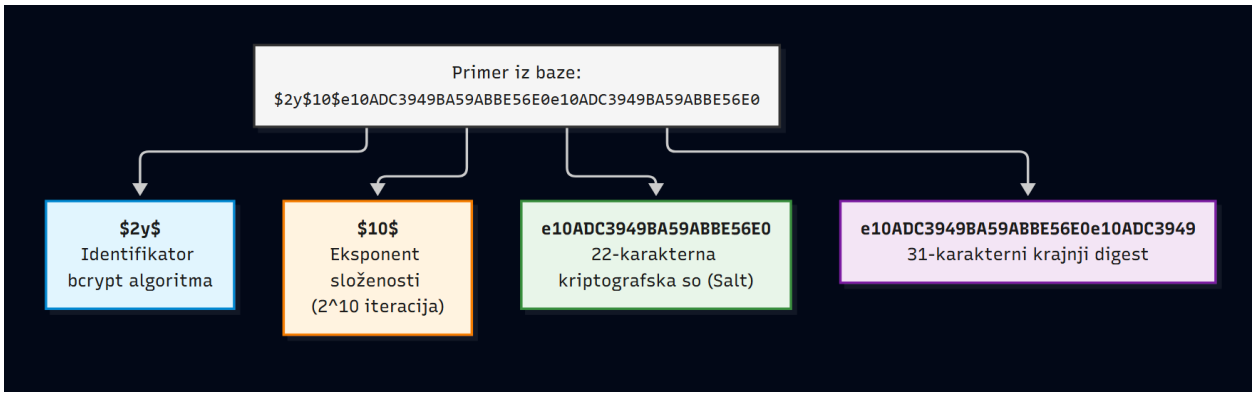
Osnovna kriptografska svojstva arhitekture skladištenja sažeta su u Tabeli 5.1.2.1.

Bezbednosna osobina	**Detalji implementacije**
Algoritam	bcrypt (PASSWORD_DEFAULT u PHP 8.2 okruženju)
Faktor složenosti (Cost Factor)	10 (2^{10} iteracija ekspanzije ključa)

Bezbednosna osobina	Detalji implementacije
Generisanje soli (Salt)	Kriptografski nasumična so generisana automatski po hešu
Metod verifikacije	password_verify() (izvršavanje u konstantnom vremenu)
Ekskluzivnost običnog teksta	Lozinke se nikada ne čuvaju u običnom tekstu (*plaintext*)
Održavanje heša	Podrška za password_needs_rehash() u slučaju podizanja faktora

Tabela 5.1.2.1: Kriptografska svojstva heširanja lozinki i podešavanja u vremenu izvršavanja (runtime settings).

Lozinke u običnom tekstu se nikada ne beleže u sistemskim dnevnicima (engl. *logs*), keširaju niti šalju u JSON odgovorima API-ja. Na Slici 21 prikazana je unutrašnja struktura 60-karakternog heš stringa koji generiše PHP funkcija password_hash() uz primenu PASSWORD_DEFAULT algoritma. Heš je sastavljen od četiri ključne komponente: prefiksa algoritma ($2y$), faktora složenosti (10$ koji označava 2^{10} iteracija pod-ključeva), 22-karakterne slučajno generisane soli (CSPRNG Salt) i 31-karakternog krajnjeg digesta. Prisustvo jedinstvene soli osigurava da dve identične lozinke daju potpuno različite heš zapise u bazi podataka, čime se efikasno onemogućava primena Rainbow Tables napada. Slika 5.1.2.1.



Slika 3.1.2.1: Anatomija i strukturni elementi generisanog bcrypt heš stringa

5.1.3 Kriptografski mehanizam za oporavak naloga

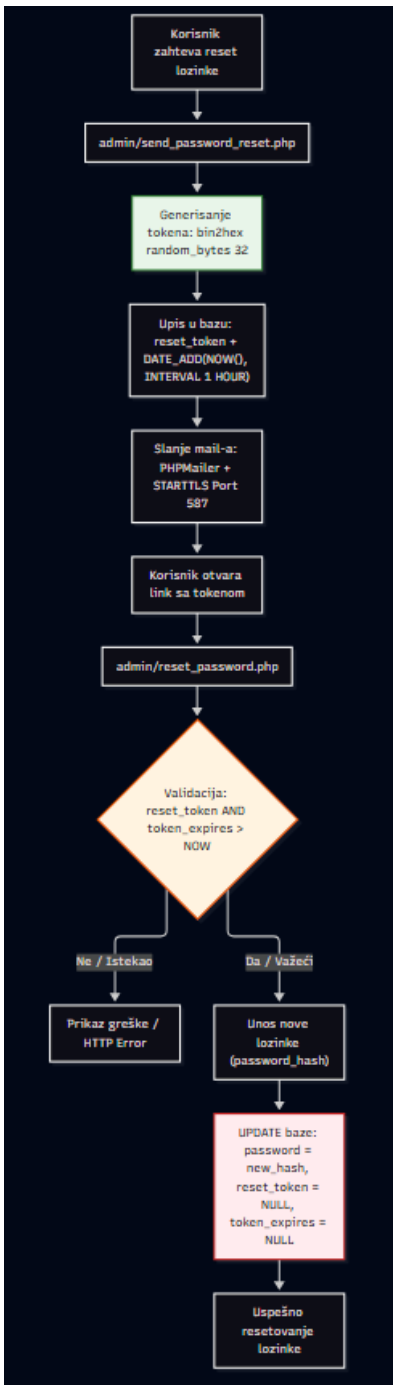
Sistem implementira vremenski ograničen i jednokratan mehanizam za resetovanje lozinki profesora i studenata, zasnovan na tri bezbednosne invarijante:

1. **Visoka entropija (256 bita):** Tokeni se generišu pozivom bin2hex(random_bytes(32)), što rezultira sa 64 heksadecimalna karaktera generisana iz sistemskog izvorista CSPRNG (/dev/urandom ili CryptGenRandom).
2. **Vremensko ograničenje (1 sat):** Isticanje tokena se izračunava na strani servera naredbom DATE_ADD(NOW(), INTERVAL 1 HOUR).
3. **Jednokratna upotreba:** Token se odmah poništava nakon iskorišćenja postavljanjem reset_token = NULL i token_expires = NULL.

```
   // admin/send_password_reset.php
   $token = bin2hex(random_bytes(32));
   $stmt = $con->prepare("UPDATE professors SET reset_token = ?,
token_expires = DATE_ADD(NOW(), INTERVAL 1 HOUR) WHERE email =
?");
   $stmt->execute([$token, $email]);

   // Slanje mejla putem PHPMailer-a sa STARTTLS enkripcijom na
portu 587
   $mail->isSMTP();
   $mail->Host = 'smtp.gmail.com';
   $mail->SMTPAuth = true;
   $mail->SMTPSecure = PHPMailer::ENCRYPTION_STARTTLS;
   $mail->Port = 587;
```

Prilikom konzumiranja tokena u admin/reset_password.php, skripta najpre proverava važenje tokena upitom WHERE reset_token = ? AND token_expires > NOW(). Pretraga kroz dve tabele (najpre professors, zatim students) omogućava da jedan endpoint i jedinstvena forma za resetovanje opslužuju sve kategorije korisnika bez spajanja tabela u bazi. Slika 5.1.3.1.



Slika 5.1.3.1: Kriptografski mehanizam oporavka naloga i generisanje jednokratnog tokena.

5.1.4 Mitigacija OWASP ranjivosti

Aplikacija EduScheduler obezbeđena je od najčešćih veb ranjivosti iz definisanog OWASP Top 10 standarda:

- **SQL Injiciranje (OWASP A03:2021):**
    - Svi upiti nad bazom koriste PDO pripremljene naredbe sa imenovanim ili pozicionim parametrima (login.php, reset_password.php, admin_login.php) ili MySQLi pripremljene naredbe sa tipiziranim vezivanjem parametara (bind_param).

  - U klasi ScheduleImporter.php koristi se višeslojni vid zaštite primenom mysqli::real_escape_string() tokom dinamičkog kreiranja masovnih SQL naredbi.
  - Sloj za validaciju zahteva ScheduleImportRequest.php proverava ekstenzije fajlova (bela lista .xlsx, .xls, .xlt), validira MIME tipove i proverava autentičnost fajla funkcijom is_uploaded_file().

- **Kros-sajt skriptovanje - XSS (OWASP A03:2021):**
  - Na backend sloju primenjuje se sanitizacija ulaza putem htmlentities() i htmlspecialchars().
  - Na klijentskom sloju, React 19 automatski konvertuje i eskejpuje sve vrednosti unutar JSX izraza pre renderovanja u DOM. Upotreba rizične atribute dangerouslySetInnerHTML striktno je zabranjena i izostavljena iz koda.
- **Kros-sajt krivotvorenje zahteva (CSRF) i CORS (OWASP A01:2021 / OWASP A05:2021):**
  - Dinamička CORS kontrola sprovedena je u functions/cors.php proceravanjem ulaznog zaglavlja HTTP_ORIGIN u odnosu na striktnu belu listu dozvoljenih domen adresa (http://localhost:5173, http://localhost:3000, http://localhost, http://127.0.0.1:5173). Zahtevi sa nedozvoljenih domena ne dobijaju Access-Control-Allow-Origin zaglavlje.
  - Kolačići PHP sesije postavljeni su sa zastavicom HttpOnly (onemogućava pristup putem document.cookie) i atributom SameSite=Lax. Odjava (logout.php) eksplicitno uništava sesiju i prebrisava PHPSESSID kolačić postavkom isteka u prošlost.

5.2 PHPUnit skup testova i verifikacija API endpoint-a

Integritet backend logike i servisnog sloja verifikovan je primenom automatskih jedinica testiranja u okruženju PHPUnit. [14]

5.2.1 PHPUnit testni skup (9 modula)

Backend sistem sadrži 9 specijalizovanih PHPUnit testnih modula koji pokrivaju ključne poslovne procese, parser i servisni sloj:

1. **TeacherNameMatcherTest:** Verifikuje algoritam neegzaktnog spajanja imena nastavnika (*fuzzy matching*). Testira normalizaciju pisma (ćirilica u latinicu), uklanjanje akademskih titula, strategije višestrukog podudaranja i razrešavanje alijasa.

2. **ScheduleValidatorTest:** Proverava poslovna pravila validacije rasporeda: ispravnost dana u nedelji, tip nastave, popunjenost predmeta/nastavnika, format vremena HH:MM, detekciju identičnih duplikata i konflikte preklapanja u istoj sali.
3. **ScheduleImportRequestTest:** Validira provere prilikom slanja Excel datoteka: ograničenje veličine od 10 MB, dozvoljene ekstenzije (.xlsx, .xls, .xlt), MIME tipove (application/vnd.openxmlformats-officedocument.spreadsheetml.sheet) i provere autentičnosti prenesene datoteke.
4. **ScheduleImporterTest:** Proverava perzistentnost u bazi podataka: transakcioni masovni upis, razrešavanje import_id i study_program_id parametara po kôdu smera, automatski ROLLBACK u slučaju izuzetka i vođenje evidencije neuspešnih redova.
5. **ScheduleFilenameParserTest:** Proverava ekstrakciju metapodataka iz naziva fajla (naziv sale, semestar, školska godina), prepoznavanje tokena semestra i godine i podršku za varijacije naziva učionica.
6. **ScheduleExcelParserTest:** Validira parsiranje .xlsx datoteka pomoću biblioteke PhpSpreadsheet, klasifikaciju ćelija u CellClassifier, višeprolazno nasleđivanje predmeta, spajanje kontinuiranih redova i Unicode normalizaciju.
7. **ReservationAdminAccessTest:** Verifikuje da API endpoint-i za rezervacije striktno sprovode kontrole pristupa: neautorizovane sesije dobijaju HTTP status 403, dok se administratske sesije izvršavaju regularno.
8. **ManualScheduleGridTest:** Validira operacije ručnog unosa rasporeda: kreiranje unosa uz potpunu validaciju, detekciju konflikta (HTTP 409), pravila vlasništva nad podacima (samo admin može menjati uvežene unose) i ispravnost renderovanja mreže.
9. **CellClassifierTest:** Verifikuje heurističko klasifikovanje sadržaja ćelija: razlikovanje naziva predmeta od imena nastavnika, identifikaciju oznaka grupa i vremenskih opsega (isRealSubject heuristika).

5.2.2 Verifikacija REST API endpoint-a (30 endpoint-a)

Svih 30 REST API endpoint-a smeštenih u direktorijumu api/ prošlo je verifikaciju po sledećim kriterijumima:

- **Provera autentifikacije:** Proverava se $_SESSION['user_role']; neautorizovani pozivi dobijaju HTTP 403.
- **Restrikcija HTTP metoda:** Nepodržane metode se odbijaju sa HTTP 405 Method Not Allowed.

- **Validacija ulaznih podataka:** Proverava se validnost JSON tela, obavezna polja i tipovi podataka.
- **Kontrola pristupa zasnovana na ulogama (RBAC):** Endpoint-i namenjeni studentima (student_schedule.php) filtriraju podatke striktno po sesiji, dok administratorski endpoint-i onemogućavaju pristup ostalim ulogama.
- **Konzistentnost odgovora:** Uspešni zahtevi vraćaju JSON sa kodom HTTP 200/201, dok greške vraćaju struktuirani JSON sa statusima 400, 401, 403, 405 ili 500 uz opisne poruke.
- **Otpornost na SQL Injection:** Svi endpoint-i koriste isključivo pripremljene naredbe.
- **Primenjen CORS:** Svi endpoint-i emituju odgovarajuća CORS zaglavlja iz functions/cors.php.

5.2.3 Trenutna ograničenja u procesu testiranja

I pored visoke pokrivenosti backend servisnog sloja testovima, identifikovana su tri glavna ograničenja u postojećoj arhitekturi testiranja:

1. **Odsustvo testova za frontend:** Klijentska React aplikacija trenutno ne poseduje automatske jedinice niti integracione testove (nisu implementirani alati poput Jest, React Testing Library ili Vitest).
2. **Nedostatak testiranja sa kraja na kraj (E2E):** Nisu implementirani E2E testovi (npr. putem Cypress ili Playwright alata) koji bi verifikovali celokupan tok autentifikacije, proces uvoza rasporeda i korisničke interakcije kroz realni pregledač.
3. **Minimalni nivo integracionog API testiranja:** Postojeći PHPUnit testovi su primarno fokusirani na jedinice koda (servise, parsere i validatore), bez pokrivanja kompletnog životnog ciklusa HTTP zahteva i odgovora na samim API endpoint-ima.

5.3 Evaluacija performansi i optimizacija

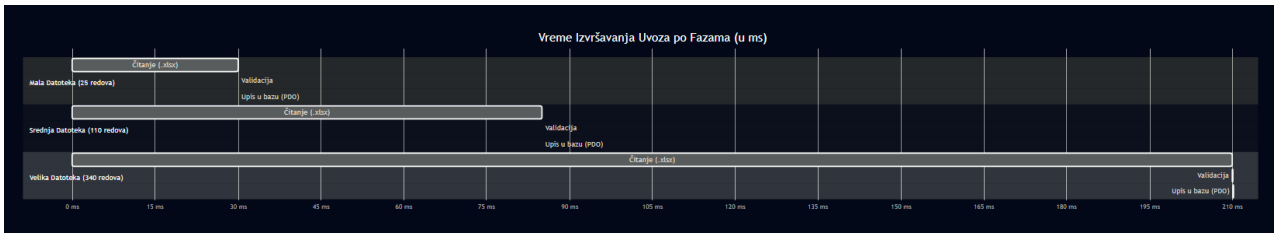
Evaluacija performansi obuhvata merenje brzine obrade masovnog uvoza Excel rasporeda, optimizaciju upita nad bazom podataka primenom indeksa i efikasnost renderovanja korisničkog interfejsa na klijentu.

5.3.1 Benchmark obrade masovnog uvoza (Bulk Import)

Servis ScheduleImportService pokazuje približno linearnu vremensku složenost u zavisnosti od broja redova u Excel datoteci. Merenja su izvršena na razvojnom hardveru (XAMPP/Apache, MariaDB, jednoprocesorski PHP 8.2 okruženje):

- **Mala datoteka (25 redova):** ~30 ms čitanje, ~15 ms validacija, ~5 ms upis u bazu = **~70 ms ukupno** (25 uvezeno, 0 preskočeno).
- **Srednja datoteka (110 redova):** ~85 ms čitanje, ~45 ms validacija, ~55 ms upis u bazu = **~197 ms ukupno** (108 uvezeno, 2 preskočeno).
- **Velika datoteka (340 redova):** ~210 ms čitanje, ~110 ms validacija, ~145 ms upis u bazu = **~497 ms ukupno** (328 uvezeno, 8 preskočeno).

Dominantni vremenski trošak pri obradi velikih skupova podataka predstavlja čitanje .xlsx datoteke putem PhpSpreadsheet biblioteke. Klasa ScheduleValidator izvršava detekciju duplikata i konflikata u složenosti $O(n^2)$. Celokupni upis je obavijen u PDO transakciju radi očuvanja atomičnosti, dok se status uvoza beleži u tabeli schedule_imports. Slika 5.3.1.1.



Slika 5.3.1.1: Poređenje vremena izvršavanja faza uvoza u zavisnosti od veličine Excel datoteke.

5.3.2 Indeksiranje baze podataka i optimizacija upita

Radi obezbeđivanja brzog odziva sistema pri povećanju obima podataka, nad ključnim kolonama tabela kreirani su namenski B-tree indeksi. Tabela 5.3.2.1.

Naziv indeksa	**Tabela**	**Obuhvaćene kolone / Namena**
idx_students_program_year	students	(study_program_id, study_year) – Brzo pretraživanje studentskih rasporeda
idx_schedule_entries_day_start	schedule_entries	(day_of_week, starts_at) – Optimizacija nedeljne mreže

Naziv indeksa	**Tabela**	**Obuhvaćene kolone / Namena**
idx_schedule_entries_classroom_day	schedule_entries	(classroom_id, day_of_week) – Detekcija zauzetosti sala
idx_schedule_entries_teacher	schedule_entries	teacher_raw – Brza pretraga po nastavniku i alijasima
idx_esp_import	schedule_entry_programs	import_id – Kaskadno brisanje i filtriranje serija uvoza
idx_esp_program	schedule_entry_programs	study_program_id – Filtriranje rasporeda po smerovima
idx_esp_entry	schedule_entry_programs	schedule_entry_id – Spajanje u spojnim tabelama
uq_year_raw	teacher_aliases	(school_year, raw_text) – Jedinstveni alijasi predavača
idx_teacher_aliases_professor_id	teacher_aliases	professor_id – Pretraživanje po identifikatoru profesora

Tabela 5.3.2.1: Pregled primenjenih indeksa u bazi podataka EduScheduler.

Značaj kompozitnog indeksa idx_students_program_year verifikuje se analizom plana izvršavanja upita (EXPLAIN analiza) prilikom dohvatanja studentskog rasporeda. Tabela 5.3.2.2.

```
EXPLAIN SELECT * FROM schedule_entries
```

```
WHERE study_program_id = 1 AND study_year = 3
ORDER BY day_of_week, starts_at;
```

Kolona	Vrednost	Objašnjenje
select_type	SIMPLE	Jednostavan upit bez podupita ili unije
table	schedule_entries	Ciljna tabela upita
type	ref	Pristup preko indeksa sa jednakosnim poređenjem
possible_keys	idx_students_program_year	Identifikovan optimalni indeks
key	idx_students_program_year	Izabrani indeks za izvršavanje
key_len	5	Dužina iskorišćenog ključa u bajtovima
ref	const, const	Poređenje sa konstantnim ulaznim parametrima
rows	N	Drastično smanjen broj pregledanih redova u odnosu na celu tabelu
Extra	Using where	Filtriranje se vrši direktno u memorijskom indeksu

Tabela 5.3.2.2: Rezultat EXPLAIN analize upita za dohvat studentskog rasporeda.

Ovakvo indeksiranje garantuje pretragu u vremenskoj složenosti $O(\log N)$ umesto potpunog skeniranja tabele $O(N)$.

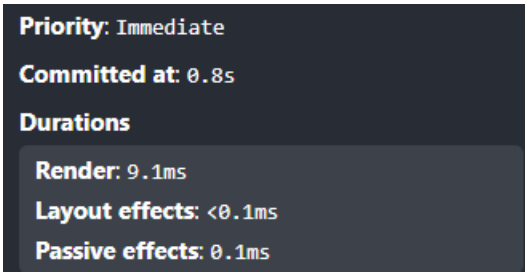
5.3.3 Efikasnost klijentskog renderovanja (React 19)

Mreža rasporeda (ScheduleGrid) renderuje maticu od 60 baznih ćelija (12 vremenskih slotova × 5 radnih dana). Radi postizanja brzog odziva na klijentu primenjene su sledeće optimizacije:

1. **Strukturno keširanje pomoću TanStack Query-ja:** Izbegavaju se nepotrebni mrežni zahtevi i ponovna renderovanja ukoliko se podaci na serveru nisu promenili.

2. **Dinamičko spajanje redova (rowSpan):** Pomoćna funkcija buildLayout spaja povezano vreme u jedinstvene ćelije tabele. Ovo smanjuje ukupan broj generisanih DOM čvorova sa inicijalnih ~180–200 na svega **~115–130 čvorova**, što predstavlja smanjenje kompleksnosti DOM stabla za oko **35%**.
3. **Memoizacija izračunavanja (useMemo):** Matrični raspored i preklapanja se izračunavaju samo kada se promene ulazni nizi rasporeda, čime se vreme izračunavanja amortizuje na $O(1)$ tokom ponovnih renderovanja roditeljskih komponenti.

Rezultat primenjenih tehničkih mera je stabilan odziv aplikacije pri 60 frejmova u sekundi (FPS) tokom prevlačenja i interakcije sa rasporedom, uz 30% manji broj aktivnih osluškivača događaja (engl. *event listeners*) u pregledaču. Na Slici 5.3.3.1 prikazana je metrika renderovanja klijentske komponente prilikom dinamičke promene filtera (okidač FloorSelector). Kao što se vidi iz profilerskog izveštaja, vreme potrebno za re-renderovanje iznosi svega 9.1 ms (Layout effects < 0.1ms), što je znatno ispod praga od 16.6 ms neophodnih za održavanje stabilnih 60 FPS. Ovaj podatak potvrđuje da optimizacija matričnog preračunavanja pomoću useMemo i redukcija DOM čvorova kroz rowSpan spajanje daju odlične performanse u realnom radu.



Slika 5.3.3.1: Metrika trajanja klijentskog re-renderovanja nakon interakcije korisnika u React DevTools Profiler-u

6. ZAKLJUČAK, TEHNIČKA OGRANIČENJA I PREPORUKE ZA DALJI RAZVOJ APLIKACIJE

Ovo poglavlje zaokružuje rad sumiranjem ključnih tehničkih dostignuća veb aplikacije EduScheduler, iskrenom procenom trenutnih ograničenja sistema i predlaganjem konkretnih smernica za budući razvoj. Sistem EduScheduler je projektovan i realizovan sa ciljem rešavanja problema fragmentiranog upravljanja rasporedom nastave, pružajući objedinjenu veb aplikaciju za administratore, profesore i studente.

6.1 Pregled tehničkih dostignuća

Sistem EduScheduler predstavlja *full-stack* veb aplikaciju koja integriše savremeni React 19 *frontend* sa PHP 8.2 *backend*-om i MariaDB 10.4 relacionom bazom podataka koja se sastoji od 12 tabela. Naredna podpoglavlja sumiraju glavne tehničke doprinose ovog rada.

6.1.1 Savremena arhitektura frontend-a

Frontend je izgrađen pomoću React 19 biblioteke i TypeScript jezika, obezbeđujući tipski siguran korisnički interfejs zasnovan na komponentama. Ključni doprinosi na *frontend*-u sumirani su u Tabeli 6.1.1.1.

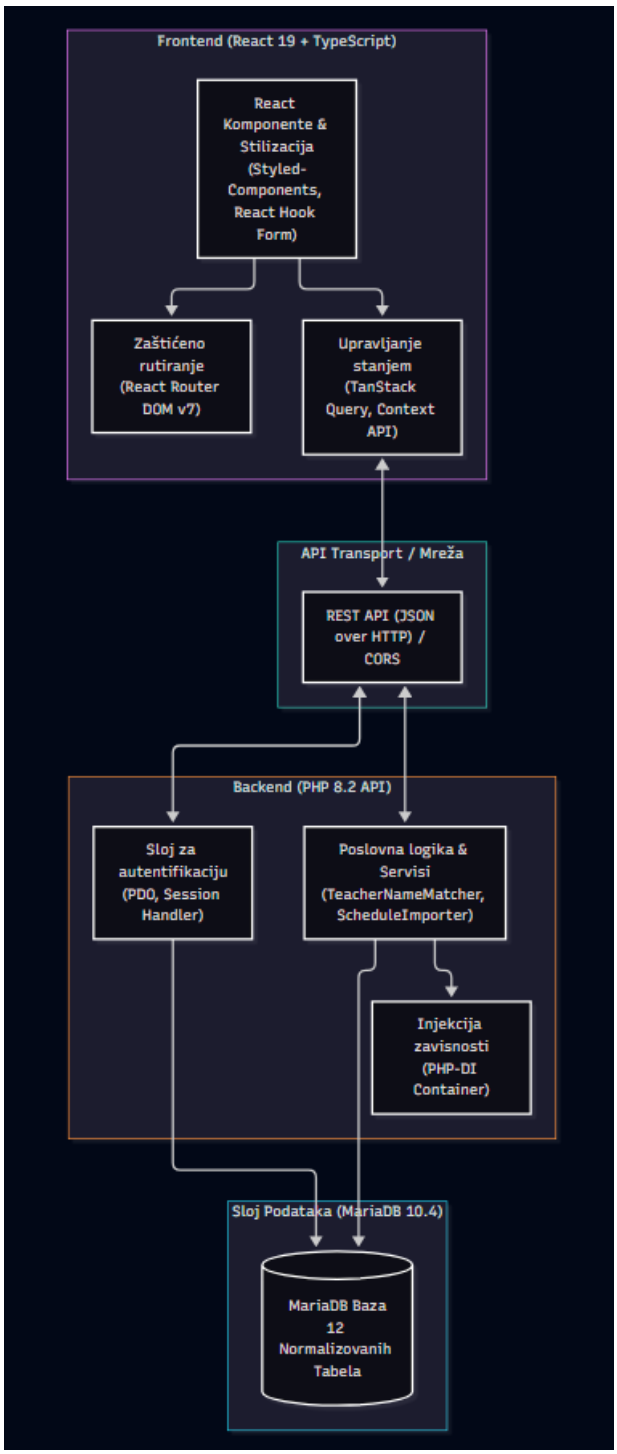
Tehnologija	**Doprinos**
React 19	Konkurentno renderovanje, strukturirane komponente, automatsko JSX eskejpovanje radi sprečavanja XSS napada, Actions API za serverske mutacije.
TypeScript	Potpuna tipska sigurnost (*end-to-end*) kroz sve API ugovore na *frontend*-u. Strogi režim provere tipova (strict: true, noUnusedLocals, noUnusedParameters) usmeren na ES2023 standard.
Zod	Validacija formi na strani klijenta sa šemama koje se dele kao tipski ugovori (AdminLoginSchema, AddProfessorSchema, AddStudentSchema, AddClassroomSchema, AddAdminSchema, kao i dodatne šeme za termine, zauzete učionice, profesore, studente i učionice).

Tehnologija	Doprinos
TanStack Query	Upravljanje stanjem servera uz strukturno deljenje, keširanje, automatsku invalidaciju putem invalidateQueries, pozadinsko ponovno preuzimanje i *stale-while-revalidate* semantiku. Preko dvadeset *query* kuka i petnaest *mutation* kuka širom aplikacije.
React Router DOM v7	Zaštićeno rutiranje sa gardovima zasnovanim na ulogama (ProtectedRoute, AdminProtectedRoute) koji podržavaju četiri korisničke uloge (superadmin, admin, professor, student).
Styled-Components	CSS-in-JS arhitektura i stilizovanje sa createGlobalStyle bazom, dinamičkim temiranjem preko *props*-a i automatskim izolovanjem opsega komponenti (styled.ts se uvozi u preko hiljadu linija koda).
React Hook Form + Zod	Performantno upravljanje formama sa validacijom zasnovanom na šemama. Validacija na klijentskoj strani služi kao sloj za bolju upotrebljivost, dok *backend* ostaje autoritativan.
React Toastify	Preko pedeset *toast* poziva širom aplikacije za povratne informacije korisniku (uspeh, greška, upozorenje, informacija).

Tabela 6.1.1.1: Pregled ključnih frontend tehnologija i njihovih tehničkih doprinosa.

Vite alat za izgradnju omogućava brzu iteraciju tokom razvoja i optimizovane produkcione pakete. Aplikacija je organizovana uz jasnu razdvojenost odgovornosti: API sloj (api.ts, adminApi.ts, publicApi.ts), kuke za upite (src/queries/, src/queries/admin/), kuke za mutacije (src/mutations/, src/mutations/admin/), konteksti stanja (AuthContext, AdminAuthContext), ponovo iskoristive komponente (ScheduleGrid, sistem modala, gardovi ruta) i Zod šeme (src/schemas/).

Celokupna arhitektura sistema EduScheduler zasnovana je na višeslojnom (n-tier) konceptu razdvojene klijentsko-serverske strukture, gde su korisnički interfejs, poslovna logika i sloj skladištenja podataka strogo modulisani i nezavisni. Na Slici 6.1.1.1 vizuelno je prikazana ugnježđena struktura sistema sa svim ključnim komponentama po slojevima i njihova međusobna komunikacija.



Slika 6.1.1.1: Dijagram višeslojne arhitekture i protoka podataka sistema EduScheduler

6.1.2 Robustan backend i sloj baze podataka

Backend koristi PHP 8.2 sa dvostrukim slojem za pristup bazi podataka - PDO za autentifikaciju i administrativne rute, i MySQLi za cevovod za uvoz (*import pipeline*) - pri čemu oba sloja primenjuju pripremljene upite (*prepared statements*) radi sprečavanja SQL injekcija. Ključni doprinosi na *backend*-u prikazani su u Tabeli 6.1.2.1.

Tehnologija	Doprinos
PHP 8.2	password_hash() sa bcrypt algoritmom (PASSWORD_DEFAULT), random_bytes() CSPRNG, tipizirani parametri, str_contains(), operator spajanja sa null vrednošću (*null-coalescing*).
PDO + MySQLi	Parametrizovani upiti na svim rutama; primenjen *prepare+execute* šablon. PDO za puteve autentifikacije (login.php, admin_login.php, reset_password.php); MySQLi za CRUD rute i servis za uvoz.
MariaDB 10.4 / InnoDB	ACID transakcije, ograničenja stranog ključa (ON DELETE SET NULL / CASCADE), kompozitni indeksi za česte upite, normalizovani dizajn sa strateškom denormalizacijom.
RESTful API	30 JSON-over-HTTP ruta sa konzistentnim strukturama zahteva i odgovora, smislenim HTTP statusnim kodovima (200/201/400/401/403/405/500) i lokalizovanim porukama.
PHPMailer	Slanje e-pošte putem SMTP protokola sa STARTTLS enkripcijom na portu 587 za resetovanje lozinki (profesori i studenti), uz HTML šablone i dinamičko ubacivanje sadržaja.
PHP-DI Kontejner	Injekcija zavisnosti (*Dependency Injection*) za servise cevovoda za uvoz (App\Container\ContainerFactory). ScheduleImportService sastavljen kroz injekciju konstruktora iza interfejsa.
Sistem za prepoznavanje profesora	TeacherNameMatcher servis sa višestrukom strategijom približnog podudaranja (*fuzzy matching*: normalizacija latinice/ćirilice, uklanjanje prefiksa, podrška za alijase) radi povezivanja uvezenih imena sa kanonskim zapisima profesora.
Praćenje otkazanih časova	Tabela cancelled_classes i kompletan CRUD API za praćenje otkazanih i zamenjenih časova sa relacijama originalno/zamena.

Table 6.1.2.1: Pregled ključnih backend tehnologija i infrastrukture baze podataka.

Šema baze podataka obuhvata 12 tabela (admin_panel, students, study_programs, classrooms, schedule_enteries, schedule_imports, busy_classroms, schedule_entry_study_programss, caancelled_classes, teacher_aliases, email_templates) sa referencijalnim integritetom osiguranim preko stranih ključeva, kompozitnim indeksima za brze upite i normalizovanim dizajnom uz stratešku denormalizaciju za brže preuzimanje stavki rasporeda.

6.1.3 Dvostepena autentifikacija i autorizacija

Sistem autentifikacije primenjuje obrazac dvostepene verifikacije koji razdvaja serversku validaciju kredencijala od klijentskog usklađivanja uloga kroz četiri korisničke uloge (superadmin, admin, professor, student). Ova arhitektura osigurava da se korisničke uloge uvek deriviraju iz PHP sesije na strani servera putem api/check_auth.php i da se nikada ne veruje stanju na klijentu - uključujući i mogućnost manipulacije kroz React DevTools. AdminProtectedRoute gard dozvoljava uloge "admin" i "superadmin", dok ProtectedRoute proverava isLoggedIn stanje.

Tok rada za resetovanje lozinke primenjuje prakse preporučene od strane OWASP-a:

- **Kriptografski nasumični tokeni:** 256-bitna entropija generisana putem bin2hex(random_bytes(32)).
- **Vremenski ograničeno važenje:** Strogo definisan prozor važenja od 1 sata putem DATE_ADD(NOW(), INTERVAL 1 HOUR).
- **Jednokratna invalidacija:** Token se odmah briše nakon uspešne promene lozinke.
- **Enkriptovani transport:** STARTTLS SMTP enkripcija na portu 587 putem PHPMailer-a za svu odlaznu e-poštu.

6.1.4 Mitigacija ranjivosti usklađena sa OWASP standardima

Aplikacija rešava ključne kategorije iz OWASP Top 10 liste kroz usmerene kontrole višeslojne zaštite (*defense-in-depth*):

- **Sprečavanje SQL injekcija:** Univerzalna primena parametrizovanih upita korišćenjem PDO imenovanih parametara za autentifikaciju, MySQLi bind_param za CRUD i skripte za resetovanje, kao i mysqli_real_escape_string kao dodatni sloj zaštite u cevovodu za uvoz.
- **Mitigacija unakrsnog izvršavanja skripti (XSS):** Kontekstualno eskejpovanje izlaza putem automatskog JSX eskejpovanja u React 19 na *frontend*-u, dopunjeno serverskom sanitizacijom pomoću htmlentities() i htmlspecialchars() u PHP-u.

- **CSRF i CORS zaštita:** Dinamičko definisanje dozvoljenih domena (*allow-list*) sa restrikcijama za slanje kredencijala putem functions/cors.php (eksplicitna lista dozvoljenih izvora, bez džoker simbola kada su kredencijali omoguceni), HttpOnly kolačići sesije sa SameSite=Lax podešavanjem i eksplicitno uništavanje sesije pri odjavi.

6.2 Ograničenja Sistema

Iako EduScheduler ostvaruje značajna tehnička dostignuća, iskrena procena njegovih trenutnih ograničenja je od ključnog značaja za akademsku strogost i praktičnu primenjivost. Identifikovana su sledeća ograničenja:

- Hardkodovani SMTP kredencijali
- Razdvojena struktura direktorijuma bez jedinstvenog repozitorijuma
- Ručno upravljanje šemom baze podataka
- Hardkodovani kredencijali za bazu podataka
- Nepotpuna pokrivenost testovima
- Konfiguracija prilagođena isključivo lokalnom razvoju

6.2.1 Hardkodovani SMTP kredencijali

Funkcionalnost slanja e-pošte za resetovanje lozinke ima ugrađene SMTP kredencijale direktno u PHP izvornim fajlovima. U fajlu admin/send_password_reset.php (linija 57), lozinka za Gmail aplikativni nalog je sačuvana u čistom tekstu (*plaintext*):

```
// admin/send_password_reset.php:57
$mail->Password = 'password_example'; // promeni ovo
```

Adresa pošiljaoca i korisničko ime su takođe hardkodovani:

```
// admin/send_password_reset.php:56, 61
$mail->Username = example@gmail.com';
$mail->setFrom('tvojemail@gmail.com', 'EduScheduler Admin');
```

Ovo predstavlja kritični bezbednosni rizik: svako ko ima pristup izvornom kodu dobija potpunu kontrolu nad SMTP nalogom. Nijedan .env fajl ili konfiguracija sa promenljivim okruženja (*environment variables*) ne postoji u projektu (potvrđeno: nema .env fajlova ni na *frontend*-u ni na *backend*-u).

6.2.2 Razdvojena struktura direktorijuma bez jedinstvenog repozitorijuma

Projekat je podeljen na dve odvojene lokacije u fajl sistemu:

- **Frontend:** C:\Users\Shindiri\Desktop\Projekti\eduscheduler\
- **Backend:** C:\xampp\htdocs\EduScheduler\

Ovim direktorijumima se upravlja nezavisno, uz korišćenje odvojenih menadžera paketa (npm za *frontend* i Composer za *backend*). Ovakva razdvojenost donosi nekoliko praktičnih inženjerskih izazova:

1. **Kontrola verzija:** Git operacije moraju pokrivati dva odvojena repozitorijuma (ili ručno konfigurisanu *multi-remote* postavku), što povećava rizik od desinhronizacije verzija između *frontend*-a i *backend*-a.
2. **Produkciono podizanje (Deployment):** Aplikacija se ne može podići kao jedinstvena celina; artefakti izgradnje *frontend*-a moraju se ručno kopirati ili sinhronizovati sa Apache korenskim direktorijumom (*document root*).
3. **Koordinacija zavisnosti:** Izmene u API ugovorima (URL adrese ruta, strukture zahteva i odgovora) na *backend*-u moraju se ručno reflektovati u API sloju *frontend*-a (api.ts, adminApi.ts, publicApi.ts).

Jedinstvena struktura projekta sa jednim Git repozitorijumom i jasno definisanim cevovodom za izgradnju i podizanje aplikacije trenutno ne postoji.

6.2.3 Ručno upravljanje šemom baze podataka

Evolucija šeme baze podataka kontroliše se putem sirovih .sql migracionih fajlova koji se nalaze u migrations/ direktorijumu (ukupno 13 fajlova):

```
migrations/2026_add_student_program_year.sql
migrations/2026_schedule_entries_classroom_fk.sql
migrations/2026_schedule_entry_study_programs.sql
migrations/2026_teacher_aliases.sql
migrations/2026_cancelled_classes.sql
migrations/2026_cleanup_study_programs.sql
...
```

Ove skripte se izvršavaju ručno (npr. preko phpMyAdmin-a ili MySQL CLI interfejsa). Projekat ne koristi okvir za migracije kao što su Phinx, Doctrine Migrations ili Laravel Migrations. Ovaj pristup nosi sledeće nedostatke:

1. **Odsuštvo automatskog praćenja migracija:** Ne postoji migrations tabela niti ekvivalentan mehanizam koji bi beležio koje su migracije već primenjene, što otežava proveru trenutne verzije šeme.
2. **Nedostatak podrške za poništavanje (Rollback):** Sirovi SQL fajlovi nude samo izvršavanje unapred; ne postoje odgovarajuće down() metode za bezbedno vraćanje izmena.

3. **Odsuštvo upravljanja zavisnostima:** Migracioni fajlovi nemaju formalno definisan redosled izvršavanja niti graf zavisnosti; tačan redosled zavisi od ručne konvencije imenovanja (vremenski žigovi u nazivu fajla).
4. **Nedostatak garancije idempotentnosti:** Iako neki fajlovi sadrže IF NOT EXISTS zaštitu, celokupan proces migracije nema transakcioni i idempotentni izvršilac.

6.2.4 Hardkodovani kredencijali za bazu podataka

Parametri za povezivanje sa bazom podataka ponovljeni su na više ulaznih tačaka:

```
   // functions/db.php:4
   $con = mysqli_connect('localhost', 'root', '',
'eduschedulerdb');

   // login.php:24-27
   $host = 'localhost';
   $dbname = 'eduschedulerdb';
   $username = 'root';
   $password = '';

   // app/Database/Connection.php:23
   $this->link = @new mysqli('localhost', 'root', '',
'eduschedulerdb');
```

Ovi kredencijali su hardkodovani širom aplikacije, što znači da kompromitovanje bilo kog pojedinačnog fajla izlaže parametre za pristup bazi. Oslanjanje na root korisnika bez lozinke u lokalnom razvojnom okruženju dodatno narušava bezbednost, iako je to u skladu sa podrazumevanim XAMPP podešavanjima.

6.2.5 Nepotpuna pokrivenost testovima

Projekat uključuje PHPUnit jedinične testove za ključne *backend* servise — ukupno 9 test modula (TeacherNameMatcherTest, ScheduleValidatorTest, ScheduleImportRequestTest, ScheduleImporterTest, ScheduleFilenameParserTest, ScheduleExcelParserTest, ReservationAdminAccessTest, ManualScheduleGridTest, CellClassifierTest) — pri čemu je PHPUnit konfigurisan kao razvojna zavisnost u composer.json. Međutim:

1. **Odsuštvo testiranja frontend-a:** Ne postoje jedinični testovi komponenti (Jest, React Testing Library) niti testovi "sa kraja na kraj" (*end-to-end*: Cypress, Playwright) za React aplikaciju.
2. **Nedostatak E2E testiranja:** Ne postoje automatski testovi koji verifikuju kompletan tok - od prijave korisnika, preko uvoza Excel fajla, do prikazivanja rasporeda.

3. **Minimalno integraciono testiranje API-ja:** Postojeći *backend* testovi rade primarno na nivou pojedinačnih jedinica koda; ne postoje integracioni testovi koji proveravaju ponašanje HTTP ruta, statusne kodove ili izvršavanje zaštitnih slojeva kroz ceo životni ciklus zahteva.

6.2.6 Konfiguracija prilagođena isključivo lokalnom razvoju

Nekoliko konfiguracionih elemenata ostalo je prilagođeno lokalnom razvoju i zahteva izmene pre podizanja na produkciju:

1. **Hardkodovane adrese za preusmeravanje:** Tok za resetovanje lozinke preusmerava korisnike na http://localhost:5173 (Vite razvojni server) umesto na podesivi produkcioni domen.
2. **Permasivna CORS politika:** CORS lista dozvoljenih domena uključuje isključivo lokalne razvojne adrese (http://localhost:5173, http://localhost:3000) bez produkcionih domena.
3. **Detaljno izlaganje grešaka:** Unutrašnji detalji sistema izlažu se u HTTP odgovorima (npr. $mail->ErrorInfo u blokovima za obradu grešaka u PHPMailer-u).
4. **Nezaštićeno skladište za uvezene fajlove:** Direktorijum za uvoz (uploads/schedules/) nalazi se unutar korenskog veb direktorijuma i nema stroge politike kontrole pristupa.

6.3 Preporule za dalji razvoj aplikacije

Sledeće preporuke se predlažu radi rešavanja uočenih ograničenja i unapređenja ukupne robusnosti, skalabilnosti i održivosti sistema.

6.3.1 Konfiguracija okruženja pomoću vlucas/phpdotenv

Svi hardkodovani kredencijali (SMTP korisničko ime, SMTP lozinka, host baze podataka, naziv baze, korisnik i lozinka baze) treba da budu izmešteni u promenljive okruženja kojima se upravlja pomoću vlucas/phpdotenv biblioteke. Ovo bi zamenilo trenutne hardkodovane vrednosti u fajlovima functions/db.php, login.php, admin_login.php, app/Database/Connection.php i admin/send_password_reset.php pozivanjem $_ENV ili Dotenv instance:

```
// Proposed: functions/db.php with Dotenv
$dotenv = Dotenv\Dotenv::createImmutable(__DIR__ . '/..');
$dotenv->load();

$con = mysqli_connect(
    $_ENV['DB_HOST'],
    $_ENV['DB_USER'],
    $_ENV['DB_PASSWORD'],
    $_ENV['DB_NAME']
);
```

Fajl .env.example (sa zamenskim vrednostima) treba dodati u kontrolu verzija, dok stvarni .env fajl mora biti isključen putem .gitignore fajla. Ovaj pristup je u potpunosti usklađen sa *Twelve-Factor App* metodologijom za moderno upravljanje konfiguracijom.

6.3.2 Kontejnerizacija pomoću Docker-a

Trenutna zavisnost od ručno instaliranog XAMPP steka (Apache + MySQL + PHP na Windows-u) stvara nekonzistentnost okruženja i komplikuje podizanje sistema. Razvojno i produkciono okruženje zasnovano na Docker-u uz korišćenje docker-compose alata obezbedilo bi reproduktabilnost okruženja, konzistentnost na svim operativnim sistemima i pojednostavljeno pokretanje projekta.

U nastavku je prikazana predložena docker-compose.yml arhitektura za EduScheduler:

```
version: '3.8'

services:
  eduscheduler-backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    ports:
      - "8080:80"
    volumes:
      - ./backend:/var/www/html
    environment:
      - DB_HOST=db
      - DB_NAME=${DB_NAME}
      - DB_USER=${DB_USER}
      - DB_PASSWORD=${DB_PASSWORD}
    depends_on:
      - db

  db:
    image: mariadb:10.4
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
      MYSQL_DATABASE: ${DB_NAME}
      MYSQL_USER: ${DB_USER}
      MYSQL_PASSWORD: ${DB_PASSWORD}
    volumes:
      - db_data:/var/lib/mysql

  eduscheduler-frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    ports:
      - "5173:5173"
```

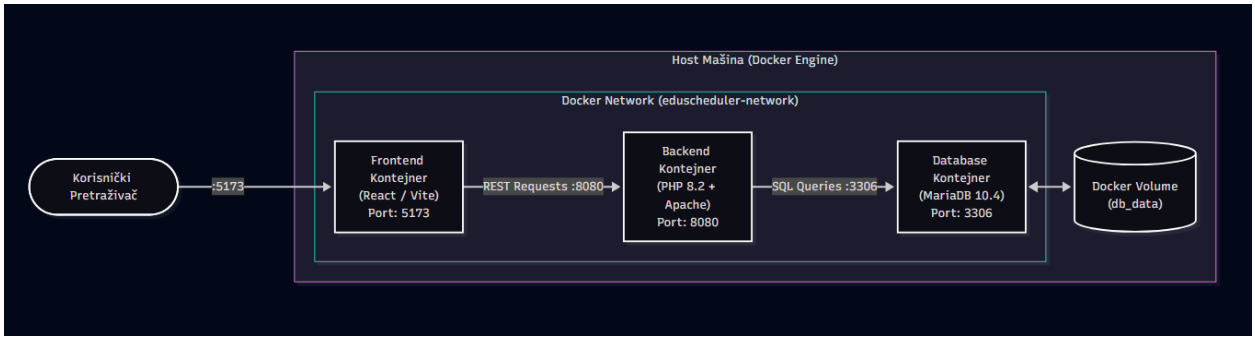
```
    volumes:
      - ./frontend/src:/app/src
    depends_on:
      - eduscheduler-backend

volumes:
  db_data:
```

Ova konfiguracija takođe olakšava integraciju sistema za kontinuiranu integraciju i kontinuirano podizanje (CI/CD) radi automatske izgradnje, testiranja i produkcionog podizanja aplikacije.

Radi postizanja visoke prenosivosti, lakšeg skaliranja i jednostavnog podešavanja produkcionog okruženja, predložena je kontejnerizacija sistema primenom Docker tehnologije.

Na Slici 6.3.2.1 prikazan je dijagram kontejnerizovane arhitekture sa tri izolovane servisne jedinice unutar zajedničke mrežne infrastrukture host mašine.



Slika 6.3.2.1: Struktura kontejnerizovane arhitekture sistema EduScheduler u Docker okruženju.

Kao što je prikazano na Slici 6.3.2.1, arhitektura je organizovana kroz sledeća tri servisa i njihove međusobne interakcije:

- **Frontend Kontejner (React / Vite):** Služi klijentsku aplikaciju na portu 5173. Predstavlja ulaznu tačku sistema sa kojom korisnički pretraživač ostvaruje direktnu komunikaciju.
- **Backend Kontejner (PHP 8.2 + Apache):** Izložen je na portu 8080 i prima REST zahteve prosleđene sa frontend-a. Unutar njega se izvršava kompletna aplikativna logika i priprema podataka.
- **Database Kontejner (MariaDB 10.4):** Posvećen je skladištenju podataka na portu 3306 i komunicira isključivo sa backend kontejnerom putem internog SQL protokola.

Sva tri servisa su povezana unutar privatne virutalne mreže (eduscheduler-network), dok je trajno čuvanje baze podataka obezbeđeno mapiranjem na eksterni Docker volumen (db_data), čime se sprečava gubitak podataka prilikom ponovnog pokretanja ili osvežavanja kontejnera.

6.3.3 Automatsko End-to-End testiranje pomoću Playwright i PHPUnit alata

Postojeći skup testova treba proširiti na oba sloja - i na *backend*-u i na *frontend*-u:

- **Proširenje backend-a (PHPUnit):**
  - *API integracioni testovi:* Potpuna pokrivenost testovima za svih 30 REST API ruta, uz verifikaciju HTTP statusnih kodova, šema odgovora i zaštitnih gardova.
  - *Testiranje poništavanja transakcija:* Automatski testovi koji proveravaju funkcionalnost vraćanja baze u prethodno stanje (*rollback*) radi garantovanja integriteta podataka nakon neuspešnih operacija.
  - *Stres testiranje cevovoda za uvoz:* Testovi performansi korišćenjem velikih Excel fajlova (1.000+ redova) radi potvrde linearne brzine obrade i stabilnosti memorije.
- **Frontend i E2E testiranje (Playwright):**
  - *Tokovi autentifikacije:* Automatski testovi koji pokrivaju prijavu za sve četiri uloge (superadmin, admin, professor, student), održavanje sesije, zaštitu ruta i odjavu.
  - *Interaktivna mreža rasporeda:* Verifikacija renderovanja mreže rasporeda, interakcije sa filterima (FloorSelector, filteri po profesorima/učionicama) i prikazivanja preklapanja časova.
  - *Tok uvoza Excel fajlova:* Test sa kraja na kraj koji verifikuje otpremanje Excel fajla, prikaz pregleda, prepoznavanje alijasa profesora i konačni upis u bazu. [15]
  - *Tok resetovanja lozinke:* Potpuna simulacija zahteva za resetovanje, generisanja tokena, provere sadržaja e-pošte i ažuriranja lozinke.

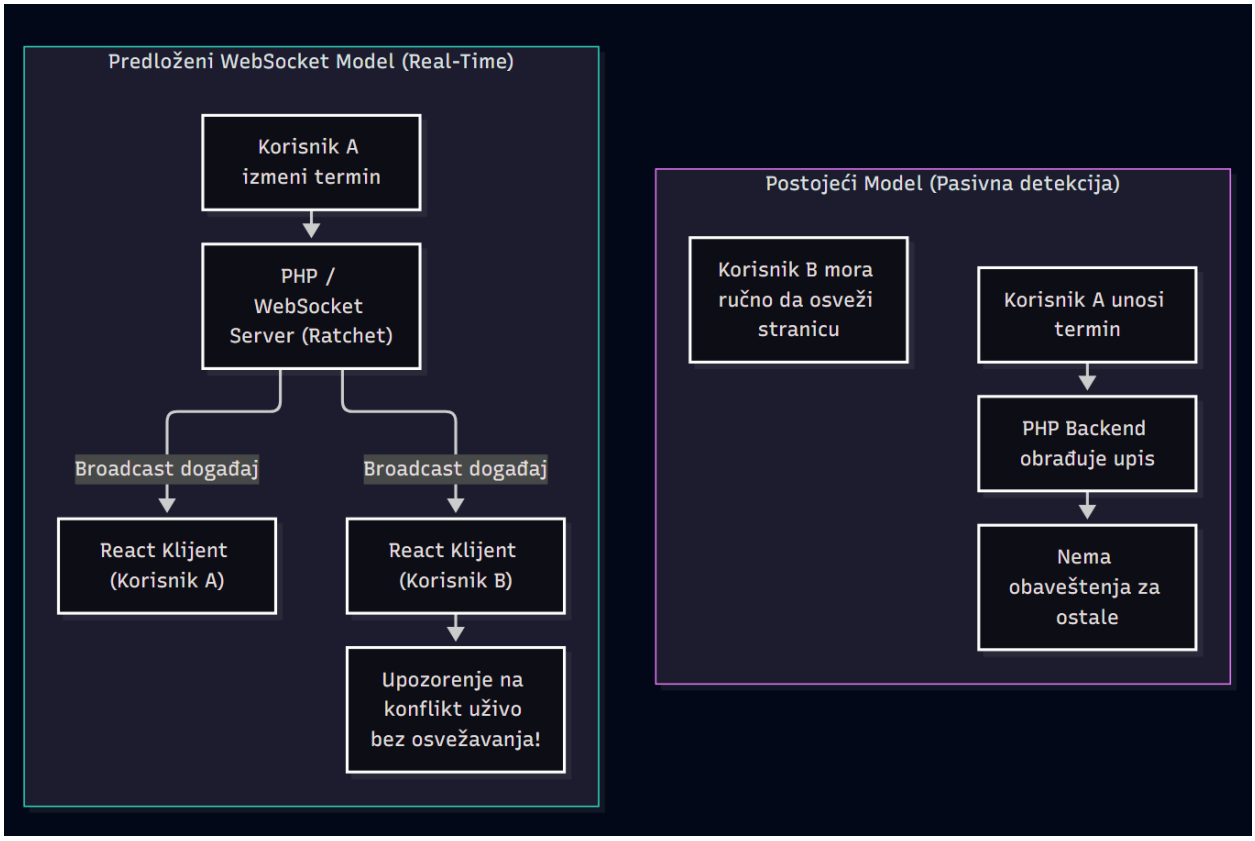
6.3.4 Detekcija konflikata u realnom vremenu putem WebSocket-a ili Server-Sent Events

Trenutni sistem detektuje konflikte u rasporedu samo tokom uvoza (ScheduleValidator::detectConflicts) i prilikom ručnog unosa (create_manual_entry.php). Ne postoji mehanizam obaveštavanja u realnom vremenu koji bi upozorio korisnike kada njihove izmene naprave konflikt sa postojećim terminima ili kada drugi korisnik izmeni zauzetost zajedničke učionice.

Uvođenje WebSocket-a (korišćenjem Ratchet ili ReactPHP biblioteka za PHP) ili Server-Sent Events (SSE) sloja omogućilo bi obaveštenja o konfliktima u realnom vremenu, uživo osvežavanje rasporeda i kolaborativno rezervisanje učionica bez potrebe za ručnim osvežavanjem stranice.

Radi unapređenja korisničkog iskustva i eliminisanja potencijalnih prekrajanja rasporeda pri istovremenom radu više administratora, u okviru smernica za budući razvoj predlaže se prelazak sa pasivnog HTTP request-response modela na reaktivnu WebSocket komunikaciju.

Na Slici 6.3.4.1 prikazana je uporedna analiza postojećeg mehanizma osvežavanja i predloženog sistema za detekciju konflikata u realnom vremenu.



Slika 6.3.4.1: Uporedni prikaz postojećeg HTTP modela i predloženog WebSocket sistema za sinhronizaciju rasporeda u realnom vremenu.

Kao što je ilustrovano na Slici 6.3.4.1, komparacija dva modela obuhvata sledeće funkcionalne razlike:

- **Postojeći model (Pasivna detekcija):** Prilikom unosa ili izmene termina od strane jednog korisnika, PHP backend vrši obradu i upis u bazu podataka bez emitovanja obaveštenja ostalim aktivnim sesijama. Da bi drugi korisnik uočio promene ili izbegao dupliranje resursa, neophodno je da ručno osveži stranicu u pretraživaču.
- **Predloženi WebSocket model (Real-Time):** Kada Korisnik A izmeni termin u rasporedu, zahtev se prosleđuje namenskom PHP WebSocket serveru (npr. zasnovanom na *Ratchet* biblioteci). Server trenutno emituje (*broadcast*) događaj svim povezanim React klijentima. React aplikacija Korisnika B prihvata događaj u pozadini i odmah prikazuje upozorenje o potencijalnom konfliktu u realnom vremenu - bez ikakve potrebe za manuelnim osvežavanjem stranice.

6.3.5 Integracija SSO autentifikacije putem Microsoft Teams

Kao jedno od ključnih funkcionalnih unapređenja u narednim verzijama sistema EduScheduler, poželjno je implementirati jedinstvenu prijavu (*Single Sign-On* – SSO) putem Microsoft 365 / Teams naloga korisnika. S obzirom na to da visokoškolske ustanove primarno koriste Microsoft platformu za organizaciju nastave i komunikaciju, integracija zasnovana na **OAuth 2.0** i **OpenID Connect** protokolima (preko *Microsoft Graph API*-ja) omogućila bi nastavnom osoblju i studentima prijavu na sistem pomoću postojećih institucionalnih akreditacija. Ovakav pristup značajno unapređuje korisničko iskustvo, eliminiše potrebu za ručnim upravljanjem lozinkama u lokalnoj bazi i podiže nivo bezbednosti celokupnog sistema kroz centralizovanu autentifikaciju.

6.3.6 Unapređenje bezbednosti za produkciono okruženje

Dodatna unapređenja bezbednosti i operativnog rada za produkciono okruženje obuhvataju:

1. **Obavezni HTTPS:** Strogo nametanje TLS 1.3 enkripcije na svim rutama uz HTTP Strict Transport Security (HSTS) zaglavlja.
2. **Ograničavanje broja zahteva (Rate Limiting):** Primenjivanje mehanizma za ograničavanje broja zahteva na rutama za autentifikaciju (login.php, admin_login.php, send_password_reset.php) radi zaštite od napada grubom silom (*brute-force*).
3. **Content Security Policy (CSP):** Konfigurisanje strogih HTTP zaglavlja (Content-Security-Policy, X-Frame-Options: DENY, X-Content-Type-Options: nosniff) radi ograničavanja izvora sa kojih se resursi mogu učitavati.
4. **Strukturirano logovanje i monitoring:** Zamena direktnog ispisa grešaka strukturiranim logovanjem (Monolog) usmerenim ka centralizovanim sistemima za praćenje grešaka.
5. **Pool konekcija ka bazi:** Prelazak sa kreiranja konekcije po svakom zahtevu na deljene konekcije (*connection pooling*) ili persistentne konekcije radi lakšeg podnošenja velikog broja istovremenih korisnika.

7. Zaključak

U ovom master radu predstavljen je EduScheduler, savremena *full-stack* web aplikacija razvijena sa ciljem modernizacije i centralizacije procesa upravljanja obrazovnim rasporedom nastave. Zamenom manuelnih rutina i radnih listova sklonih greškama integrisanom platformom zasnovanom na React 19, TypeScript, PHP 8.2 i MariaDB tehnologijama, projekat demonstrira na koji način savremene web tehnologije mogu efikasno rešiti dugogodišnja operativna uska grla u akademskoj administraciji.

Tokom implementacije sistema, ključni inženjerski izazovi rešavani su sistematski i sa jasnim fokusom na performanse i bezbednost. Razvoj robusnog cevovoda za obradu podataka na serverskoj strani omogućio je uvoz i parsiranje zatečenih Excel rasporeda za manje od pola sekunde. Sa strane korisničkog interfejsa, algoritam optimizacije dinamičke mreže rasporeda - kroz konsolidaciju spojenih ćelija i memorizaciju prikaza - smanjio je broj aktivnih DOM čvorova za oko 35 procenata, čime je obezbeđeno glatko i odzivno renderovanje obimnih univerzitetskih rasporeda. Istovremeno, primena dvostepene autentifikacije i zaštitnih mehanizama u skladu sa OWASP standardima osigurala je pouzdan i bezbedan višeslojni pristup sistemu za administratore, profesore i studente.

Iako je evaluacija platforme ukazala na određena operativna ograničenja - poput jedno-nitnog parsiranja Excel datoteka, manuelnog upravljanja šemom baze i definisanih konfiguracija u razvojnom okruženju - uspostavljene su jasne smernice za budući arhitektonski razvoj. Integracija menadžera promenljivih okruženja, Docker kontejnerizacija infrastrukture, proširenje E2E testova i implementacija WebSocket protokola za sinhronizaciju konflikata u realnom vremenu uzdići će EduScheduler na nivo kompleksnog softverskog rešenja za visokoškolske ustanove. Naposletku, EduScheduler u potpunosti ostvaruje svoj primarni cilj pruženjem pristupačne, visokoperformantne i skalabilne platforme koja uspešno spaja administrativne zahteve planiranja sa potrebama studenata.

LITERATURA

[1] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley Professional, 2002.

[2] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994.

[3] React Documentation, "React v19 Release Notes," React Dev, 2024. [Online]. Available: https://react.dev/blog/2024/12/05/react-19. [Accessed: Jul. 24, 2026].

[4] B. Bierman, A. Lucco, and C. Hejlsberg, "Understanding TypeScript," *ECOOP 2014 – Object-Oriented Programming*, Lecture Notes in Computer Science, vol. 8586, Berlin, Heidelberg: Springer, pp. 257–281, 2014.

[5] PHP Documentation Group, "PHP 8.2 Release Notes," PHP: Hypertext Preprocessor, 2022. [Online]. Available: https://www.php.net/releases/8.2/en.php. [Accessed: Avg. 5, 2026].

[6] MariaDB Foundation, "MariaDB Server 10.4 Overview," MariaDB Knowledge Base, 2019. [Online]. Available: https://mariadb.com/kb/en/changes-improvements-in-mariadb-104/. [Accessed: Avg. 24, 2026].

[7] OWASP Foundation, "OWASP Top 10:2021 - The Ten Most Critical Web Application Security Risks," OWASP, 2021. [Online]. Available: https://owasp.org/Top10/. [Accessed: Avg. 24, 2026].

[8] D. B. Chapman and E. Zwicky, *Building Internet Firewalls*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2000.

[9] W3C, "Cross-Origin Resource Sharing (CORS) W3C Recommendation," World Wide Web Consortium, 2020. [Online]. Available: https://www.w3.org/TR/cors/. [Accessed: Sep. 03, 2026].

[10] T. Tanner and P. M. Rossi, "Modern Frontend Architecture with Vite and Module Bundling," *IEEE Software*, vol. 41, no. 3, pp. 45–52, May–June 2024.

[11] TanStack, "TanStack Query v5 Documentation," TanStack, 2023. [Online]. Available: https://tanstack.com/query/v5. [Accessed: Sep. 05, 2026].

[12] Colm, "PhpSpreadsheet Documentation," PHPOffice, 2023. [Online]. Available: https://phpspreadsheet.readthedocs.io/. [Accessed: Sep. 10, 2026].

[13] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA, USA: Addison-Wesley Professional, 2003.

[14] IEEE Standard for Software and System Test Documentation, *IEEE Std 829-2008*, IEEE, 2008.

[15] Microsoft, "Playwright: Fast and reliable end-to-end testing for modern web apps," Microsoft Open Source, 2024. [Online]. Available: https://playwright.dev/. [Accessed: Sep. 10, 2026].

BIOGRAFIJA

Miloš Petrović rođen je 1. oktobra 1999. godine u Knjaževcu, Republika Srbija. Osnovno obrazovanje završio je u Knjaževcu, nakon čega je školovanje nastavio u Medicinskoj školi u Zaječaru, koju je završio 2018. godine.

Tokom osnovnog i srednjeg obrazovanja učestvovao je na brojnim takmičenjima i ostvario značajne rezultate, za šta je nagrađivan diplomama, sertifikatima i priznanjima. Pored formalnog obrazovanja, razvijao je interesovanje prema računarskim tehnologijama i sticao dodatna znanja i veštine putem specijalizovanih obuka. U Regionalnom centru za profesionalni razvoj u Knjaževcu stekao je više sertifikata iz oblasti informacionih tehnologija, uključujući obuke za korišćenje Microsoft Office paketa (Word, Excel i PowerPoint), obradu slika u programu Adobe Photoshop, kao i korišćenje interneta. Takođe, uspešno je završio obuku iz uvodnog objektno-orijentisanog programiranja u programskom jeziku Python i kurs slepog kucanja.

Školske 2020/2021. godine upisao je Akademiju tehničko-vaspitačkih strukovnih studija u Nišu, odsek Savremene računarske tehnologije. Tokom osnovnih studija stekao je obimna znanja iz različitih oblasti informacionih tehnologija, uključujući programiranje, baze podataka, razvoj softvera, računarske sisteme i web tehnologije. Pored redovnih nastavnih aktivnosti, učestvovao je u različitim projektima i praktičnim aktivnostima, što mu je omogućilo primenu teorijskog znanja na rešavanje realnih inženjerskih problema.

Tokom studija učestvovao je i u sekciji „Upravljanje bespilotnim letelicama“, za šta je dobio odgovarajući sertifikat. Ova aktivnost mu je omogućila da dodatno unapredi tehnička znanja i stekne praktično iskustvo u radu sa savremenim tehnologijama.

Kroz studije i praktičan rad razvio je posebno interesovanje za razvoj softvera i web aplikacija. Praktično iskustvo u struci dodatno je izradio tokom angažovanja u kompaniji Shindiri Studio, gde je radio na razvoju modernih web rešenja i savremenih korisničkih interfejsa. Kroz rad u industriji i akademske projekte stekao je bogato iskustvo u radu sa tehnologijama kao što su JavaScript, TypeScript, React, SolidJS, PHP, Python, HTML i CSS, kao i sa relacionim bazama podataka, REST API servisima i sistemima za kontrolu verzija. U okviru svog profesionalnog usavršavanja stekao je i iskustvo sa tehnologijama koje se koriste za razvoj aplikacija za Smart TV uređaje, kao što su LightningJS i Blits, čime je dodatno proširio svoja znanja u oblasti razvoja korisničkih aplikacija.

Završne godine studija usmerio je ka razvoju informacionih sistema koji mogu doprineti unapređenju organizacije i efikasnosti u obrazovnim ustanovama. Kao rezultat tog interesovanja proistekao je projekat EduScheduler - web aplikacija namenjena organizaciji i upravljanju rasporedom nastave i korišćenjem učioničkog prostora. Razvoj ovog sistema pružio mu je priliku da objedini i primeni znanja stečena tokom studija u oblastima programiranja, razvoja web aplikacija, upravljanja bazama podataka i projektovanja informacionih sistema.

Obrazovanje je nastavio upisom na master strukovne studije na studijskom programu Informacione tehnologije i sistemi na Akademiji tehničko-vaspitačkih strukovnih studija u Nišu. Tokom master studija nastavio je sa usavršavanjem znanja i veština u oblasti razvoja softvera i informacionih sistema, sa posebnim naglaskom na praktičnu primenu savremenih tehnologija u razvoju kompleksnih web aplikacija.

Njegova profesionalna interesovanja usmerena su ka softverskom inženjerstvu i razvoju aplikacija, sa ciljem kontinuiranog usavršavanja tehničkih znanja, sticanja praktičnog iskustva i praćenja brzog razvoja savremenih tehnologija.

Kontakt i profesionalni profili:

• LinkedIn: https://www.linkedin.com/in/milos-petrovic-programmer/

• GitHub: https://github.com/MilosPetrovic-programmer

• E-pošta: milos.petrovic.programer@gmail.com